

面向协作边缘计算场景的 LLM 推理优化：算法研究与应用综述

Collaborative Edge Computing for LLM Inference: Algorithms and Applications

研究综述报告

协作边缘计算赋能的大模型推理优化

版本： v2.0

日期： 2026 年 8 月

摘要

大语言模型 (LLM) 推理正从云端集中式服务逐步向网络边缘迁移, 以满足低时延、高带宽效率及数据隐私等需求。然而, 单个边缘节点 (如基站、边缘服务器、边缘网关) 受限于计算、存储和通信资源, 难以独立承载完整的大模型部署。协作边缘计算 (CEC) 通过将地理分布、能力异构的终端设备、边缘服务器及 AI 加速板卡整合为统一的协作资源池, 为突破这一瓶颈提供了可行路径。在 CEC 框架下, 模型切分、请求调度、键值 (KV) 缓存与上下文管理、服务路径选择等关键环节均可跨节点联合优化。然而, 边缘场景下算力异构, 资源受限, 动态网络条件和请求, 以及用户移动性等特性给 LLM 推理系统带来了新的研究挑战。

本文围绕 CEC 场景下的大语言模型推理优化展开综述。首先, 本文梳理 LLM inference 的基本执行机制、问题空间和“优化对象-执行范围”分类框架, 并分析 CEC 相较云侧集中式 serving 新增的资源、网络、状态和移动性约束。在此基础上, 本文重点讨论三类算法问题: **资源自适应模型切分部署、业务感知 batching, 以及用户移动场景下的请求卸载与状态管理**。相关工作部分覆盖 layer、tensor、expert、prefill 阶段和 decoer 阶段分离等切分方式; 动态组批与 chunked prefill 等 batching 调度机制; 以及 request/session routing、context/KV/runtime state migration 和 multi-agent workflow orchestration 等请求调度方法。面向验证环境, 报告进一步给出三项具体方案: 以统一代价模型比较 TP、PP 和 PP+TP, 并采用动态规划求解非均匀 PP 分割; 以 CMDP 建模并采用带长期风险约束和安全投影的 Safe-RL 调节 batching 策略, 以确保不违反 SLO 约束; 联合选择请求执行节点和 KV 获取方式, 优化移动会话的长期服务成本。最后, 本文讨论三类算法的协同关系及其在 Ascend 和 vLLM-Ascend 基础设施上的实现与评估方法。

目录

一、引言	5
1.1 研究背景与应用动机	5
1.2 CEC 场景下边缘 AI 推理的关键挑战	5
1.3 相关研究基础与问题主线	6
1.4 研究范围与内容组织	7
二、LLM 推理基础、问题空间与优化分类	9
2.1 LLM 推理的基本执行机制	9
2.2 LLM 推理的问题空间：从执行机制到系统瓶颈	12
2.3 LLM 推理优化的“优化对象-执行范围”分类	13
2.4 主流 LLM 推理框架及优化方式	18
2.5 小结	19
三、CEC 场景下的大模型推理	20
3.1 CEC LLM 推理的系统形态	20
3.2 CEC 相较云侧集中式 LLM Serving 新增的约束	21
3.3 各类 LLM Inference 优化在 CEC 中的关注重点	21
3.4 本文采用的 CEC LLM Inference 综述框架	22
3.5 小结	23
四、CEC 下的资源自适应模型切分部署方法	25
4.1 Layer-wise / Vertical Partitioning	28
4.2 Tensor / Operator-level Partitioning	30
4.3 Expert / Component-level Partitioning	32
4.4 Inference Phase Partitioning	33
4.5 Semantic / Token-level Partitioning	35
4.6 面向华为验证环境的方案建议	36
4.7 小结	51
五、CEC 下的业务感知 Batching 参数优化方法	53
5.1 Queue-level Dynamic / Admission Batching	56

5.2 Continuous Batching / Iteration-level Batching	57
5.3 Chunked Prefill / Blended Batching	59
5.4 Disaggregated Prefill-Decode Batching	61
5.5 面向华为验证环境的方案建议	62
5.6 小结	75
六、CEC 中的 LLM Inference Task Offloading	77
6.1 Request / Session-level Routing and Offloading	78
6.2 Multi-model / Multi-agent Workflow Orchestration	80
6.3 Context / KV / Runtime State Offloading and Migration	81
6.4 面向华为验证环境的方案建议	83
6.5 小结	101
七、三类算法的协同优化	103
7.1 三类算法的基本耦合关系	105
7.2 Model / Service Placement – Request Offloading 协同	106
7.3 Model Partitioning – Batching / Scheduling 协同	106
7.4 Request / Workflow Routing – Runtime State Management 协同	107
7.5 Mobility-aware Hierarchical Control	108
7.6 小结	109
八、面向 Ascend 验证环境的 CEC 推理工程实现方案	110
8.1 工程定位	110
8.2 总体架构	110
8.3 分阶段实现路线	111
8.4 关键工程模块	113
8.5 验证路线	114
8.6 项目进度与交付安排	114
8.7 小结	115
九、挑战与未来方向	116
9.1 异构资源与性能建模	116
9.2 状态迁移与一致性	116
9.3 在线策略稳定性	117

9.4 隐私与合规	117
9.5 未来研究方向	117
十、总结	119
参考文献	120

一、引言

1.1 研究背景与应用动机

大语言模型推理的部署正在从云端集中式服务扩展到多个边缘节点参与的协作式服务。现有优化研究主要覆盖两类环境：端侧推理通过模型压缩、量化、小模型和端侧 NPU/GPU 加速，使部分请求能够在用户设备上完成；云端或数据中心推理服务则通过高性能算子、连续批处理、KV Cache 管理、模型并行和多实例调度提高吞吐并降低尾时延。已有系统与综述分别从执行运行时、内存管理、批处理调度和分布式服务等角度总结了这些机制 [1], [2], [3], [4], [5], [6], [7], [8]。

两类环境各有适用边界。端侧推理具有低时延和数据本地性优势，但受算力、内存、功耗和散热限制，难以长期承载复杂大模型和高并发服务。云侧推理资源充足、框架成熟，但网络传输会影响端到端时延、带宽消耗、隐私边界和服务可用性。与此同时，基站、网关、园区和近用户位置已经部署了大量分散、异构且动态变化的边缘算力；这些资源规模大于单个端侧设备，又缺少数据中心同构集群的稳定互联与统一规格。

本文所讨论的 CEC (Collaborative Edge Computing, 协作边缘计算) 将地理分布、能力异构的终端设备、边缘服务器和 AI 板卡组织为协作资源池，并借助 Wi-Fi 接入点、路侧单元、5G 基站和网关等基础设施交换请求、中间结果与运行时状态。如图 1 所示，多个边缘资源可以共同完成模型部署和推理执行，请求路径不依赖预设的端、边、云三级结构。终端可以作为请求入口或计算节点，CEC 不是为了替代中心化的云，而是作为补充，外部云服务则在业务允许时作为可选服务或异常回退路径。分布式协同智能研究为异构资源、链路和服务位置的联合优化提供了参考 [9]；本文聚焦协作边缘资源池内部的模型切分、请求调度和状态管理。相应的系统问题是：如何在资源与网络持续变化的条件下，以可控成本满足业务时延、吞吐、服务连续性和数据隐私要求。

1.2 CEC 场景下边缘 LLM 推理的关键挑战

然而，CEC 场景给 LLM 推理带来了新的挑战，具体如下

CEC 场景下的边缘 AI 推理首先受到资源异构和碎片化的约束。终端设备以及部署在基站、网关、园区和边缘服务器中的 GPU、NPU 与 AI 板卡，在算力、内存、显存、功耗预算和加速库支持上差异显著，模型部署不能直接套用数据中心中“节点规格相近、互联稳定、控制面统一”的假设。同一个模型在不同硬件上可能面临完全不同的显存瓶颈、算子支持和通信代价，部署策略也必须随资源状态动态调整。

LLM 推理本身还具有显著的状态和内存压力。Prefill 阶段会为后续 decode 生成 KV Cache，decode 阶段又会随着输出长度不断追加状态；在长上下文、多轮会话和高并发场景中，KV Cache、prefix

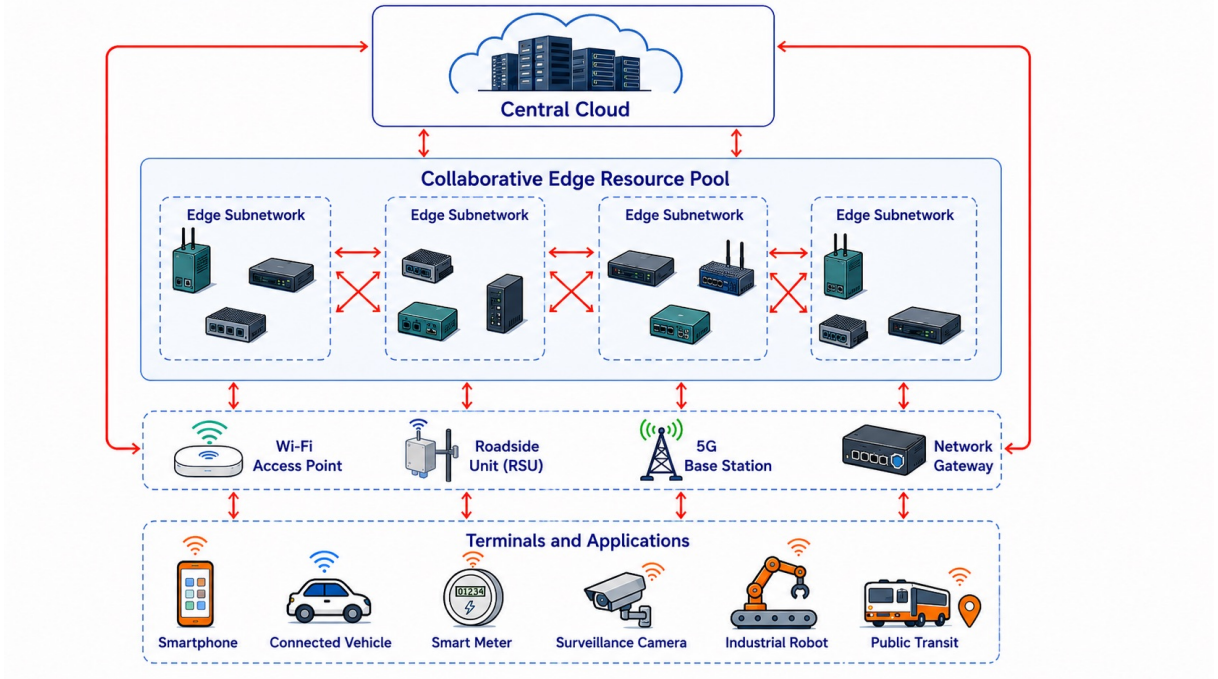


图1 协作边缘计算示意图。协作边缘计算汇聚泛在、地理分布设备的计算资源以共同执行计算任务，从而扩展可用资源池，实现低时延数据处理、灵活设备接入与更广的服务覆盖。

cache 和 session state 会长期占用显存，并要求系统管理其生命周期、位置和一致性。一旦这些状态跨节点访问、迁移或重算，通信成本、服务连续性和隐私边界都会同时受到影响。

请求动态性进一步放大了调度难度。用户并发数、prompt 长度、输出长度、请求复杂度和时延 SLO 都在持续变化，边缘节点又常常缺乏云侧的大规模请求池，静态 batching、固定 batch size 和固定服务路径难以稳定满足业务要求。对于移动用户，接入点和合适的服务实例还会随位置变化；距离最近的节点未必具有最短的排队时间或端到端时延。若用户会话中的 KV 或上下文状态仍保留在原服务节点，服务切换还可能引发状态迁移、远程访问或重新 prefill。

这些因素相互影响：模型切分会改变各节点的计算量和跨节点 activation 传输，batching 会改变排队时间、KV 占用和设备利用率，请求卸载又会改变流量分布与状态位置。因而，CEC LLM inference 需要联合比较计算收益、通信开销和状态迁移成本，并在业务时延、隐私和服务连续性约束下选择可执行的部署与调度方案。

1.3 相关研究基础与问题主线

现有研究为 CEC LLM inference 提供了多方面基础。一方面，on-device inference 关注模型压缩、低比特执行、小模型设计和端侧硬件加速，使部分请求能够在用户设备本地完成；FlexGen [10]、Petals [11]、HexGen [12] 等工作则进一步展示了资源受限或异构环境下的大模型推理可以通过分层存储、协作式 block 放置和异构并行来突破单节点容量限制。另一方面，cloud/datacenter LLM serving 已经围绕 continuous batching、PagedAttention、chunked prefill、prefill/decode disaggregation、KV cache manage-

ment 和多实例调度形成了较成熟的系统路线, Orca [1]、vLLM [2]、Sarathi-Serve [3]、DistServe [4]、SGLang [5] 和 Mooncake [13] 等系统分别代表了不同层面的关键机制。

边缘计算领域长期研究任务卸载、资源分配、移动性管理和网络感知调度, 其中许多卸载模型将任务描述为具有固定输入大小、计算量、deadline 和能耗约束的作业。LLM 推理的服务时间还取决于 prompt、输出长度、decode 进度、batch 组成和 KV 状态, 复杂请求也可能涉及多模型、多智能体或多阶段执行。因此, CEC LLM 推理同时包含 LLM serving 中的 prefill/decode、KV Cache、batching、推测解码和分布式执行问题, 以及协作边缘环境中的资源异构、动态网络、用户移动、隐私约束和服务连续性问题。

基于这一交叉特征, 本文将相关工作组织为三条主线。第一条是 **CEC 下的资源自适应模型部署方法**, 关注模型结构、模型组件和推理阶段如何在异构协作节点之间切分与放置。第四章的综述覆盖 layer、tensor、expert、推理阶段和生成职责等不同切分对象, 面向验证环境的具体方案则先比较 TP、PP 和 PP+TP, 再采用动态规划求解非均匀 PP 的连续 layer 分割。第二条是 **CEC 中的业务感知 batching 参数优化算法**, 关注请求进入推理实例后如何根据 SLO、队列状态、KV 占用和 prefill/decode 干扰选择 batch 形成与更新策略, 具体方案采用 Safe-RL 联合调节 batch 参数、请求顺序和运行时机制。第三条是 **CEC 中的 LLM inference task offloading**, 关注完整请求、会话、multi-agent workflow 以及 context/KV/runtime state 如何在协作边缘节点之间路由、卸载、迁移和复用。这三条主线分别对应模型部署、实例内调度和跨节点服务路径选择。

1.4 研究范围与内容组织

本文聚焦 **CEC 场景下的大语言模型推理优化问题**。研究范围限定在推理阶段, 不讨论大模型训练、预训练或微调过程; 重点关注终端设备、边缘服务器、AI 板卡以及部署在基站、网关和园区中的推理节点之间的大模型推理协同, 包括资源自适应模型切分与部署、动态 batching、prefill/decode 调度、分布式 KV/状态管理、任务卸载、请求路由、移动性感知调度和服务连续性保障。

底层算子优化、硬件实现和模型结构设计仅在影响 CEC 推理系统时纳入讨论。例如, 低比特算子会影响模型能否部署到目标设备, KV Cache 布局会影响状态迁移和缓存复用, 便于切分的模型结构会影响跨异构节点执行的可行性。本文从系统角度分析这些因素对部署、调度、状态管理和策略的影响。

本文首先从 prefill/decode、KV Cache、batching/scheduling、decoding runtime 和主要服务指标出发, 梳理 LLM inference 的基本执行机制, 并解释模型规模、KV 状态、请求动态性、decode 串行性和多实例协同带来的系统瓶颈。在此基础上, 本文构建“优化对象-执行范围”二维分类, 将方法按主要优化对象划分为 Model、Kernel、Runtime 和 Serving Policy & Routing 四层, 并区分单实例或本地控制域内的优化与跨设备、跨节点的分布式优化。

随后，本文分析 CEC 环境如何改变 LLM inference 的部署与调度条件，并围绕资源自适应模型部署、业务感知 batching 和 LLM inference task offloading 三条主线梳理代表工作。在相关工作分析之后，第四章给出基于动态规划的非均匀 PP 方案，第五章给出基于 CMDP 的 Safe-RL batching 方案，第六章给出联合选择执行节点与 KV 获取方式的长期会话感知卸载方案。第七至第十章进一步讨论三类算法的协同关系、Ascend/vLLM-ascend 实现路线、实验设计及后续研究问题。

二、LLM 推理基础、问题空间与优化分类

2.1 LLM 推理的基本执行机制

本文关注的 LLM 推理以在线生成服务为主：系统接收输入上下文，并以自回归方式逐 token 生成输出。与输入输出形状较固定的传统 DNN 推理相比，LLM 请求的输入长度、输出长度和到达时间持续变化，decode 还依赖已经生成的 token 和 KV Cache。随着上下文与生成长度增加，KV Cache 也会增长，因此系统需要同时控制吞吐、时延、显存占用和服务质量。

图 2 给出了典型服务流程。上层 serving policy 负责请求准入、路由、排队和执行顺序；batching/scheduling 模块根据服务目标、请求优先级、长度、实例负载和 KV 容量，调整等待窗口、token 上限、prefill/decode 资源配额以及当前参与执行的请求集合。该集合既可包含尚未处理完输入上下文的 prefill 请求，也可包含正在逐 token 生成的 decode 请求。底层 inference runtime 完成模型计算和 KV 管理，并把 TTFT、TPOT、ITL、队列长度和 KV 使用量反馈给控制模块，用于下一轮调度。

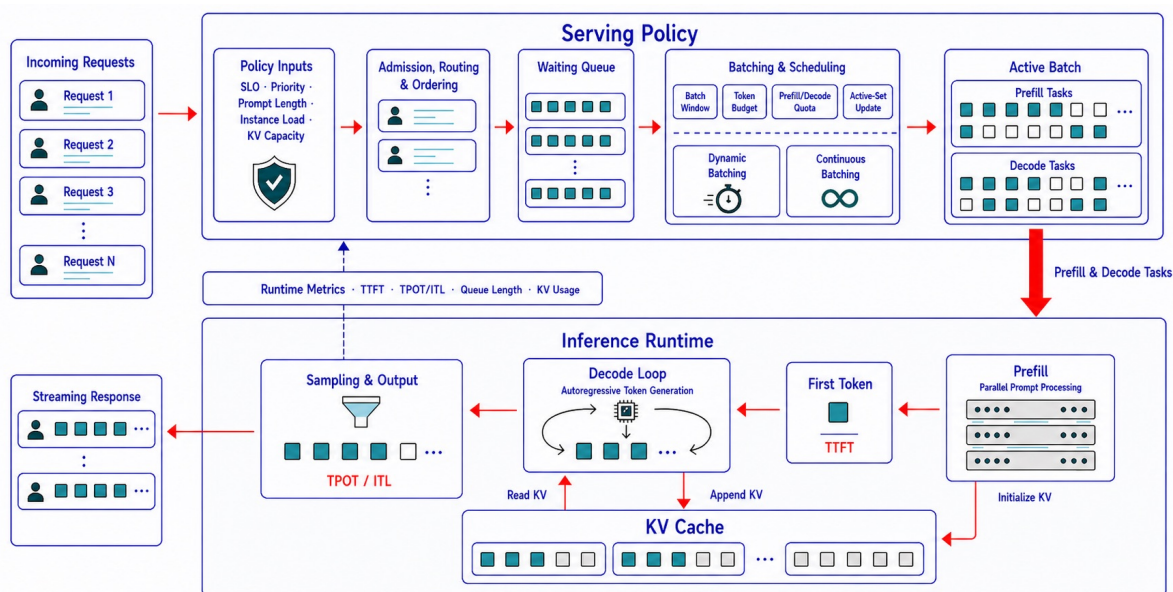


图 2 LLM serving 基础服务流程。Serving policy 作为上层控制面，内部包含准入、路由、排序、请求队列和 batching/scheduling，并根据业务与运行状态形成同时包含 prefill tasks 与 decode tasks 的活动批次；inference runtime 分别执行 Prefill、KV Cache 管理和逐 token Decode，最终形成流式响应并将运行指标反馈给 serving policy。

2.1.1 Prefill 与 Decode

当今主流的大模型是基于 decoder 架构的。对于 decoder-only 自回归大模型，一次请求通常分为 **Prefill** 和 **Decode** 两个阶段。

Prefill 阶段负责处理完整输入上下文，包括用户 prompt、历史对话、系统提示词和检索增强内容等。模型对整段上下文并行计算 attention 和 FFN，为每一层生成后续 decode 复用的初始 KV Cache，

并根据最后位置的输出分布产生首个输出 token。该阶段矩阵规模较大、并行度较高，通常更受计算能力限制，主要影响 TTFT，即 Time To First Token。

Decode 阶段负责逐 token 生成输出。每一步 decode 输入上一步生成的新 token，同时读取历史 KV Cache，计算当前 token 的 attention 和 FFN，并将新增 KV 写回缓存。该阶段每步计算粒度较小、状态依赖强、访存频繁，通常更受显存带宽和单步时延限制，主要影响 TPOT、ITL 和持续生成吞吐。

Prefill 和 Decode 的资源特征差异直接影响 LLM serving 的系统设计。Prefill 以大规模矩阵计算为主，Decode 则由高频、小粒度并且依赖历史状态的迭代组成。因此，推理系统通常分别考虑两个阶段的 batching、调度、KV 管理和分布式执行方式。

2.1.2 KV Cache

KV Cache 是 LLM decode 阶段最核心的运行时状态。自回归生成中，第 t 个 token 的 attention 需要访问前面所有 token 的 key/value。如果每一步都重新计算历史 token 的 key/value，会产生大量重复计算。KV Cache 的作用是在 prefill 阶段保存历史 token 的 key/value，在 decode 阶段复用这些历史 KV，只为新 token 计算新增 KV。

KV Cache 降低了 decode 阶段的重复计算，但也引入了显著的内存状态问题。KV 占用会随 batch size、上下文长度、输出长度、模型层数和 KV head 数增长。在长上下文、多轮对话和高并发场景中，KV Cache 往往成为显存容量和显存带宽的重要瓶颈。vLLM 的 PagedAttention [2]、vAttention [14] 的动态 KV 内存管理和 Mooncake [13] 的 KVCache-centric serving，分别从分页分配、动态地址映射和分布式 KV 存储等角度降低状态管理开销；相关综述进一步覆盖缓存淘汰、卸载、量化和持久化等方向 [6], [8]。

典型 KV 相关优化如表 1 所示：

表 1 典型 KV Cache 相关优化及作用

方向	作用
PagedAttention / memory paging	降低 KV 分配碎片，提高显存利用率
Prefix cache / RadixAttention	复用共享前缀，减少重复 prefill
KV compression / quantization	降低 KV 存储和传输成本
KV eviction	在显存不足时淘汰低价值 KV
KV offloading	将部分 KV 放到 CPU、本地存储、远端内存或其他节点
Distributed KV cache	在多 worker / 多节点场景下管理 KV 状态分布

2.1.3 Batching 与 Scheduling

Batching 是 LLM serving 提高硬件利用率和吞吐量的主要机制。单个请求在 decode 阶段的每步计算粒度较小，只服务少量请求时加速器容易出现计算单元空闲。Batching 通过合并多个请求扩大矩阵运算规模，提高 GPU/NPU 的并行执行效率，并分摊算子启动和调度等固定开销。

LLM batching 与传统 DNN batching 的执行条件不同。传统推理服务中的动态批处理器通常在请求执行前按等待窗口和 batch size 形成批次 [15]；LLM 请求的输入长度、到达时间和输出长度差异较大，decode 请求会逐步完成并释放 batch slot，长 prompt 的 prefill 还可能阻塞短请求的 decode。因此，现代 LLM serving 系统通常结合 dynamic batching、continuous batching、iteration-level scheduling 和 chunked prefill。Orca [1] 代表迭代级调度路线，Sarathi-Serve [3] 代表 chunked prefill 路线；近期综述也将 batching/scheduling 与算子优化、内存管理并列为 LLM 推理系统的主要组成部分 [6], [7], [8]。

典型调度机制如表 2 所示：

表 2 典型 LLM Serving 调度机制及作用

机制	作用
Dynamic batching	根据请求到达情况动态组成 batch
Continuous batching	在每个 decode iteration 动态加入新请求，移除完成请求
Iteration-level scheduling	以 token iteration 为粒度调度请求
Chunked prefill	将长 prompt prefill 拆成多个 chunk，降低对 decode 的阻塞
Priority / fairness scheduling	根据 SLA、优先级和公平性调整请求顺序
Admission control	在资源紧张时控制进入系统的请求量

Batching 需要根据负载在 throughput、TTFT、TPOT、ITL、显存占用和 SLA 达标率之间动态选择合适的 batch 规模与执行顺序。

2.1.4 主要性能指标

LLM inference 通常同时关注表 3 中的指标：

表 3 LLM 推理主要性能指标

指标	含义
TTFT	Time To First Token，从请求到达首个输出 token 返回的时间
TPOT	Time Per Output Token，首 token 之后每个输出 token 的平均生成时间
ITL	Inter-token Latency，相邻两个输出 token 的时间间隔，通常同时报告均值和尾部分位数
E2E latency	从请求到完整响应结束的端到端时延
Throughput	单位时间处理的 request 数或 token 数
Goodput	单位时间内满足指定 SLA/SLO 的请求数或 token 数
GPU/NPU utilization	加速器计算单元利用率
Memory footprint	权重、activation、KV Cache 和运行时空间形成的当前或峰值内存占用
Cost per token	单位 token 的计算、显存和服务成本

本文将具体的时延、吞吐和可靠性阈值称为服务目标（SLO），将业务约定的一组服务要求称为 SLA。后文讨论“满足 SLA 的请求处理速度”时，表示只有各项指定 SLO 均达标的请求或运行点才计入结果。

2.2 LLM 推理的问题空间：从执行机制到系统瓶颈

LLM 推理的系统瓶颈与其执行机制密切相关。大规模模型参数以及 attention、FFN 计算带来较高的算力和显存开销。Prefill 并行处理完整输入上下文，Decode 则受自回归依赖限制逐 token 生成，两个阶段对计算、显存带宽和时延的要求不同。KV Cache 随上下文长度和并发请求持续增长，又会增加显存容量、访存带宽和状态管理压力。请求到达时间、输入长度和输出长度的变化进一步增加了 batching 与 scheduling 的复杂度；推理扩展到多 GPU、多实例或多节点后，还会引入通信、状态传输和服务编排开销。基于这些问题，本节从五个方面分析主要系统瓶颈。

2.2.1 计算复杂度与模型规模问题

模型规模首先决定权重的常驻显存需求和每次前向计算的基本开销；长上下文还会增加 attention 计算与中间状态存储。相关优化需要同时考虑参数表示、模型结构和算子执行效率。

Attention 与 FFN 构成主要矩阵计算，长上下文还会推高 attention 计算和 KV 存储压力。MoE 等条件计算结构可以降低单 token 激活参数量，同时会引入 expert routing、expert placement 和 all-to-all 通信等系统问题。常见优化包括量化、剪枝、蒸馏和低秩分解，以及 GQA/MQA/MLA、高效 attention、MoE 执行优化和低比特算子等方法。

2.2.2 KV Cache 与内存状态问题

KV Cache 将 decode 从重复计算历史上下文变为增量计算当前 token，但代价是引入持续增长的运行时状态。上下文越长、batch 越大、并发越高，KV Cache 越容易成为显存容量和带宽瓶颈。

KV Cache 是随请求生命周期动态增长和释放的运行时状态，这使其管理方式不同于静态模型权重。KV Cache 的显存占用会随上下文长度和并发度增长，动态分配释放又容易产生碎片；在多轮对话和共享 prompt 场景中，不同请求可能重复执行相同前缀的 prefill；在跨 worker 或跨节点 serving 中，KV 的传输、迁移、复用和远程访问会进入端到端时延关键路径。PagedAttention、prefix cache、KV compression、KV quantization、KV eviction、KV offloading、distributed KV cache 和 remote KV serving 等方法，分别用于降低状态成本、提高状态复用或控制状态移动。

2.2.3 请求动态性与调度问题

LLM 请求具有高度动态性：prompt 长度不同、输出长度未知、请求到达时间随机、服务质量目标不同。Prefill 和 decode 的资源特征也不同，长 prefill 可能阻塞短 decode，请求完成时间差异又会导致 batch 内部动态变化。

调度问题的核心是吞吐和时延之间的动态权衡。等待凑 batch 可以提高硬件利用率，但会增加

TTFT；过大的 batch 会推高单请求延迟和显存占用；长 prompt prefill 可能阻塞已经进入流式输出的 decode 请求；不同租户、优先级和 SLA 又会使简单 FIFO 策略失效。因此，现代 LLM serving 通常采用 dynamic batching、continuous batching、iteration-level scheduling、chunked prefill、deadline-aware scheduling、priority scheduling 和 admission control 等机制，在 batch 形成、active set 更新和请求优先级之间做在线决策。

2.2.4 Decode 串行性与生成加速问题

LLM decode 是自回归过程，每一步依赖前一步输出，天然存在串行依赖。即使单步 forward 被加速，逐 token 生成仍可能造成较高 TPOT 和 ITL。

这一串行依赖存在于不同规模的自回归模型中，但单步开销仍会随模型规模、上下文长度和 batch 状态变化。单步 decode 的计算粒度较小，硬件利用率往往不如 prefill；sampling、beam search 或复杂生成策略还会增加控制开销。Speculative decoding、assisted decoding、parallel decoding、tree decoding 和 multi-token prediction 等方法，试图通过草稿生成、批量验证或一次预测多个 token 来减少目标模型的串行调用次数，但仍需权衡草稿接受率、额外计算、输出质量和系统复杂度。

2.2.5 多实例、多 GPU 与服务级编排问题

当模型规模、并发量或服务可用性需求超过单设备能力时，系统需要跨多个 GPU、worker、replica 或节点协同。此时问题从单个 engine 内部执行扩展为分布式执行和服务级调度。

分布式化带来的收益与代价同时存在。Tensor、pipeline、expert 或 context parallel 可以突破单设备容量和吞吐限制，但会引入 activation、KV 和中间状态的跨设备传输；prefill 与 decode 的资源特征差异使 disaggregation 成为可能，也使 KV handoff 成为新的关键路径；多 replica 和多模型服务可以提高可用性和弹性，却需要在负载均衡、质量、时延和成本之间做服务级选择。因此，distributed parallelism、prefill/decode disaggregation、distributed KV cache、KV transfer、replica routing、cluster load balancing、cascade serving 和 speculative serving orchestration 等方法，本质上都在处理“如何将单实例推理扩展成可控的分布式服务”这一问题。

2.3 LLM 推理优化的“优化对象-执行范围”分类

围绕 LLM inference 相关优化方向分类的 overview 和 survey 工作较多，表 4 汇总了其中代表性工作的分类方式及其特点。

基于这些工作，本报告采用“优化对象-执行范围”二维分类。第一维按照方法直接改变的对象，将其分为 Model、Kernel、Runtime 和 Serving Policy & Routing 四层；第二维按照执行范围，区分单

表 4 现有 LLM Inference Overview 与 Survey 的分类方式

工作	分类视角	主要分类	分类特点
<i>A Survey on Efficient Inference for Large Language Models</i> [6]	优化层次	Data-Level、Model-Level、System-Level	覆盖从输入数据、模型结构到系统实现的多层优化，适合从整体上理解推理效率优化
<i>A Survey of LLM Inference Systems</i> [8]	系统流水线与系统形态	Request processing、model optimization and execution、memory management，以及 single-replica、multi-replica、disaggregated、serverless systems	对 system-level 方法拆分较细，突出 kernel、batching/scheduling、memory management 与不同部署形态
<i>Taming the Titans: A Survey of Efficient LLM Inference Serving</i> [7]	Serving 层级与场景	Instance-Level Approaches、Cluster-Level Strategies、Emerging Scenarios、Miscellaneous Areas	强调从单实例优化扩展到集群部署、负载均衡和云服务等 serving 问题

实例或本地控制域内的 Local 优化，以及需要跨设备、worker、replica 或节点协同的 Distributed 优化。

图 3 展示了两条分类轴，后续 2.3.1–2.3.4 分别说明四类优化对象及其本地和分布式实现。

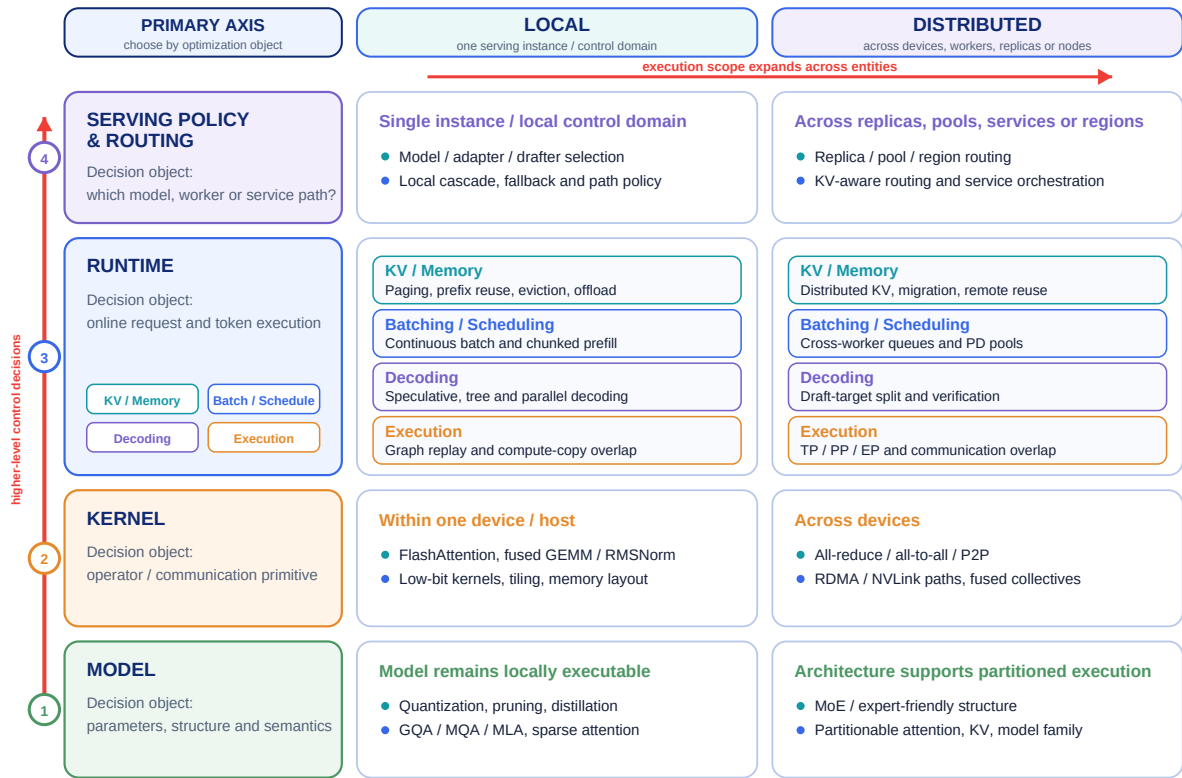


图 3 LLM 推理优化的“优化对象–执行范围”二维分类。纵轴按照主要优化对象区分 Model、Kernel、Runtime 与 Serving Policy & Routing 四个层级；横轴按照执行范围区分 Local 与 Distributed，表示方法从单实例或本地控制域扩展到跨设备、worker、replica、service 或 node 的协同。Runtime 层进一步包含 KV/Memory、Batching/Scheduling、Decoding 和 Execution 四类在线机制。

其中，Local 表示优化限制在单个 serving instance 或本地控制域内，可以包含 CPU 与 GPU/NPU、本地内存和存储之间的协同；Distributed 表示方法需要跨多个设备、worker、replica 或节点分配计算、请求、KV 状态或 activation。四类方法并不互斥，真实系统经常同时使用多层机制。因此，归类时主要判断三个问题：方法解决什么问题，直接改变的对象是什么，以及是否需要跨设备或跨节点协同。

2.3.1 Model-Level Optimization

Model-Level Optimization 直接改变模型参数、参数表示、结构设计或算法语义，以降低推理计算和存储开销。此类修改不依赖某个特定 serving engine，其优化对象包括参数量、权重表示、FLOPs、activation 计算量、attention 复杂度、KV 大小以及条件计算路径。

Local scope: 常见方法包括权重量化、剪枝、蒸馏、低秩分解、权重共享、GQA/MQA/MLA、线性或稀疏 attention、状态空间模型、early exit、token pruning 和动态稀疏等，主要用于单实例或单设备上的模型压缩和结构简化。

Distributed scope: 主要关注模型结构对跨设备执行的支持，例如适合 expert parallelism 的 MoE 结构、适合 tensor/context parallel 的 attention 或 KV 结构，以及便于按 layer、expert、context 或 activation 切分的模型设计。

CUDA Graph、算子融合和 TensorRT execution graph 等方法主要改变执行路径或部署图，分别归入 Runtime-Level 或 Kernel-Level；它们不会改变模型参数和数学语义。

2.3.2 Kernel-Level Optimization

Kernel-Level Optimization 在保持模型数学语义不变的条件下，优化张量算子或通信原语在硬件上的执行效率。主要对象包括 attention、GEMM、normalization、sampling 和 collective communication，目标是提高 GPU/NPU 利用率、减少 HBM 访问、增强片上缓存复用，并改善算子吞吐。

Local scope: 典型方法包括 FlashAttention、FlashDecoding 和 paged attention kernel。其他方法还包括 MLP、RMSNorm 与 softmax 融合，低比特 GEMM，FP8、INT8 与 INT4 算子，硬件感知 tiling、内存布局优化和 GPU-side sampling。

Distributed scope: 主要包括集合通信和点对点通信原语的实现优化，例如 all-reduce、all-gather、reduce-scatter、all-to-all、RDMA/NVLink 传输路径、通信融合、集合通信算法选择，以及 MoE expert dispatch/combine 原语。

需要区分的是，如果优化对象是 collective / P2P primitive 的实现路径，归为 Kernel-Level × Distributed；如果优化对象是通信与计算在 runtime pipeline 中的编排、重叠和调度，则归为 Runtime-Level × Distributed。

2.3.3 Runtime-Level Optimization

Runtime-Level Optimization 是现代 LLM serving 的主要系统优化层。它发生在请求进入 inference engine 之后，管理在线 token 执行、KV 状态、batch 组成、decode 循环和执行流水线。Runtime-Level 不改变模型语义，也不一定修改单个 kernel，而是决定请求到达后如何组织执行。本文将进一步分

为 KV / Memory、Batching / Scheduling、Decoding 和 Execution 四类运行时机制。

1. **KV / Memory Runtime.** KV / Memory Runtime 关注 KV lifecycle、memory state 和 cache reuse。它的核心问题是：KV Cache 如何存储、复用、压缩、淘汰、迁移或远程访问。

Local scope: 主要包括 KV cache management、PagedAttention / memory paging、Prefix cache / RadixAttention、KV compression / quantization、KV eviction、KV offloading、cache persistence 和 runtime memory allocator 等机制。

Distributed scope: 问题进一步扩展为 distributed KV cache、KV transfer / migration、remote KV serving、multi-replica KV reuse、distributed prefix cache、KV-aware placement 和 cross-worker state synchronization。

这一类优化的核心目标是降低 KV 显存占用、减少重复 prefill、提升 cache reuse，并在分布式场景下控制 KV 传输和状态迁移成本。

2. **Batching / Scheduling Runtime.** Batching / Scheduling Runtime 关注请求和 token 在 inference engine 内如何组织执行。它回答的是：哪些请求何时执行，哪些 token 可以组成 batch，prefill 和 decode 如何交错，如何在 throughput、TTFT、TPOT、ITL、公平性和 SLA 之间做权衡。

Local scope: 典型机制包括 dynamic batching、continuous batching、iteration-level scheduling、chunked prefill、prefill/decode interleaving、engine-local admission control、priority / fair scheduling 和 deadline-aware scheduling。

Distributed scope: 调度对象会跨越 worker、replica 或资源池，因此需要 cross-worker scheduling、distributed continuous batching、prefill worker / decode worker scheduling、heterogeneous GPU scheduling、multi-replica queue coordination、distributed admission control 和 SLO-aware worker scheduling。这一类优化的核心目标是在动态请求负载下提高硬件利用率，同时控制排队时延和服务质量。

3. **Decoding Runtime.** Decoding Runtime 关注在线 token generation procedure。它回答的是：输出 token 如何生成、验证、选择和接受。它不主要决定请求如何组 batch，也不主要决定计算如何 launch，而是改变 decode loop 的生成逻辑。

Local scope: 这类方法包括 speculative decoding、assisted decoding、parallel decoding、tree decoding、beam search optimization、n-gram speculation、prompt lookup decoding 和 serving-time multi-token prediction / verification。

Distributed scope: Decoding Runtime 会涉及 distributed verification、draft / target execution split、multi-draft execution、batched verification across workers、draft model 与 target model 分布式协同，以及 speculative decoding 与 prefill/decode disaggregation 的结合。

Speculative decoding 有时也被称为 decoding algorithm。本文将其主要归入 Runtime-Level，因为它改变的是服务阶段的 token 生成与验证过程，而不是原模型参数或主体结构。如果某种 multi-

token prediction 方法新增训练 head 或修改模型结构，则还应同时标注其 Model-Level 属性。

4. **Execution Runtime.** Execution Runtime 关注已经排定的计算如何提交、同步、重放和重叠执行。当前面的运行时已经决定请求集合、batch 组成和 decode 方式后，Execution Runtime 负责把算子、数据搬运和通信提交到硬件，并组织为流水线。

Local scope: 主要包括 CUDA Graph / HIP Graph replay、async execution、stream scheduling、CPU-GPU overlap、host-device communication hiding、compute / memory-copy overlap、runtime metadata / buffer reuse 和 GPU-side sampling execution path。

Distributed scope: 主要包括 tensor parallel runtime、pipeline parallel runtime、expert parallel runtime、context / sequence parallel runtime、prefill/decode disaggregation、distributed execution pipeline、communication-computation overlap scheduling 以及 activation / KV transfer orchestration。

Execution Runtime 与 Kernel-Level 的边界在于：Kernel-Level 优化单个 operator 或 communication primitive 的实现；Execution Runtime 优化一组 operator、copy 和 communication 的 launch、同步、overlap 和流水化组织方式。

2.3.4 Serving Policy & Routing-Level Optimization

Serving Policy & Routing-Level Optimization 关注服务级控制决策。它不直接决定单个 engine 内部的 token 如何执行，而是选择请求使用哪个模型、adapter、replica、worker pool 或服务路径，并处理回退、级联和多服务编排。典型决策包括请求分配、模型或 adapter 选择、实例选择、服务路径选择，以及面向 SLO、成本和时延的路由策略。

Local scope: 主要包括 local model selection、adapter / LoRA routing、local cascade serving、normal decoding 与 speculative decoding path selection、drafter selection、local/cloud fallback、budget-aware local inference 和 latency-aware local policy。

Distributed scope: 进一步扩展为 replica routing、worker-pool routing、cluster load balancing、SLO-aware routing、cost-aware routing、region-aware routing、KV-aware routing、speculative serving orchestration、draft-target worker orchestration 和 multi-service graph orchestration。

Runtime-Level 和 Serving Policy & Routing-Level 在真实系统中相互依赖，但主要决策对象不同。前者解决“请求进入某个 engine 后如何执行”，后者解决“请求交给哪个模型、worker、replica 或服务路径”。服务级决策通常需要读取队列长度、KV Cache 压力、batch 状态和请求距离 SLO 阈值的剩余时间。

2.4 主流 LLM 推理框架及优化方式

LLM 推理框架把模型加载、算子执行、KV Cache 管理、请求调度和服务接口组合成可部署的推理系统。vLLM 以 PagedAttention 和 continuous batching 为基础，重点提高通用 GPU 服务器上的显存利用率与在线服务吞吐；TensorRT-LLM 面向 NVIDIA GPU，通过专用 kernel、低精度计算、in-flight batching 和多 GPU 并行提供高度优化的执行路径；SGLang 从结构化生成程序出发，将 RadixAttention、请求调度和分布式执行整合到同一运行时；LMDeploy/TurboMind 面向开源模型部署，提供模型转换与量化、persistent batching、blocked KV Cache 和多 GPU 执行等能力 [2], [5], [16], [17]。表 5 按主要优化对象和执行范围汇总这些框架的代表性机制；该表不覆盖各框架的全部功能。

表 5 主流 LLM Inference Framework 代表性机制映射

Framework	代表性方法 / 机制	解决的具体问题	优化方向分类层 & Scope	具体方法说明
vLLM [2]	PagedAttention	KV Cache 动态分配和显存碎片问题	Runtime / KV-Memory $\times$ Local	通过分页式 KV 管理降低显存碎片，提高 KV Cache 利用率
	Continuous batching	Decode 阶段请求动态进出，静态 batch 利用率低	Runtime / Batching-Scheduling $\times$ Local	在 decode iteration 动态加入和移除请求，提高高并发吞吐
	Chunked prefill	长 prompt prefill 阻塞 decode，影响 TTFT / ITL	Runtime / Batching-Scheduling $\times$ Local	将长 prefill 拆分为 chunk，与 decode 交错执行
	Prefix caching	多请求共享前缀导致重复 prefill	Runtime / KV-Memory $\times$ Local/Distributed	复用共享 prompt 或 prefix，减少重复 prefill 计算
TensorRT-LLM [16]	In-flight batching	高并发请求长度不同，传统 batching 效率低	Runtime / Batching-Scheduling $\times$ Local	对不同阶段和长度的请求进行动态批处理
	Paged KV Cache	KV Cache 分配、复用和显存管理问题	Runtime / KV-Memory $\times$ Local	通过 paged KV 管理改善 KV Cache 利用率
	FP8 / INT8 / INT4 execution	权重、activation 和算子执行成本高	Model + Kernel $\times$ Local/Distributed	通过低比特模型表示和低比特 kernel 降低显存和计算成本
	Overlap scheduler	compute、copy、communication、prefill/decode 没有充分重叠	Runtime / Execution $\times$ Local/Distributed	将计算、通信和数据搬运流水线化，减少等待
SGLang [5]	RadixAttention	多轮、多分支请求存在大量共享前缀	Runtime / KV-Memory $\times$ Local	用 radix tree 组织 prefix cache，提高共享前缀复用
	Zero-overhead CPU scheduler	高并发场景下 CPU 调度开销影响吞吐	Runtime / Batching-Scheduling $\times$ Local	降低 CPU 侧调度和 metadata 管理开销
	Prefill-decode disaggregation	Prefill 和 decode 资源特征不同，混部互相干扰	Runtime / Execution $\times$ Distributed	将 prefill 与 decode 分离到不同资源池，减少资源干扰
	Multi-LoRA batching	多 adapter 请求混合服务时 batch composition 复杂、吞吐下降	Serving Policy & Routing + Runtime/Scheduling $\times$ Local/Distributed	将不同 LoRA / adapter 请求合并调度，提高多 LoRA serving 效率
Hugging Face TGI [18]	Continuous batching	incoming requests 动态变化，传统 batching 吞吐不足	Runtime / Batching-Scheduling $\times$ Local	动态合并请求，提高在线 serving 吞吐
	Flash Attention	attention HBM traffic 高，operator throughput 受限	Kernel $\times$ Local	通过高效 attention kernel 降低显存访问并提升算子吞吐
	Paged Attention	KV Cache / attention memory 管理效率不足	Runtime / KV-Memory $\times$ Local	通过 paged attention 改善 attention/KV memory 管理
	Tensor parallelism	单 GPU 容量或吞吐不足，需要跨 GPU 执行	Runtime / Execution $\times$ Distributed	将模型执行切分到多个 GPU，提高模型容量和吞吐
LMDeploy / TurboMind [17]	Persistent batching	高并发请求动态调度，提高吞吐	Runtime / Batching-Scheduling $\times$ Local	通过持续批处理机制维持 batch 执行效率
	Blocked KV Cache	KV Cache 显存管理和访问效率问题	Runtime / KV-Memory $\times$ Local	通过 block-based KV 管理改善显存利用率
	Dynamic split & fuse	长 prompt 和 decode 混合执行效率低	Runtime / Batching-Scheduling $\times$ Local	动态拆分和融合 prefill / decode 任务，提高混合负载效率
	Tensor parallelism	大模型需要跨 GPU 执行	Runtime / Execution $\times$ Distributed	支持模型在多 GPU 上并行执行

从主要部署对象看，上述框架目前仍以数据中心或服务器场景为主。其核心能力围绕单机多 GPU、多机多 GPU、高并发请求、稳定高速互联和统一加速器软件栈展开。具有兼容 GPU 或 NPU 的边缘服务器可以复用其中部分执行引擎，但这些框架本身通常不负责处理移动终端的功耗限制、端侧内存约束、间歇性链路、跨边缘节点状态放置和用户移动等问题。

边缘侧较有代表性的 LLM 推理框架包括 llama.cpp [19] 和 MLC LLM [20]。llama.cpp 使用轻量 C/C++ 运行时和 GGUF 模型格式，支持多种低比特量化、CPU/GPU 混合执行以及 Android 和 Apple 平台；MLC LLM 通过机器学习编译生成面向不同硬件后端的模型库，并以同一引擎支持 Web、iOS、Android、Python 和 REST 部署。这类框架的重点是让量化后的模型在单台手机、PC 或嵌入式设备上运行，其请求并发、跨节点 KV Cache 管理和全局调度能力仍弱于数据中心推理框架。

除单设备运行时外，已有部分研究围绕模型切分、异构资源调度和跨节点通信等问题，对多节点协同推理开展了方法设计与实验探索。总体来看，边缘侧单设备推理已具备较为成熟的软件实现，而多节点协同推理仍以面向特定问题的研究方案为主，尚未形成能够统一处理资源变化、链路波动、推理状态管理和用户移动的动态 CEC 服务框架。

2.5 小结

本章首先介绍了 LLM inference 的基本执行机制，包括 prefill/decode、KV Cache、batching/scheduling 和主要性能指标。随后，从执行机制出发，总结了 LLM inference 的主要问题空间，包括计算复杂度与模型规模、KV Cache 与内存状态、请求动态性与调度、decode 串行性与生成加速，以及多实例、多 GPU 与服务级编排问题。

在已有 overview 和 survey 工作的基础上，本章采用“优化对象-执行范围”二维分类：按照方法直接改变的对象区分 Model、Kernel、Runtime、Serving Policy & Routing，并按照是否跨设备或节点区分 Local 与 Distributed。Runtime 层进一步细分为 KV / Memory、Batching / Scheduling、Decoding 和 Execution 四类机制。该分类可同时说明一个方法改动了什么，以及它在单实例内执行还是需要分布式协同。

最后，本章通过主流推理框架的代表性机制映射说明，实际系统通常会组合模型表示、kernel、KV 管理、batching/scheduling、执行运行时和服务策略等多层机制。现有通用服务框架主要面向数据中心，边缘侧单设备运行时已经具备工程可用性，而多节点协同服务仍以研究原型为主。该分类框架将用于后续分析 CEC 场景下的模型部署、请求调度、KV 状态迁移和服务路径选择。

三、CEC 场景下的大模型推理

3.1 CEC LLM 推理的系统形态

CEC 场景下的大模型推理运行在由多个地理分布、能力异构的边缘节点组成的协作系统中 [9]。协作节点可以是具备推理能力的终端设备、边缘服务器和 AI 板卡，也可以是部署在基站、网关或园区中的推理实例；请求可从终端、基站或边缘网关等任一入口进入。模型权重、adapter（参数高效微调模块）、KV Cache、prefix cache、session state 和中间状态可以集中在单个节点，也可以分布在多个协作节点上。一次推理既可以在本地实例内完成，也可以通过节点间的计算与状态传输协同完成；外部云服务仅在具体业务允许时作为可选服务或回退路径。

与云侧集中式 LLM serving 相比，CEC 的主要变化是执行范围从单个实例或集中式集群扩展到分布式边缘资源池。云侧 serving 通常具有相对集中的资源和稳定互联，优化重点是单实例或集群内部的吞吐、显存和调度效率。CEC 还需要把节点位置、无线与边缘网络、异构硬件、移动接入、状态分布和隐私边界纳入推理路径。推理系统需要同时确定执行实例、跨节点模型部署、状态保留位置、节点间传输方式和服务路径调整条件。图 4 展示了模型分割部署、计算任务调度和 KV/runtime state 调度在协作边缘节点之间的关系。

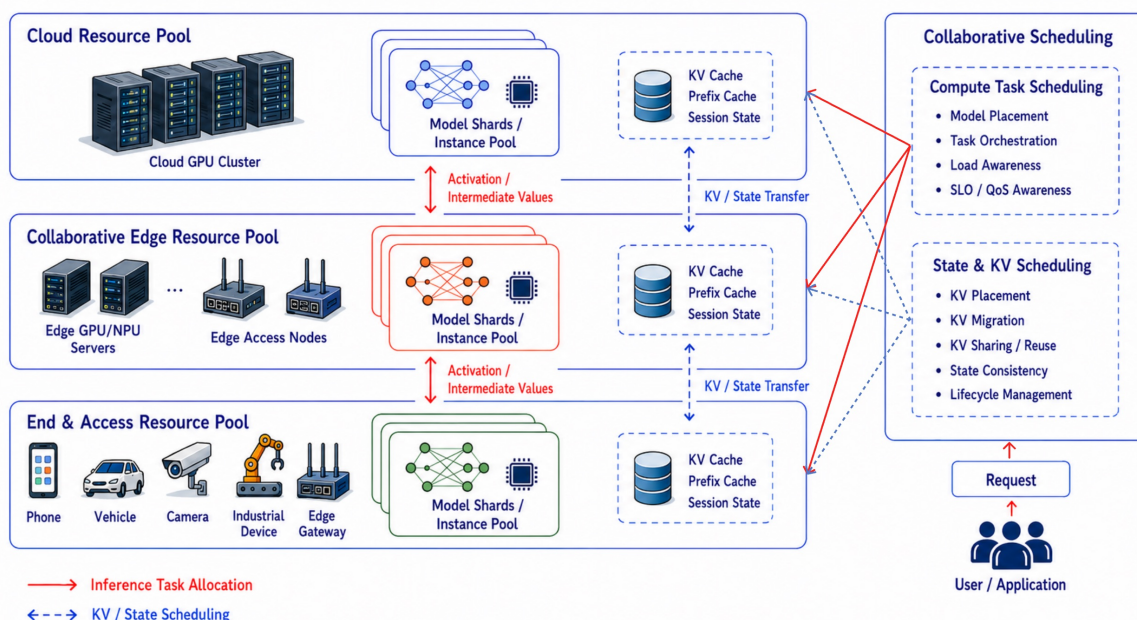


图 4 CEC 环境下的 LLM 推理 Serving 系统形态。Serving Policy 根据协作节点的资源、链路与运行状态，协调模型分割与部署、Prefill/Decode 等计算任务调度，以及 KV Cache、Prefix Cache 和 Session State 的复用、迁移与远程访问。

CEC LLM inference 主要包含三类相互关联的决策。模型部署决定模型的哪些部分由哪些协作节点执行；实例内调度决定请求如何组成 batch，以及 prefill 和 decode 如何安排；服务路径选择决定请

求、会话和运行时状态在协作节点之间如何路由、迁移或复用。后续章节分别围绕这三类问题展开。

3.2 CEC 相较云侧集中式 LLM Serving 新增的约束

资源受限与硬件异构：模型规模、KV Cache、batching、并行执行和多实例调度在云侧同样存在，CEC 则进一步面临单节点资源受限和设备能力差异。协作资源池中的终端设备、边缘服务器和 AI 板卡在算力、显存、内存、功耗及散热预算上差异明显，同一个模型未必能完整放入目标设备；即便容量足够，也可能受到低比特算子、算子覆盖范围、推理后端和数据布局转换的限制。因此，部署方案必须同时考虑设备容量、实际执行能力和节点间通信。

网络波动与状态移动成本：终端接入链路和边缘节点间链路的带宽、RTT、抖动与拥塞状态会直接影响模型切分、task offloading、prefill/decode 分离和 KV 迁移是否值得执行。对于普通 DNN 推理，跨节点传输的主要对象通常是输入或中间 activation；而在 LLM inference 中，KV Cache、prefix cache、session state 和多轮上下文也会成为需要迁移或远程访问的状态资源。网络成本因此不再只是请求入口到服务实例的传输延迟，而是贯穿推理全过程的状态移动成本。

请求负载突发且 SLO 要求异构：单个请求入口附近的边缘节点通常资源受限，请求到达具有突发性，并且不同请求的 SLO 要求各不相同。等待较大的 batch 会增加 TTFT，完全放弃 batching 又会降低 GPU/NPU 利用率。prefill 与 decode 的资源需求不同，长短请求和不同 SLA 请求混合到达时，固定 batch size 与 FIFO 队列很难同时保证吞吐和尾时延。因此，边缘运行时需要根据当前队列、请求时限、KV 占用和阶段负载动态调整组批与执行顺序，以满足不同请求的 SLO 要求。

用户移动与服务连续性：对于单轮短请求，移动可能只改变入口节点；对于多轮对话、长上下文和 agent workflow，移动会改变服务实例与状态位置之间的关系。KV Cache、prefix cache、session state 或 agent memory 可能仍然停留在原服务节点，新请求却已经从新的入口进入系统。系统需要在迁移状态、远程访问原有状态、重新 prefill 或缩短可用上下文之间做选择。因此，评价指标还需要覆盖会话连续性、状态复用和移动场景下的 QoS 稳定性。

隐私边界与数据本地性：部分 prompt、传感器数据、业务上下文或用户状态可能不能离开本地边缘域，也可能只允许在指定节点集合内流动。隐私约束会限制模型选择、offloading 范围、KV 迁移路径和 fallback 策略，使系统不能仅按最小时延或最低成本选择节点。换言之，CEC 中的推理优化必须同时处理资源可行性、链路代价、状态位置、隐私边界和业务 QoS。

3.3 现有 LLM Inference 优化在 CEC 中的关注重点

Model-Level：关注模型表示与结构能否适配目标设备。量化、剪枝和小模型替代会改变模型的容量与计算需求，GQA、MoE 等结构还会影响 KV 大小、条件计算和通信模式。模型结构是否便于按

layer、tensor 或 expert 拆分，会影响跨节点部署的可行性；具体分割位置、并行度和执行顺序则由分布式 Runtime 决定。

Kernel-Level: 关注异构硬件的算子支持与执行效率。CEC 系统可能同时包含不同规格的 NPU、GPU、CPU 及 AI 板卡。各后端支持的数据精度、算子、attention kernel、KV 布局和通信原语并不一致。一个模型在数据中心 GPU 上能够高效执行，并不意味着它在边缘设备上具有相同性能；不受支持的算子、数据布局转换和 CPU 回退都可能抵消协同执行的收益。

Runtime-Level: 把网络 and 状态纳入在线执行控制。云侧 runtime 主要管理单个 serving engine 或集中式集群内的 KV、batch 和执行流水线；CEC runtime 还要读取链路状态、节点负载、KV 位置和用户入口变化。KV 管理由单实例显存分配扩展为跨节点状态管理，batching 需要适应 SLO 要求异构且动态的请求池，分布式执行则要同时安排节点计算和跨节点数据传输。

Serving Policy & Routing-Level: 联合选择执行节点和状态处理方式。一个请求是在入口节点执行，还是交给其他协作节点，需要同时考虑模型可用性、链路状态、节点负载、KV/cache 位置、用户入口变化、隐私限制和 SLA 风险。除选择执行节点外，服务策略还要决定状态是本地复用、跨节点同步、远程访问还是重新计算。

在 CEC 中，模型部署、运行时调度和服务路径选择共享同一组设备、链路和状态条件。模型切分决定执行拓扑和通信量，batching 改变实例容量与排队时间，KV 状态位置又会影响请求卸载代价。基于这种关系，后续章节按模型切分、业务感知 batching 和请求卸载三类具体问题组织相关工作与方案。

3.4 本文采用的 CEC LLM Inference 综述框架

基于上述分析，本文将 CEC 场景下的 LLM inference 优化归纳为三条主线。第一条是资源自适应模型部署，对应模型计算如何分配到协作节点。第 4 章的相关工作综述覆盖 layer-wise、tensor/operator-level、expert/component-level、inference phase 和 semantic/token-level partitioning；面向验证环境的方案则用统一代价模型比较 TP、PP 和 PP+TP，并采用动态规划求解非均匀 PP 的连续 layer 边界。

第二条是业务感知 batching，对应请求进入推理实例后的组批与执行顺序。第 5 章先分析 dynamic batching、continuous batching、chunked prefill 和 prefill/decode 分池等机制，再将在线控制建模为 CMDP，采用带拉格朗日风险约束和安全投影的 Safe-RL 调节 batch 容量、请求排序和运行时机制。

第三条是 LLM inference task offloading，对应跨节点服务路径选择，关注完整请求、会话、复杂 workflow 以及 context、KV Cache、prefix cache 和 session state 如何在协作节点之间路由、同步、复用或重新计算。第 6 章在相关工作综述后，进一步给出联合选择执行节点和 KV 获取方式的长期会话感知方案。

如图 5 所示,这三条主线分别从空间部署、时间调度和服务路径三个角度刻画 CEC LLM inference 的关键优化问题。它们并非彼此独立:模型部署决定可用的执行拓扑,batching 决定实例内部的时间组织,task offloading 决定请求和状态如何在协作网络中移动。第 7 章将进一步讨论三类算法之间的协同优化关系。

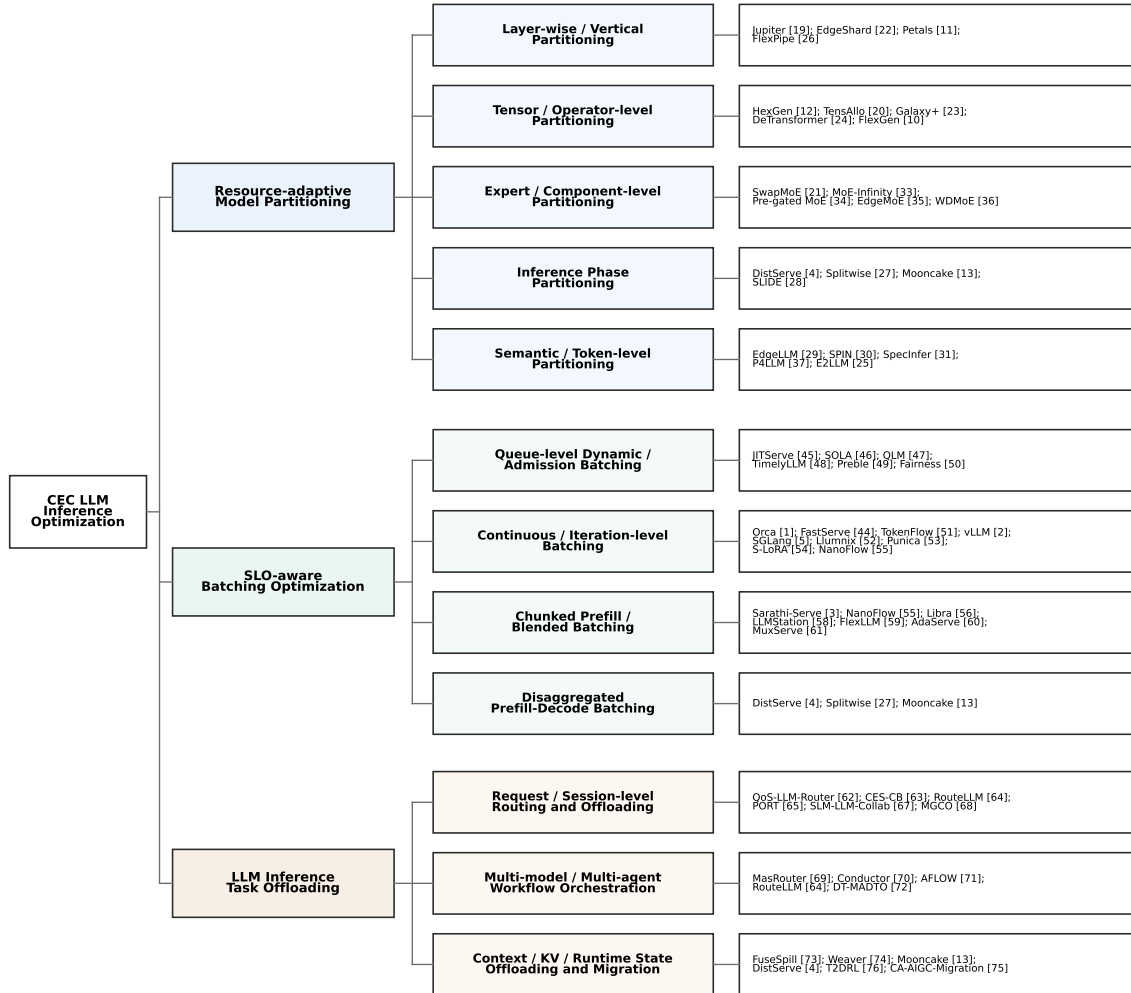


图 5 CEC LLM inference 优化主线、子分类与代表工作总览

3.5 小结

本章从系统形态出发,说明 CEC LLM inference 由多个异构边缘节点协作提供服务。相较数据中心环境,CEC 需要进一步处理资源异构、网络波动、局部请求池规模有限、用户移动、状态连续性和隐私边界等约束,使模型部署、实例内调度和跨节点服务路径使用一致的资源与状态信息。

按照“优化对象-执行范围”分类,CEC 中的 Model-Level 更关注模型表示和结构能否适配目标设备,Kernel-Level 更关注异构硬件的算子支持与执行效率,Runtime-Level 需要同时管理模型切分

执行、网络、状态、batch 和执行流水线，Serving Policy & Routing-Level 则联合选择执行节点与状态处理方式。

基于这些变化，本报告后续围绕三类 CEC LLM inference 优化主线展开：CEC 下的资源自适应模型部署方法、CEC 中的业务感知 batching 参数优化算法，以及 CEC 中的 LLM inference task offloading。

四、CEC 下的资源自适应模型切分部署方法

CEC 下的模型切分部署需要解决如何把一次推理中的模型结构、计算阶段和运行时状态分配到多个协作边缘节点。端侧设备靠近用户并具有数据本地性优势，但算力、内存、功耗和散热约束严格；边缘服务器和 AI 板卡能够提供更强算力，却常常呈现资源碎片化、设备异构和负载波动。模型切分通过重新组织计算与状态的放置，突破单个节点的容量或执行能力限制。Petals [11] 展示了协作式 block 放置，HexGen [12] 分析了异构 GPU 与异构网络下的生成式推理并行，FlexGen [10] 则展示了权重、activation 和 KV Cache 的分层放置。这些工作表明，部署性能同时受到模型权重、activation、KV Cache、互联带宽和运行时调度影响。模型切分部署直接改变推理内部的执行结构，包括模型段放置、层内并行、阶段边界或 token 生成职责。多个完整模型或完整服务实例之间的请求分配属于服务路径选择与负载均衡，将在任务卸载部分讨论；路由若同时触发模型段、推理阶段或 expert 组件迁移，其中改变执行结构的部分仍属于本章范围。本文按主要切分对象将相关方法划分为五类：

1. **Layer-wise / Vertical Partitioning**: 切的是 Transformer 深度方向，决定 layer、block 或 model shard 放在哪些节点上执行。
2. **Tensor / Operator-level Partitioning**: 切的是同一层内部的计算单元，决定 attention head、MLP 矩阵、tensor shard 或 communication primitive 如何并行。
3. **Expert / Component-level Partitioning**: 切的是 MoE expert 等模型组件，决定哪些 expert 常驻、换入、映射或跨节点放置。
4. **Inference Phase Partitioning**: 切的是 prefill、decode、verification、download 等执行阶段，利用不同阶段在计算、访存和时延需求上的差异。
5. **Semantic / Token-level Partitioning**: 切的是生成过程本身，决定 draft、verify、semantic segment、token-wise partition 等语义单元如何协作。

这些分类并不互斥。Jupiter [21] 同时涉及边缘协作执行和 prefill/decode 阶段组织。

TensAllo [22] 同时包含设备选择、模型部署和张量并行；Mooncake [13] 以 prefill/decode 解耦为主，其收益又依赖分布式 KV Cache；SwapMoE [23] 则涉及专家级动态驻留与换入。本文依据方法直接改变的主要对象归类，并在总览表中保留交叉关系，使模型深度、层内计算、专家组件、执行阶段和生成职责等切分对象能够清楚区分。

表中“切分类型”表示论文中与本章分类最接近的主要优化对象，不概括论文的全部贡献。例如，Mooncake [13] 还包含 KVCache-centric 架构，本章依据其 prefill/decode disaggregation 服务路径将其归入阶段切分。

表 6 汇总了本章五类模型切分部署方法的代表工作、发表位置、切分类型、优化目标与加速机制；图 6 进一步以树状结构展示五类方法、主要切分对象和代表工作之间的对应关系。

表 6 CEC 下资源自适应模型切分部署方法相关工作

序号	System / Framework	Venue / Year	切分类型	Objectives	Optimization Technology	加速机制 / 方法核心
1	Jupiter: Fast and Resource-Efficient Collaborative Inference of Generative LLMs on Edge Devices [21]	IEEE INFOCOM 2025	层级切分 / 推理阶段切分	prefill/decode 协同执行	intra-sequence pipeline parallelism; outline-based pipeline parallel decoding; speculative decoding	Prefill 阶段用序列内流水线并行分推长 prompt 计算; decode 阶段用 outline-based pipeline parallel decoding 和 speculative decoding 减少自回归串行步数与流水线气泡
2	TensAllo: Adaptive Deployment of LLMs on Resource-Constrained Heterogeneous Edge Devices [22]	IEEE INFOCOM 2025	层级切分 / 张量切分	在资源受限异构边缘设备上部署 LLM	adaptive device selection; tensor allocation; tensor parallelism; quantization	通过设备选择和 tensor allocation 把计算分配 to 更合适的异构设备; 再用量化降低内存占用, 使更多 tensor shard 能留在边缘设备上执行
3	EdgeShard: Efficient LLM Inference via Collaborative Edge Computing [24]	IEEE IoT Journal 2025	层级切分 / 垂直切分	模型分片放置	model shard placement; joint device selection; dynamic programming	将 LLM 切成 model shards, 并联合选择设备和 shard placement; 动态规划寻找计算时延、传输代价和 pipeline 吞吐之间更优的放置方案
4	Petals: Collaborative Inference and Fine-tuning of Large Models [11]	ACL 2023 Demo	层级切分 / 垂直切分	将大模型 block 分布到多个参与节点	decentralized layer/block placement; pipeline across volunteered servers	把 Transformer block 分散存放在志愿节点 GPU 上, 客户端按 pipeline 顺序调用连续 block, 避免本地 RAM/SSD offloading 反复搬运完整权重
5	HexGen: Generative Inference of Large Language Model over Heterogeneous Environment [12]	ICML 2024	层级切分 / 张量切分	在异构 GPU 和异构网络上组织生成式推理	asymmetric tensor parallelism; pipeline parallelism; constrained scheduling	同时允许 pipeline stage 和 tensor parallel degree 非对称配置, 让快慢 GPU、异构链路承担不同计算份额, 而不是强制所有 stage 使用同构并行度
6	Resource-Efficient Collaborative Edge Transformer Inference With Hybrid Model Parallelism [25]	IEEE TMC 2025	层级切分 / 张量切分	混合并行协同推理	hybrid model parallelism; heterogeneity- and memory-aware planning; communication overlap	用 hybrid model parallelism 结合 layer/pipeline 与 tensor 并行; 通过 ReduceScatter/AllGather 替代部分 AllReduce, 并用通信-计算重叠降低边缘弱链路上的同步等待
7	Co-Designing Transformer Architectures for Distributed Inference With Low Communication [26]	IEEE TPDS 2025	张量/算子级切分	降低分布式 Transformer 推理通信	low-communication architecture co-design; block parallelism; two-phase planning	通过把原 Transformer layer 重构为多个 decoupled sub-block 暴露 block parallelism, 减少层内强依赖和 all-reduce 次数, 再用离线权重放置与在线并行规划选择低通信执行路径
8	FlexGen: High-throughput Generative Inference of Large Language Models with a Single GPU [10]	ICML 2023	张量/算子级切分	在单 GPU 资源受限环境中服务大模型	tensor/KV/activation placement; GPU-CPU-disk offloading; linear programming	把权重、activation 和 KV cache 在 GPU、CPU、磁盘之间分层放置, 用线性规划选择 offloading 和访问模式, 并结合压缩扩大 batch size 与吞吐
9	E2LLM: Structure-Guided Efficient Inference for LLMs in Distributed Edge-IoT Environments [27]	IEEE TMC 2026	推理阶段切分 / 语义级协同切分	边缘结构规划与 IoT 并行解码	structure-guided planning; static-dynamic KV cache partitioning; distributed decoding	边缘节点负责结构规划和 segment-specific KV 生成, IoT 设备并行执行轻量 decoding; 静态/动态 KV 分离减少重复传输, 并用 response skeleton 保持片段语义一致

10	FlexPipe: Adapting Dynamic LLM Serving Through Inflight Pipeline Refactoring in Fragmented Serverless Clusters [28]	EuroSys 2026	层级切分 / 垂直切分	运行时 pipeline stage 重构	fine-grained model partitioning; inflight pipeline refactoring; topology-aware allocation	将模型预切成细粒度 stage, 并在请求执行中动态调整 pipeline granularity; 通过一致的 cache transition 和拓扑感知资源分配适配碎片化 GPU
11	DistServe: Disaggregating Prefill and Decoding for Goodput-optimized LLM Serving [4]	OSDI 2024	推理阶段切分	prefill/decode 物理解耦	phase-specific workers; phase-aware parallelism; KV transfer	将 compute-heavy prefill 与 memory/latency-sensitive decode 分配给不同 worker pool; 分别选择并行度和 batch 策略, 再通过 KV transfer 连接两阶段
12	Splitwise: Efficient Generative LLM Inference Using Phase Splitting [29]	ISCA 2024	推理阶段切分	prompt/token 阶段分离	prompt pool; token pool; hardware matching; overlapped KV transfer	将 prompt computation 和 token generation 拆到不同机器池, 使两阶段匹配不同硬件; 同时用 layer-wise/overlapped KV transfer 降低阶段切换的关键路径开销
13	Mooncake: Trading more storage for less computation – A KVCache-centric architecture for serving LLM chatbot [13]	USENIX FAST 2025	推理阶段切分	prefill/decode disaggregation	distributed KV cache; global scheduling; KVCache-centric architecture	在 prefill/decode 解耦基础上构建分布式 KVCache, 把 CPU、DRAM、SSD、NIC 作为共享状态层; 调度器按 KV 复用、本地性和 SLO 选择 prefill/decode 实例
14	SLIDE: Simultaneous Model Downloading and Inference at the Wireless Network Edge [30]	IEEE TMC 2026	推理阶段切分	下载与推理重叠	layer-wise download/inference overlap; bandwidth and compute allocation	按 layer 递归依赖组织下载和执行: 用户一边接收后续 layer, 一边执行已下载 layer; 联合优化带宽和计算分配以隐藏模型下载等待
15	EdgeLLM: Fast On-Device LLM Inference With Speculative Decoding [31]	IEEE TMC 2025	语义/token 级协同切分	减少端侧逐 token 解码开销	on-device speculative decoding; branch navigation; self-adaptive fallback; compute-I/O pipeline	小 draft LLM 常驻内存生成候选 token, 大 target LLM 批量验证; 用分支导航避免把算力浪费在低概率分支, 并用 compute-I/O pipeline 隐藏大模型权重加载
16	SPIN: Accelerating Large Language Model Inference with Heterogeneous Speculative Models [32]	IEEE INFOCOM 2025	语义/token 级协同切分	异构草稿模型选择与验证流水线	heterogeneous SSM selection; request decomposition; speculation/verification pipelining	为不同难度请求选择合适的异构 speculative model; 把长请求拆分以降低 verification batch padding, 并流水化 speculation 与 verification 阶段
17	SpecInfer: Accelerating Large Language Model Serving with Tree-based Speculative Inference and Verification [33]	ASPLOS 2024	语义/token 级协同切分	token tree 草稿与批量验证	tree-based speculative inference; parallel verification	多个小模型生成候选 token tree, 目标 LLM 不再逐 token 增量生成, 而是作为 verifier 一次并行校验多条候选路径, 从而减少目标模型调用次数
18	SwapMoE: Serving Off-the-shelf MoE-based Large Language Models with Tunable Memory Budget [23]	ACL 2024	专家/组件级切分	专家驻留可调	virtual experts; expert swapping; tunable memory budget	只把动态选择的重要专家作为 virtual experts 常驻主存, 其余专家按需映射和换入, 在不剪枝模型结构的情况下减少 MoE 推理内存占用和换入开销
19	Taming Latency-Memory Trade-Off in MoE-Based LLM Serving via Fine-Grained Expert Offloading [34]	EuroSys 2026	专家/组件级切分	细粒度 expert offloading	fine-grained expert offloading; latency-memory trade-off	以 expert 为换入/卸载单元, 在显存预算下决定哪些专家保留在 GPU、哪些下沉到主机或存储层, 从而在显存节省和专家加载延迟之间折中
20	MoE-Infinity: Efficient MoE Inference on Personal Machines with Sparsity-Aware Expert Cache [35]	ICML 2025	专家/组件级切分	稀疏感知 expert cache	sparsity-aware expert cache; expert prefetching; memory hierarchy	利用 MoE token 路由稀疏性和专家访问局部性, 只缓存即将使用或高频专家, 并通过预取和分层内存降低专家缺页造成的 decode 停顿

21	Pre-gated MoE: An Algorithm-System Co-Design for Fast and Scalable MoE Inference [36]	ISCA 2024	专家/组件级切分	提前 expert routing 与系统协同	pre-gating; expert scheduling; algorithm-system co-design	在系统执行前更早获得 expert 选择信息, 使 expert 加载、通信和调度可以提前安排, 减少 MoE 推理中的路由等待和跨设备同步
22	EdgeMoE: Empowering Sparse Large Language Models on Mobile Devices [37]	IEEE TMC 2025	专家/组件级切分	移动端稀疏 LLM 部署	sparse MoE deployment; expert management; on-device inference	面向移动设备资源约束管理 MoE expert 的驻留与执行, 使稀疏大模型能够在端侧或近端资源上以可控内存开销运行
23	WDMoE: Wireless Distributed Mixture of Experts for Large Language Models [38]	IEEE TWC 2025	专家/组件级切分	无线边缘分布式 MoE	wireless distributed experts; expert selection; bandwidth allocation	将 gating/attention 和 experts 分布在基站与移动设备之间, 根据 expert 权重、设备延迟和无线带宽联合选择专家与分配通信资源
24	P4LLM: Protecting Embedding Privacy against Model Inversion Attacks for EaaS LLM Serving via Token-wise Partition and Perturbation [39]	IEEE TMC 2026	语义/token 级协同切分	token-wise partition + perturbation	insensitive token filtering; partitioning scheduler; hidden-state perturbation	在用户侧按 token 切分 Transformer 执行, 只对敏感 token 的 hidden state 做扰动并上传, 非敏感 token 过滤或轻处理, 从而减少扰动计算和传输负担

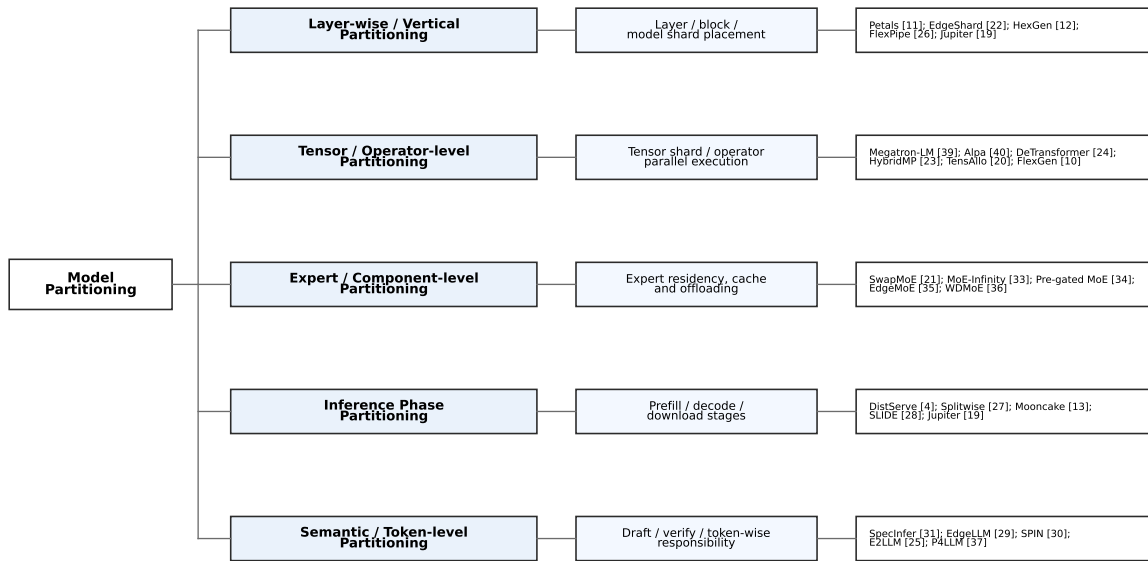


图 6 CEC 下资源自适应模型切分部署方法的分类树

4.1 Layer-wise / Vertical Partitioning

层级切分是最接近传统意义上“模型分割”的一类方法。它把 Transformer 视为由多个顺序 block 组成的计算链, 在 layer 或 block 边界上拆分模型, 并将不同模型段放置到多个协作节点执行。对于无法完整加载模型权重、activation 和运行时状态的边缘设备, 层级切分提供了一种直接的扩展方式: 单个节点不再承担完整模型, 而是多个节点共同完成同一次 forward。Petals 展示去中心化 block 放置

[11], EdgeShard 面向边缘 shard placement [24], HexGen 讨论异构生成式推理 [12], FlexPipe 则关注 serverless 集群中的 pipeline 重构 [28]; 这些工作共同验证了沿深度方向拆分模型的价值。

为了把这种“沿深度方向展开”的执行方式与普通请求路由区分开,可以参考图 7。图中以三类

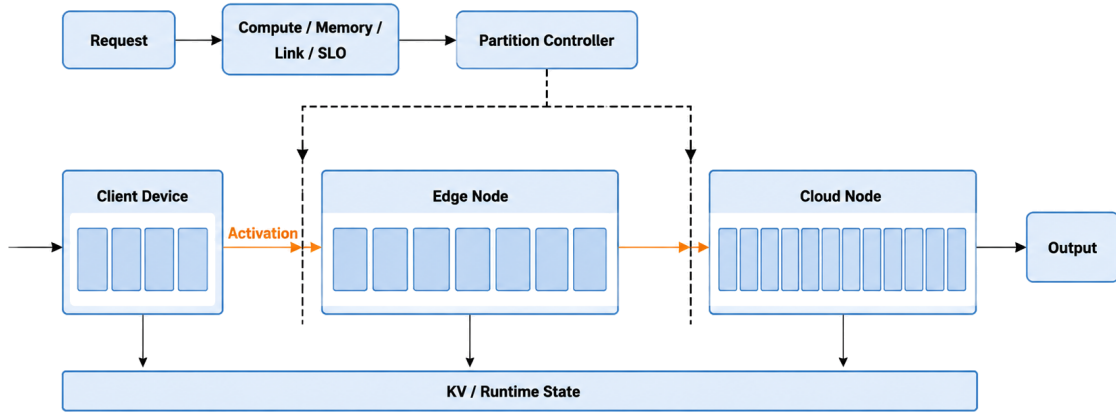


图 7 Layer-wise / Vertical Partitioning 机制示意图

能力不同的协作节点为例,将同一个 Transformer 的连续 layers 分段放置,并用不同的 layer 数量表示异构设备可以承担不同计算量。虚线表示 layer 边界上的 split point,相邻模型段之间传递 activation; 下方的 KV / Runtime State 表示各模型段执行时需要维护的推理状态。Partition Controller 综合计算能力、显存余量、链路状态和业务 SLO 确定分割点及设备放置。合理的层级切分应使各 stage 的执行时间接近,并控制跨节点传输开销。

这一类方法需要联合确定 split point 和 placement。切分点靠前,可以减轻前置节点的计算和内存压力,但会更早产生跨节点 hidden state 或 activation 传输;切分点靠后,可以降低网络依赖,却要求前置节点执行更多 layer。对于长上下文请求,activation 规模和 KV 状态增长会进一步放大通信成本;对于短请求,通信启动时延可能抵消计算节省。因此,层级切分通常需要同时估计每层计算量、activation 大小、显存占用、链路带宽、RTT、节点负载和功耗约束。

CEC 环境使层级切分比数据中心内的 pipeline parallelism 更难。数据中心内的互联带宽相对稳定,而终端接入链路和边缘节点间链路可能受到无线抖动、跨域转发和拥塞影响。一个在静态性能测量中最优的切点,到了移动用户、突发请求或边缘节点过载时可能迅速失效。因此,资源自适应的层级切分往往需要三类能力:离线阶段建立逐层性能数据,在线阶段根据节点和链路状态选择切点,运行时阶段在负载变化时进行有限频率的重配置。GPipe 体现了 pipeline parallelism 在稳定集群

中的基本价值 [40], Megatron-LM 展示了大模型 tensor parallelism 的基础路线 [41], Alpa 则进一步自动化 inter-operator 与 intra-operator parallelism [42]。CEC 需要在这些并行思想之上继续纳入异构链路和在线状态。

这类方法还要处理流水线稳定性。切分后, 每个模型段的执行时间如果差异过大, 较慢节点会成为 pipeline bottleneck; 中间节点失败或链路恶化时, 后续 layer 是否迁移、是否重新选择切点、是否切换到可用的完整模型实例, 都会影响服务连续性。对于 CEC LLM 推理, 层级切分不能只作为部署算法讨论, 还必须和 admission control、状态管理和异常回退衔接。

总体来看, 层级切分适合模型容量和单节点部署能力不足的场景, 尤其适用于模型段执行时间差异明显、节点间链路相对稳定、跨节点 activation 可控的系统。它的主要风险在于切分收益被网络传输和流水线等待抵消。因此, CEC 中的层级切分更适合采用“少量稳定切点 + 在线代价修正”的方式, 而不是频繁改变模型执行拓扑。

这里的“少量稳定切点”并不意味着静态平均切分, 而是指在线阶段只搜索少量经过性能测量确认、可部署且不会 OOM 的切点, 避免运行时任意切分造成模型重加载和状态迁移震荡。

4.2 Tensor / Operator-level Partitioning

当层级切分仍不足以缓解单层计算和显存压力时, 系统需要进入更细粒度的张量/算子级切分。LLM 的每一层内部包含 attention、QKV projection、MLP、归一化和通信同步等多个计算单元, 其中 attention head、矩阵乘和中间 activation 都可能成为单设备瓶颈。张量/算子级切分不再以完整 block 为边界, 而是在同一层内部拆分 tensor、head、矩阵维度或 operator 执行路径。Megatron-LM 和 Alpa 代表了数据中心大模型中 tensor / intra-operator parallelism 的基础路线 [41], [42]; DeTransformer 通过结构协同设计降低分布式推理通信 [26], Galaxy+ 采用混合模型并行组织边缘 Transformer 推理 [25], TensAllo 则把异构边缘设备选择和 tensor allocation 纳入部署决策 [22]。

层内拆分的系统关系可以用图 8 概括, 它突出的是同一 Transformer 层内部的并行和同步, 而不是层与层之间的顺序切点。图中展示了张量/算子级切分的核心执行关系。输入 hidden state 被分发到 Device A 和 Device B, 两个设备先并行计算各自的 attention shard, 经 Collective Sync 合并中间结果, 再并行计算 MLP shard, 并通过第二次同步得到输出 hidden state。Attention 可以按 head 或 QKV 维度拆分, MLP 可以按矩阵维度拆分, 而 all-gather、reduce-scatter 或 all-reduce 等集合通信负责恢复完整的层输出。该过程说明, 层内并行能够分摊单层计算和显存压力, 但每一层都可能引入同步; 在 CEC 中, 链路带宽和抖动会直接决定这种水平切分是否具有实际收益。

这类方法通常保持模型数学语义不变, 改变的是层内计算如何分布到多个设备。它可以表现为 tensor parallelism、hybrid model parallelism、block parallelism、低通信 Transformer 结构或 structure-guided partitioning。与层级切分相比, 它能更细地利用碎片化资源; 与单机 kernel 优化相比, 它必须

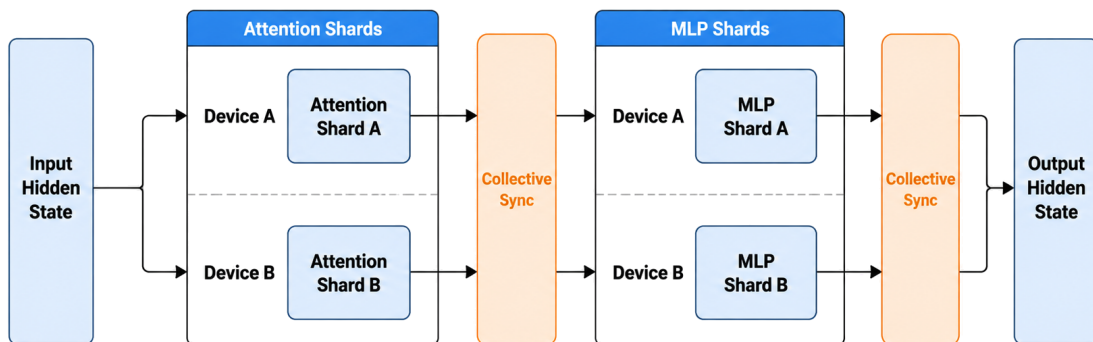


图8 Tensor / Operator-level Partitioning 机制示意图

显式处理跨设备通信、同步和拓扑选择。

本章将 Tensor / Operator-level Partitioning 视为比普通 kernel 优化更高一层的系统问题。若某项工作只优化单设备 GEMM/attention kernel，而不改变层内计算的跨设备分布，则不归入本类。

水平切分的核心矛盾是并行收益与通信惩罚。切得越细，单个设备承载的权重、activation 和中间矩阵越少，也更容易把多个弱设备组织起来；但 all-reduce、all-gather、reduce-scatter 或点对点传输会更频繁。切得越粗，通信开销下降，却可能重新遇到显存或算力瓶颈。数据中心可以依赖高速互联吸收一部分同步成本，CEC 中的边缘设备却往往只有普通以太网、无线链路或多跳边缘互联，过细的 tensor parallelism 很容易变成通信密集型执行。

因此，CEC 下的张量/算子级切分需要更详细的设备计算和通信特征。调度器要知道哪些设备适合低比特 GEMM，哪些设备适合 attention，哪些设备只能承担较小 tensor shard；还要估计层内同步频率、通信 primitive 的代价和 batch size 对算子效率的影响。某些工作还通过模型结构 co-design 减少层内依赖，使模型更适合低通信分布式推理。水平切分由并行度、模型结构、设备选择和通信路径共同决定。

张量/算子级切分适合单层计算或显存压力成为瓶颈的场景，但在 CEC 中不能简单照搬数据中心 tensor parallelism。边缘链路的带宽、抖动和同步延迟会直接决定并行收益是否存在。因此，这类方法应优先与低通信结构、异构设备选择和静态/在线性能测量结合，并尽量避免让每个 token 都经过高频跨节点同步 [22], [25], [26]。

4.3 Expert / Component-level Partitioning

专家/组件级切分主要面向 MoE 等具有显式内部组件选择机制的模型。与普通 dense Transformer 不同，MoE 模型通常在 FFN 部分包含大量 expert，而每个 token 只激活其中少数 expert。这个结构使模型天然具备“组件级可调度性”：系统不一定让所有 expert 同时常驻同一设备，而可以根据 expert 热度、token routing 分布、显存预算、链路状态和请求类型，决定哪些 expert 常驻、哪些按需换入、哪些放在其他协作节点上。SwapMoE 从 expert swapping 角度降低显存压力 [23]，MoE-Infinity 利用 sparsity-aware expert cache 支撑个人设备上的 MoE 推理 [35]，Pre-gated MoE 通过提前 expert routing 协同系统调度 [36]，EdgeMoE 面向移动端稀疏模型部署 [37]，WDMoE 则讨论无线分布式 expert 放置和带宽分配 [38]。这些工作共同说明，组件级切分已经成为 MoE 推理系统的重要组织方式。

MoE 场景下，expert 的部署位置与运行时 token routing 是两个相互关联但作用阶段不同的问题，图 9 对这两类过程进行了区分。图中上方的控制路径表示部署阶段：Expert Placement 根据硬件能力、

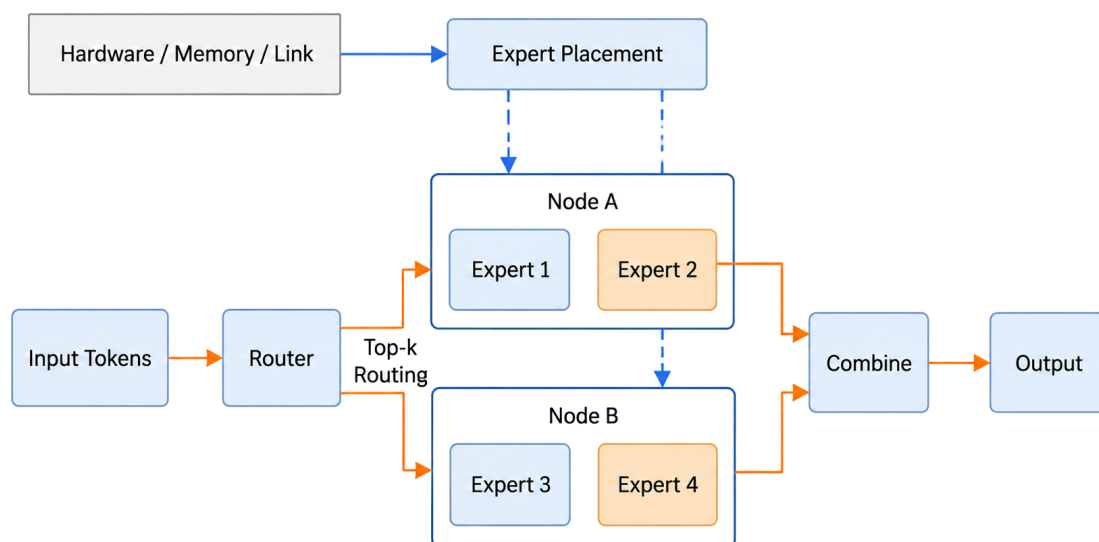


图 9 Expert / Component-level Partitioning 机制示意图

显存预算和链路状态，将 Expert 1、Expert 2 放置在 Node A，将 Expert 3、Expert 4 放置在 Node B。下方的运行时路径表示稀疏激活：输入 token 经 Router 完成 top-k routing 后，只访问本轮被选中的 Expert 2 和 Expert 4，两个 expert 的输出再由 Combine 合并。部署决策决定 expert 位于哪个节点，routing 决策则决定当前 token 实际执行哪些 experts。该机制能够利用稀疏激活降低单次推理计算量，但跨节点路由通信和 expert 负载不均仍可能成为瓶颈。

专家/组件级切分以具有模型语义的组件单元为对象，与按矩阵维度或 communication primitive 划分的张量/算子级切分不同。一个 expert 往往对应一组完整参数和计算路径，其是否被激活取决于

token-level routing；系统需要管理大量稀疏激活组件的驻留、映射和调度。因此，专家/组件级切分发生在模型内部组件层面，其收益取决于 token 分布和专家访问模式。

在 CEC 场景下，专家/组件级切分的价值在于降低大模型组件的常驻压力。资源较小的节点可以保存高频 expert、局部业务相关 expert 或经过压缩的 expert cache，容量更大的协作节点则维护更完整的 expert pool，并处理低频或复杂请求。系统需要在 expert 命中率、换入换出成本、跨节点 expert 访问时延和输出质量之间做权衡。如果 expert 热度预测准确，组件级切分可以减少显存占用并提高边缘可部署性；如果预测失误，频繁 swap 或远程访问 expert 会直接破坏 decode 时延。

专家/组件级切分的决策变量包括 expert residency、expert cache size、expert hotness prediction、token routing statistics、swap bandwidth、memory budget、fallback policy 和跨节点 expert placement。与层级切分相比，它更细粒度；与张量切分相比，它更依赖模型稀疏激活；与语义/token 级协同相比，它仍然保持同一个 MoE 模型内部的 expert 语义一致性，而不是把 draft、verify 或语义段分给不同模型完成。

总体来看，专家/组件级切分适合 MoE 模型、expert 热度具有局部性、边缘节点无法常驻完整 expert 集合的场景。它不应被简单理解为普通 request routing，因为请求仍在同一个 MoE 模型语义下执行；也不应被完全归入 token-level collaboration，因为系统主要调度的是 expert 组件而非生成职责。现有代表工作已经覆盖 expert swapping、fine-grained expert offloading、expert cache、pre-gating 和 wireless distributed experts 等不同角度；CEC 场景下仍然缺少把用户移动、区域业务热度、expert cache 迁移和无线带宽联合起来建模的工作 [34], [38]。

若目标模型是 dense LLM，本类方法一般不应作为模型切分部署的主路径；若目标模型或后续业务引入 MoE、专家模型或多 adapter 组件，本类才会成为核心部署问题。

4.4 Inference Phase Partitioning

LLM 推理不是单一形态的计算。Prefill 需要一次性处理完整 prompt，矩阵规模大、并行度高，通常更偏 compute-bound；decode 逐 token 生成，频繁访问 KV Cache，通常更偏 memory-bound 或 latency-bound；speculative decoding 中的 verification 又会形成目标模型批量校验；在无线边缘场景中，模型下载、加载和推理还可能同时出现在关键路径上。推理阶段切分正是利用这些阶段在计算、访存和时延需求上的差异，把不同阶段放到更合适的节点或资源池上执行。DistServe [4]、Splitwise [29] 和 Mooncake [13] 是 prefill/decode disaggregation 的代表；Jupiter 从边缘协作角度组织 prefill/decode 执行 [21]，E2LLM 从 Edge-IoT 结构化生成角度扩展阶段分工 [27]，SLIDE 则讨论无线边缘中的下载-推理重叠 [30]。

阶段级切分更适合放在推理流水线视角下理解，图 10 给出了 prefill、decode、verification 与 KV/state handoff 的典型组织方式。这里的 Stage Scheduler 根据 prompt length、SLO、link state 和 KV

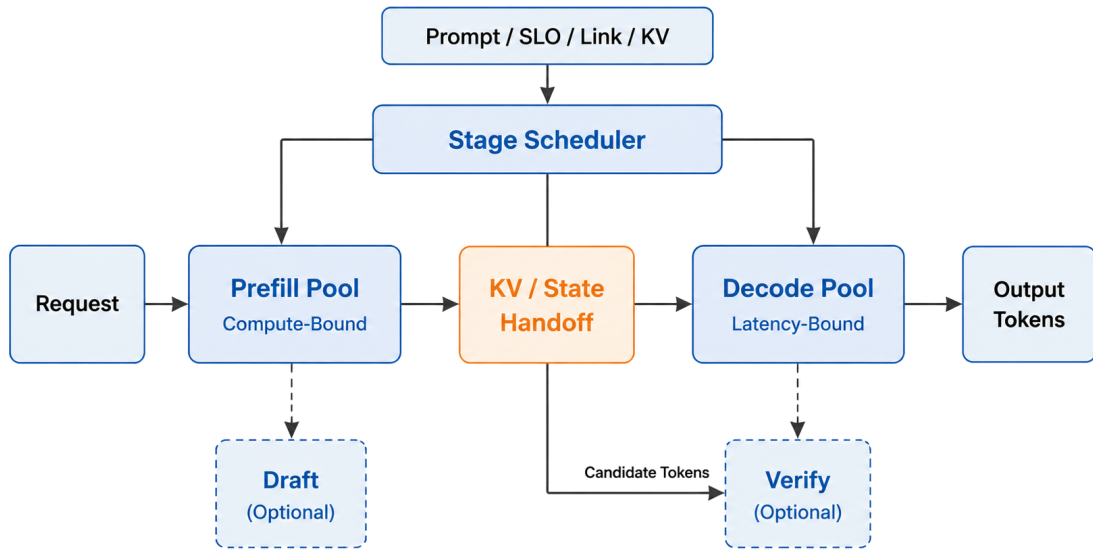


图 10 Inference Phase Partitioning 机制示意图

locality 把请求送入不同阶段资源池。Prefill Pool 处理 compute-bound 的 prompt batch，Decode Pool 维护 memory/latency-bound 的 active sequence，KV / State Handoff 负责在阶段之间传递 KV blocks、metadata 和必要状态。若系统引入 verification 或 draft stage，它们也可以作为独立阶段被调度。阶段切分的核心收益来自阶段与设备特征的匹配以及阶段间干扰隔离，核心代价则集中在 KV/state 传输与阶段队列协调。

最典型的形式是 prefill/decode disaggregation。长 prompt 的 prefill 可以放在算力更强、吞吐更高的协作节点，低时延 decode 则由靠近用户且状态可达的节点承担。这样做可以减少长 prefill 对 decode 的阻塞，也能让不同硬件分别服务 compute-heavy 和 memory-heavy 阶段。进一步地，系统还可以把 draft generation 和 target verification 分开，或将模型下载与推理执行重叠，从而缩短冷启动和模型切换时延。

阶段切分的决策变量包括 prefill worker、decode worker、stage queue、pipeline boundary、KV transfer、verification placement、download/inference overlap 和阶段间同步机制。它的收益来自资源匹配和干扰隔离，但代价集中在状态交接。Prefill 和 decode 共享 KV Cache，draft 和 verify 共享候选 token 与概率校验状态，下载与推理重叠则会竞争网络、存储和内存带宽。换言之，阶段切分能否有效，取决于拆开之后能否低成本地把状态交给下一阶段 [4], [29], [13]。

在 CEC 中，阶段切分可以利用协作节点之间的能力差异：算力较强且能够聚合请求的节点承担 prefill-heavy 请求，靠近用户或持有会话状态的节点承担低时延 decode，容量更大的节点处理长上下文、target verification 或复杂推理。但这种分工不是无条件成立：如果 prefill 完成后需要跨节点传输

大量 KV，链路抖动就会直接放大 TTFT 或后续 TPOT；如果局部并发不足，独立 decode worker 可能无法形成有效 batch；如果用户移动导致入口切换，阶段边界还可能触发状态迁移。因此，阶段切分实际是在阶段资源匹配和状态连续性之间做权衡。

推理阶段切分适合 prefill/decode 负载差异明显、阶段队列规模足够、状态传输可控的场景。若请求很短、边缘并发很低或链路抖动大，复杂阶段拆分可能不如本地顺序执行稳定。因此，阶段切分需要与 KV 状态管理、链路预测、admission control 和回退策略共同设计，并贯穿推理流水线的资源组织过程。

阶段切分与第五章的 Disaggregated Prefill-Decode Batching 视角不同。第四章关注 prefill/decode 是否跨实例或跨节点部署；第五章关注在已经分离的 prefill pool 与 decode pool 内部如何分别组批、排队和控制参数。

4.5 Semantic / Token-level Partitioning

语义/token 级协同切分与前四类不同。它不一定把同一个模型的 layer、tensor 或 expert 组件拆开，而是把一次生成过程中的职责拆开：草稿由小模型产生，验证由大模型完成；长回答可以按语义段并行生成；隐私敏感 token 或 embedding 可以被单独 partition 和 perturbation。这里“切分”的对象是生成语义和 token 处理职责，而不是模型内部参数组件。SpecInfer 展示 token tree verification [33]，EdgeLLM 讨论端侧 speculative decoding [31]，SPIN 关注异构草稿模型选择 [32]，E2LLM 则展示语义段级并行生成 [27]。

生成职责级方法的结构可参考图 11。该类方法直接分配 draft、verify、refine 和 privacy filtering 等生成职责。从系统路径看，Semantic Planner 根据 task type、confidence 和 privacy tag 判断每段生成职责交给哪个模块。Local Small Model 负责生成 draft tokens，Edge Model 可以承担部分 semantic segment generation，Privacy Guard 对敏感 token 做 partition 或 perturbation，Cloud Large Model 负责 verify/refine。Acceptance Check 决定草稿是否接受、是否回退到大模型重算。token/语义级方法的关键约束是质量一致性和错误回退，而不仅是加速。

这类方法的出发点，是自回归生成并不必然只能由一个模型连续完成。系统可以在资源较小的节点先生成 draft，再由部署目标模型的协作节点 verify；可以先生成 response skeleton，再把不同语义段交给多个设备并行生成；也可以将隐私敏感 token 从普通 token 流中分离出来，采用额外扰动或分区策略。与层级切分和张量切分相比，它改变的是生成过程中的职责边界，而不是纯粹的模型拓扑 [31], [39]。

语义/token 级协同的核心变量包括 draft length、acceptance rate、verification frequency、semantic segment boundary、token partition、privacy budget、fallback condition 和输出质量约束。它的收益通常来自减少大模型调用、并行化长回答生成或细粒度保护敏感信息；但风险也更直接地体现在输出质

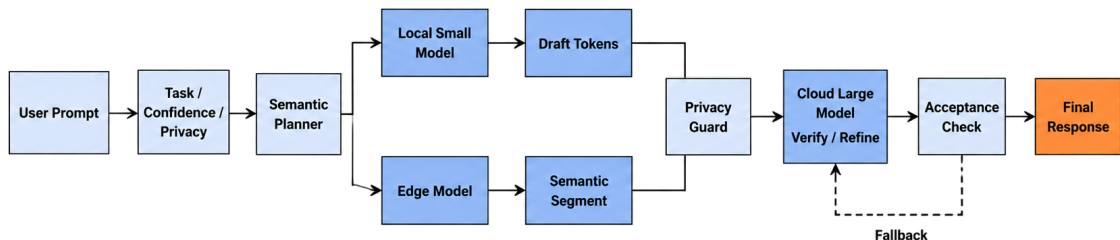


图 11 Semantic / Token-level Partitioning 机制示意图

量上。draft 接受率过低会浪费验证计算，语义段边界选择不当会破坏上下文连贯性，过强的 token perturbation 可能损害语义一致性。

在 CEC 中，这类方法的意义在于把不同规模模型之间的协作从请求级选择推进到生成级协作。资源较小的节点可以承担低成本 draft 或局部补全，能力更强的协作节点完成语义验证或复杂生成，仅在必要时调用外部大模型服务。这样既有机会减少跨域调用，也能改善交互时延和数据本地性。不过，这类方法对在线质量估计和回退机制要求最高，不能只依据资源指标判断收益。

语义/token 级协同切分适合端侧小模型可用、任务分布相对稳定、局部生成质量可被估计的场景。它的风险也最明显：低接受率、语义段不连贯或隐私扰动过强都会破坏质量。因此，这类方法需要和质量评估、置信度估计、安全回退以及外层服务策略联合设计，不能只用系统吞吐指标来评价。

本类中的 speculative decoding 工作不等价于“模型切分”本体；本章保留它们，是因为它们切分了 draft、verify、token tree 或语义段等生成职责，属于生成过程级的协同执行。

4.6 面向华为验证环境的方案建议

给定模型结构、异构 Ascend 设备和业务负载，部署设计需要先判断 TP、PP 与 PP+TP 中哪种并行方式更适合当前环境，再确定连续 layers 在不同设备之间的分配方式。并行方式决定通信发生在每一层内部还是少量 stage 边界，layer 分割则决定各设备的计算和显存负载，因此两者需要使用一致的性能模型进行分析。

整体分析分为两个阶段。首先，在统一代价模型下分别计算 TP、PP 和 PP+TP 的较优配置，比

较不同通信方式在目标组网上的性能。当前四类验证组网的计算结果均选择非均匀 PP，因此后续 Adaptive-PP 固定采用 PP，只由动态规划确定连续 layer 分割点。TP 和 PP+TP 作为性能对照保留，最终通过完整推理服务验证满足 SLA 时的请求处理速度和并发承载能力。

为给出上述选择的定量依据，本文先使用统一代价模型分别计算 TP、PP 与 PP+TP 的预期性能。比较覆盖 CodeLlama34B、Qwen2-VL-7B-Instruct 和四类组网构成的八组场景，并以按 layer 数量均分的 Equal-PP 作为基线。其中，最优 TP 指在可行设备组合与并行度中，计入设备算力和 All-Reduce 开销后处理速度最高的配置；PP+TP 则将模型划分为多个 PP stage，并在部分 stage 内使用 TP。**最优 TP 的处理速度相对基线变化范围为 -77.6%~21.5%，动态规划得到的非均匀 PP 提升 6.5%~123.6%，PP+TP 在六组可行场景中的变化范围为 -1.2%~115.6%**。八组场景中性能最高的方案均为非均匀 PP，说明在算力不一致且存在跨节点通信的环境中，根据设备能力调整各 stage 的 layer 数量更有利于提高处理速度。具体配置、计算过程和各场景结果将在 4.6.2 节展开，实际部署时再利用 Ascend 上的逐层执行、集合通信和端到端服务数据进行校准与验证 [43]。

4.6.1 系统模型

模型切分所需的信息可归纳为硬件环境、模型特征和业务负载三部分。系统模型把设备能力、网络连接、模型结构和请求特征放入同一描述中，并将其转换为后续 TP、PP 与 PP+TP 性能计算所需的统一输入。

图 12 给出了模型切分的系统模型。硬件侧以算力拓扑图记录每台设备当前可用的计算能力和显存容量，并在设备之间标注有效带宽、传输时延与连接关系；模型侧记录逐层计算量、权重、activation 和 KV Cache 增量，同时区分 CodeLlama34B 的文本解码层与 Qwen2-VL-7B-Instruct 的视觉编码器、模态投影和文本解码层；业务侧则提供输入输出长度、micro-batch 数量以及 TTFT、TPOT 和 E2E latency 等 SLA 条件。这些输入共同进入统一代价模型，再形成 TP、PP 和 PP+TP 三类可部署方案。

图中的四设备路径对应 A800T-A2 四卡组网，由 D1-D4 组成。D1 和 D2 使用完整算力，D3 只能使用一半算力，D4 只能使用四分之一算力，因此四台设备的有效算力依次为 230、230、115 和 57.5 TFP16，每台设备具有 64 GB 可用显存。D1-D4 可以通过 100 Gbps 或 25 Gbps 链路连接，其中受限算力的 D3 与 D4 还构成 56 GB/s HCCS 路径。另一条联合组网路径由 Atlas 300I Pro 的 310P、半算力 910B4 和完整算力 910B4 组成，有效算力分别为 70、115 和 230 TFP16，相邻设备通过 25 Gbps 主机网络连接。不同设备路径使算法能够同时观察算力差异和通信条件对切分方案的影响。

部署前估算以设备的可用资源为基础：A800T-A2 单个 NPU 的算力为 230 TFP16、显存为 64 GB，Atlas 300I Pro 整卡的算力为 70 TFP16、显存为 24 GB。设备限制及连接关系采用图 12 所示配置 [43]。后文将四卡 100 Gbps、四卡 25 Gbps、受限两卡 HCCS 和三设备 25 Gbps 联合组网依次记为 E1、E2、E3 和 E4。

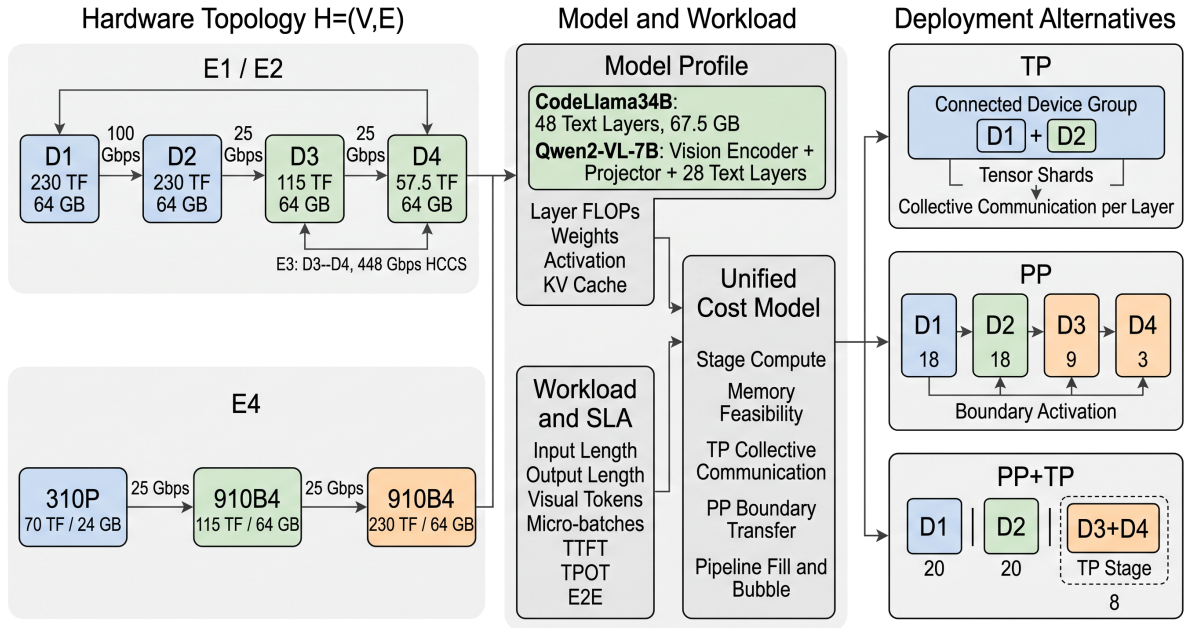


图 12 模型切分的系统模型

是否切分取决于单设备能否承载目标业务。CodeLlama34B 的两字节权重约为 67.5 GB，加上 KV Cache 和运行时空间后会超过当前 64 GB 配置。Qwen2-VL-7B-Instruct 的权重约为 16.6 GB，可以驻留在单台设备上，但目标并发下的 KV Cache、视觉输入开销和 SLA 仍可能要求扩展到多设备。正式实验保留可行的单设备部署作为参考：容量和 SLA 均由单设备满足时采用单设备结果；需要多设备扩展时，再在同一指定设备路径上比较 TP、PP 与 PP+TP。

部署前，算法根据设备规格、模型结构和代表性请求估计各连续 layer 区间的计算、显存与通信代价。进入验证环境后，再通过设备监控和微基准测试获得可用算力、逐层执行时间、显存占用和链路开销，并用实测数据更新系统模型。设备状态变化时只需更新这些输入，后续性能分析和分割算法本身保持不变。

4.6.2 并行方式性能理论分析

在相同模型、请求窗口和硬件拓扑下，TP、PP 与 PP+TP 可以通过窗口完成时间和处理速度进行统一比较。E1-E4 组网上的计算结果既用于判断更合适的并行方式，也用于衡量较优方案相对于按 layer 数量均分的 Equal-PP 具有多大的改进空间。

TP、PP 和 PP+TP 的差别主要体现在通信发生的位置。TP 将每个 Transformer layer 的矩阵计算分到多台设备，并在层内执行集合通信；它能够降低单卡权重和计算压力，但同步会在几乎每一层发生。PP 让每台设备执行一段连续 layers，只在 stage 边界传输 activation，因而更容易适应跨机低带宽链路。PP+TP 则把高速互联且能力接近的设备组成 TP stage，再用 PP 连接不同设备组，适合单个

stage 仍需多卡承载的情况。表 7 给出三种方式的比较。

表 7 不同并行方式与华为验证环境的适配性比较

方式	切分对象	主要收益与代价	适用条件
TP	单层 tensor 与算子	分摊单层参数和计算, 但每层均需集合通信并等待最慢设备	组内带宽高、设备能力接近, 且单卡计算或显存压力较大
PP	连续 layer 区间	主要传输 stage 边界 activation, 可给异构设备分配不同 layer 数	跨节点带宽有限或设备能力差异明显
PP+TP	TP 设备组与连续 layer 区间	组内利用 TP, 组间利用 PP, 同时承担集合通信和流水线开销	局部高速互联可承载 TP, 而设备组之间更适合 PP

当前环境中的设备能力存在明显差异。纯 TP 通常将每层的权重张量按相同大小切分到各设备, 使各设备承担近似相同的计算量。由于每层计算后需要同步, 其执行速度容易受到组内最低算力设备的限制。Equal-PP 按 layer 数量平均分割, 又隐含各设备处理相同数量 layers 所需时间接近的假设。当部分 NPU 只能使用二分之一或四分之一算力时, 这两种方法都可能形成明显瓶颈。PP+TP 能够在局部设备组内增加并行度, 其收益取决于计算加速能否覆盖集合通信与同步开销。

基于上述差异, 理论分析先比较 TP、PP 和 PP+TP 在给定设备路径上的预计性能。当前结果确定采用 PP 后, Adaptive-PP 再依次完成资源与模型画像、连续 layer 区间代价计算、动态规划搜索和硬件验证。前一步解释 PP 的选择依据, 后一步给出适应异构算力和链路条件的非均匀分割结果。

为使不同并行方式的结果能够复算, 计算过程分为 layer 计算、TP 集合通信、PP 边界传输和流水线窗口四部分。首先根据模型结构和请求长度计算每个 layer 的 FLOPs。对于采用 grouped-query attention 和 gated MLP 的文本 Transformer layer, 设 hidden size 为 h , KV heads 对应的投影宽度为 h_{kv} , FFN intermediate size 为 m , 则该层的主要参数量为

$$W_{\text{layer}} = 2h^2 + 2hh_{kv} + 3hm.$$

前两项对应 Q、K、V 和输出投影, 第三项对应 gated MLP 的两个输入投影和一个输出投影。设该次 prefill 处理 S 个 tokens, 并将一次乘加计为 2 FLOPs, 则单层计算量近似为

$$P_{\text{layer}}(S) = 2SW_{\text{layer}} + 4S^2h.$$

第一项表示各线性层的矩阵乘法, 第二项表示 attention score 与 attention output 两次随序列长度平方增长的矩阵乘法。CodeLlama34B 取 $h = 8192$ 、 $h_{kv} = 1024$ 、 $m = 22016$ 和 $S = 512$, 得到每层参数量约 6.92 亿、线性层计算量 708.67 GFLOPs、attention 计算量 8.59 GFLOPs, 合计 717.26 GFLOPs。

Qwen2-VL-7B-Instruct 的文本 layers 使用同一计算方法, 其中 token 数取视觉 token 与文本 token 之和。视觉模块的计算量需要另外加入首个 stage。设合并后的 visual token 数为 n_v , 空间合并倍率

为 s ，则进入视觉编码器的 patch token 数为 $n_p = s^2 n_v$ ；进一步设视觉 hidden size 为 d_v ，vision block 数为 L_v ，单个 patch 的输入维度为 v_p ，文本 hidden size 为 h ，则视觉编码器与模态合并模块的计算量为

$$P_{\text{vision}}(n_v) = 2n_p v_p d_v + L_v(24n_p d_v^2 + 4n_p^2 d_v) + 2n_v((s^2 d_v)^2 + s^2 d_v h).$$

三项依次对应 patch embedding、vision blocks 和 patch merger。取 $n_v = 640$ 、 $s = 2$ 、 $d_v = 1280$ 、 $L_v = 32$ 、 $v_p = 3 \times 2 \times 14^2$ 和 $h = 3584$ ，得到视觉模块计算量约 4359.72 GFLOPs。其 28 个文本 layers 在 640 个 visual tokens 和 256 个 text tokens 下，每层约为 429.13 GFLOPs。因此，VLM 的首个 stage 时间同时包含视觉模块计算和分配到该 stage 的文本 layer 计算。

设第 i 个 layer 的计算量为 P_i FLOPs，设备 g 的标称算力为 F_g^{peak} FLOPs/s，算子执行效率为 η_g ，则有效算力为 $F_g = \eta_g F_g^{\text{peak}}$ 。连续 layer 区间 $(i, j]$ 的计算量及其在单台设备上的计算时间分别为

$$C(i, j) = \sum_{\ell=i+1}^j P_{\ell}, \quad T_{\text{comp}}(i, j, g) = \frac{C(i, j)}{F_g} \times 10^3 \text{ ms}.$$

第 4.6.1 节列出的 230、115、70 和 57.5 TFP16 表示当前可分配的计算能力。由于硬件 profile 尚未完成，表 8 的部署前计算暂取 $\eta_g = 1$ ，直接使用这些数值；获得实测结果后，再根据同一 layer 的理论 FLOPs 与实际执行时间校准 η_g 。在当前计算中，CodeLlama34B 的一个 717.26 GFLOPs layer 在 230 TFP16 设备上的时间为 3.12 ms，在 115 TFP16 和 57.5 TFP16 设备上分别为 6.24 ms 和 12.47 ms。

TP 的通信量由参与设备数、activation 大小和集合通信次数共同决定。当前部署前估算按照常见 TP 执行过程，将 attention 输出投影和 MLP 输出投影之后的同步各记为一次 All-Reduce，因此每个文本 Transformer layer 计入两次 All-Reduce。对于包含 b 个请求、每个请求含 S_i 个 tokens、hidden size 为 h_i 、每个元素占 e byte 的 activation，一次集合通信的数据量为

$$A_i = b S_i h_i e \text{ byte}.$$

设 TP 并行度为 d ，组内有效带宽为 B Gbps，单步通信启动时延为 L ms，则 ring All-Reduce 的时间为

$$T_{\text{AR}}(A_i, d, B, L) = 2 \frac{d-1}{d} \frac{8A_i}{B \times 10^9} \times 10^3 + 2(d-1)L.$$

式中的第一项是数据传输时间：ring All-Reduce 依次执行 reduce-scatter 和 all-gather，两个阶段各传输 $(d-1)/d$ 的完整数据量；第二项表示两个阶段共 $2(d-1)$ 个通信步的启动时延。对于含 u_i 次

All-Reduce 的 layer，其通信时间为 $u_i T_{\text{AR}}$ ，因而 layer 区间内的 TP 通信总时间为

$$C_{\text{TP}}(i, j, G) = \sum_{\ell=i+1}^j u_{\ell} T_{\text{AR}}(A_{\ell}, d, B_G, L_G).$$

表 8 对文本 layer 取 $u_{\ell} = 2$ ，多设备组的 B_G 取通信路径中最低的相邻链路带宽。Qwen2-VL-7B-Instruct 的视觉编码层使用视觉 token 对应的 activation 大小，并同样按每层两次 All-Reduce 计算。若实际执行采用 Reduce-Scatter 与 All-Gather 组合，后续测量将以 CANN/HCCCL 对应通信原语的实测时间替换上述估算。

PP 不在每个 layer 内同步，而是在相邻 stages 之间传输一次边界 activation。设第 $k-1$ 个 stage 的输出大小为 $A_{l_{k-1}}$ byte，两 stages 之间的带宽和传输时延分别为 $B_{k-1,k}$ Gbps 和 $L_{k-1,k}$ ms，则当前 stage 的输入传输时间为

$$C_k^{\text{in}}(l_{k-1}) = \frac{8A_{l_{k-1}}}{B_{k-1,k} \times 10^9} \times 10^3 + L_{k-1,k}.$$

首个 stage 不接收上一 stage 的 activation，因此 $C_1^{\text{in}} = 0$ 。与 TP 的逐层集合通信相比，包含 K 个 stages 的 PP 每个 micro-batch 只发生 $K-1$ 次边界传输。

将上述计算与通信时间相加，可以得到三种并行方式的 stage 时间：

$$\begin{aligned} t_k^{\text{PP}} &= T_{\text{comp}}(l_{k-1}, l_k, g_k) + C_k^{\text{in}}(l_{k-1}), \\ t^{\text{TP}} &= \frac{C(0, N)}{d \min_{g \in G} F_g} \times 10^3 + C_{\text{TP}}(0, N, G), \\ t_k^{\text{PP+TP}} &= \frac{C(l_{k-1}, l_k)}{d_k \min_{g \in G_k} F_g} \times 10^3 + C_{\text{TP}}(l_{k-1}, l_k, G_k) + C_k^{\text{in}}(l_{k-1}). \end{aligned}$$

TP 将每层的权重张量平均切分到 d 台设备，同一次计算需要等待组内所有设备完成，因此设备组 G 的有效计算速度按 $d \min_{g \in G} F_g$ 计算。PP 的 stage 由一台设备执行，只包含该段 layers 的计算和一次输入传输；PP+TP 的每个 stage 先在组内完成 TP 计算与集合通信，再把结果传给下一 stage。单设备 stage 的 C_{TP} 为 0，首个 stage 的 C_1^{in} 为 0。

理论比较使用同一个请求窗口。设窗口包含 q 个 micro-batches，每个 micro-batch 含 b 个请求，则窗口完成时间和窗口处理速度为

$$T_q = \sum_{k=1}^K t_k + (q-1) \max_k t_k, \quad R_{\text{win}} = \frac{qb \times 10^3}{T_q} \text{ requests/s}.$$

第一个 micro-batch 需要依次经过全部 stages，之后每经过一个最慢 stage 的处理时间即可完成一个新的 micro-batch，因此 $\max_k t_k$ 同时反映 stage 不均衡和 pipeline bubble 对处理速度的影响。纯 TP 只有

一个 stage，此时 $T_q = qt^{\text{TP}}$ ；对于分割方案为 x 的 PP，stage 时间写为 $t_k(x)$ ，对应的窗口完成时间写为 $T_q(x)$ 。当前计算取 $q = 8$ 、 $b = 1$ ，CodeLlama34B 使用 512-token prefill，Qwen2-VL-7B-Instruct 使用 640 个 visual tokens 和 256 个 text tokens。纯 TP 在拓扑允许的设备子集中搜索并行度，纯 PP 搜索连续 layer 边界，PP+TP 则枚举连续设备分组和各组承担的 layer 区间。HCCS 的 56 GB/s 对应 448 Gbps。

以 CodeLlama34B 的 512-token prefill 为例，边界 activation 大小为 $512 \times 8192 \times 2 = 8,388,608$ byte，即 8 MiB。部署前计算尚未引入实测启动时延，因此在表中取 $L = 0$ 。两卡 TP 在 100 Gbps 链路上完成一次 ring All-Reduce 需要 0.671 ms，每个 layer 的两次通信共 1.342 ms；带宽降至 25 Gbps 后，两项时间分别增至 2.684 ms 和 5.369 ms；在 56 GB/s HCCS 链路上，对应时间分别为 0.150 ms 和 0.300 ms。PP 传输同一份 8 MiB activation 时只需完成一次单向传输，因此三种链路上的单个 stage 边界时间分别为 0.671、2.684 和 0.150 ms。

表中的处理速度可以由上述公式复现。

以 CodeLlama34B 为例，一个文本 layer 在 E1 的两张 230 TFP16 设备上进行两卡 TP 时，计算时间为 $717.26 / (2 \times 230) = 1.559$ ms。加上 1.342 ms 集合通信后，每层共需 2.901 ms，48 层的单个 micro-batch 时间为 139.27 ms。包含 8 个 micro-batches 的窗口完成时间为 1114.15 ms，对应 7.18 requests/s。非均匀 PP 的四个 stage 时间为 (56.13, 56.80, 56.80, 38.09) ms，代入流水线公式得到 605.47 ms 和 13.21 requests/s。TP 虽然缩短了矩阵计算时间，但逐层集合通信会在低带宽链路上不断累积；PP 的通信次数更少，并且能够通过非均匀 layer 分割减轻低算力设备形成的瓶颈。实际实验将把测得的链路启动时延、有效带宽和逐层执行时间代入同一组公式。

表 8 TP、PP 与 PP+TP 的部署前窗口处理速度 (requests/s) 及相对 Equal-PP 的提升

模型与场景	Equal-PP	最优 TP	最优 PP	最优 PP+TP	选择及相对提升
CodeLlama34B / E1	5.91	7.18	13.21	12.74	PP / 123.6%
CodeLlama34B / E2	5.82	3.01	12.92	11.80	PP / 121.9%
CodeLlama34B / E3	3.14	3.19	4.45	—	PP / 41.6%
CodeLlama34B / E4	5.45	2.45	9.29	7.55	PP / 70.4%
Qwen2-VL-7B-Instruct / E1	16.19	10.21	27.68	26.48	PP / 71.0%
Qwen2-VL-7B-Instruct / E2	15.70	3.51	26.70	24.21	PP / 70.1%
Qwen2-VL-7B-Instruct / E3	8.63	6.40	9.19	—	PP / 6.5%
Qwen2-VL-7B-Instruct / E4	7.67	3.12	14.38	7.58	PP / 87.5%

表 8 中，最优 TP 会在拓扑连通且能够容纳模型的设备子集中搜索 TP 并行度，并选择窗口完成时间最短的配置。例如，E1 上的 CodeLlama34B 选择两张 230 TFP16 设备组成两卡 TP，窗口处理速度由四卡 TP 的 4.06 requests/s 提高到 7.18 requests/s。移除两张低算力设备后，同步等待明显减少；由于每个 Transformer layer 仍需集合通信，其速度仍低于最优 PP 的 13.21 requests/s。相对 Equal-PP，八组最优 TP 结果的变化范围为 -77.6%~21.5%：CodeLlama34B / E1 提高 21.5%，E3 提高 1.6%，其余场景均受到低速设备等待或逐层集合通信的影响。由此可见，TP 的有效收益仍依赖能力较为接近的设备和较高的组内带宽。

PP+TP 在 E1 上已经接近纯 PP。以 CodeLlama34B 为例, 纯 PP 的设备组织为 1|2|3|4, layer 分配为 (18, 18, 9, 3), stage 时间为 (56.13, 56.80, 56.80, 38.09) ms。PP+TP 的最优组织为 1|2|3+4, layer 分配为 (20, 20, 8)。设备 3 和设备 4 的算力分别为 115 和 57.5 TFP16, 等宽 TP 的有效算力按 $2 \min(115, 57.5) = 115$ TFP16 计算, 同时每个 layer 增加集合通信。减少一个 stage 节省了部分流水线填充和边界传输时间, TP 同步与集合通信则使窗口完成时间从 605.47 ms 增至 628.01 ms, 处理速度从 13.21 requests/s 变为 12.74 requests/s。E2 和 E4 的链路为 25 Gbps, 逐层通信占用的时间进一步增加, 因而 PP+TP 与纯 PP 的差距更明显。

在当前两个模型和四类组网中, 最优 TP 的处理速度为 2.45–10.21 requests/s, 最优 PP 为 4.45–27.68 requests/s, 能够构造的 PP+TP 为 7.55–26.48 requests/s。统一模型最终均选择非均匀 PP, 相对 Equal-PP 的提升范围为 6.5%–123.6%。这些范围描述代表性 prefill 窗口下的部署前结果; 正式选择将进一步使用 decode 和混合请求窗口, 并由 Ascend 逐层执行时间和 CANN/HCCCL 通信测量校准。

4.6.3 非均匀 PP 的问题建模与代价模型

4.6.2 节的理论比较已经确定当前验证环境采用 PP。给定待部署模型和一条按执行顺序连接的 Ascend 设备路径, Adaptive-PP 需要确定各台设备分别承担哪一段连续 layers。分割结果必须满足显存容量、设备连通性和推理运行时支持能力等约束, 并尽量缩短流水线完成时间。最终仍以单位时间内满足 SLA 的请求数评价部署效果, 其中 SLA 包括 TTFT、TPOT、E2E latency 阈值及其 P90/P99 等统计要求。

为计算一个切分方案的代价, 算法需要三类输入。第一类是硬件环境, 用算力拓扑图 $H = (V, E)$ 表示: 每个节点记录设备类型、当前可用算力、可用显存、显存带宽和运行状态, 每条边记录有效带宽、往返时延和连通关系。第二类是模型特征, 包括 layer 数、逐层参数量、prefill/decode 执行时间、activation 大小、单 token KV Cache 增量和推理运行时预留显存; 这些信息由 PyTorch 或 HuggingFace 模型结构分析与离线测量共同获得。对于 VLM, 视觉编码器、模态投影模块和文本解码器分别建模。第三类是业务负载, 包括并发范围、输入输出长度分布以及对应的 SLA 条件。

以 CodeLlama34B 为例, 其 BF16/FP16 权重约为 67.5 GB, 加上 KV Cache、activation 和运行时预留空间后还会进一步增加显存需求。因此, 动态规划在搜索分割位置时同步检查显存约束。

模型包含 N 个顺序执行的 Transformer layers。给定设备路径包含 K 台设备, 并按照 $g_1, \dots, g_K$ 的顺序执行。PP 分割方案表示为

$$x = (l_1, \dots, l_{K-1}).$$

其中, K 既是参与执行的设备数, 也是 PP 的 stage 数; l_k 表示第 k 个 stage 的 layer 右边界。规定 $l_0 = 0$ 、 $l_K = N$ 后, 设备 g_k 负责第 $l_{k-1} + 1$ 层到第 l_k 层。因此, 算法的决策变量只有 $K - 1$ 个 layer

分割点。以 24-layer 模型和三台设备为例，若 $l_1 = 4$ 、 $l_2 = 12$ ，三个 stages 就分别执行 1–4、5–12 和 13–24 层。

区间 $(i, j]$ 放置到设备 g 时，其显存需求记为 $M(i, j, g)$ ，其中包含该区间的模型权重、activation、目标并发下的 KV Cache 和推理运行时预留空间；设备的可用显存记为 $\overline{M}_g$ 。可行方案必须满足

$$0 = l_0 \leq l_1 < \dots < l_K = N, \quad M(l_{k-1}, l_k, g_k) \leq \overline{M}_{g_k}.$$

对于纯文本模型，各 stage 至少包含一个 layer，因此还要求 $l_1 > 0$ 。对于视觉模块固定在第一个 stage 的 VLM，允许 $l_1 = 0$ ，此时第一个 stage 不含文本 layer，但仍执行视觉编码和模态投影并占用相应计算与显存资源。将第 $i + 1$ 层到第 j 层放在设备 g 上时，其计算时间直接沿用 4.6.2 节定义的 $T_{\text{comp}}(i, j, g)$ ，单位为 ms。部署前由区间计算量 $C(i, j)$ 和设备有效算力 F_g 得到，获得硬件后再用逐层实测时间校准。

相邻 stages 在第 i 层之后切分时，需要传输该层输出的 activation。若其大小为 A_i byte，相邻设备 g_{k-1} 与 g_k 之间的有效带宽和固定时延分别为 B_{g_{k-1}, g_k} Gbps 和 L_{g_{k-1}, g_k} ms，则第 k 个 stage 的输入传输时间为

$$C_k^{\text{in}}(i) = \frac{8A_i}{B_{g_{k-1}, g_k} \times 10^9} \times 10^3 + L_{g_{k-1}, g_k}, \quad k \geq 2.$$

首个 stage 的输入传输时间规定为 $C_1^{\text{in}} = 0$ 。将一段连续 layers 分配给第 k 个 stage 的区间代价定义为

$$c_k(i, j) = T_{\text{comp}}(i, j, g_k) + C_k^{\text{in}}(i).$$

对于完整分割方案 x ，第 k 个 stage 的时间为 $t_k(x) = c_k(l_{k-1}, l_k)$ 。每个 stage 分到的计算量和它接收 activation 的时间共同决定流水线速度，这一定义也直接作为下一节动态规划的状态转移代价。若一个请求窗口包含 q 个 micro-batches，第一个 micro-batch 必须依次通过全部 stages；此后，每经过一个最慢 stage 的处理周期，流水线才能再完成一个 micro-batch。因此，完整流水线时间为

$$T_q(x) = \sum_{k=1}^K t_k(x) + (q-1) \max_{1 \leq k \leq K} t_k(x).$$

式中，第一项是流水线首次填满所需的时间，第二项是其余 micro-batches 按瓶颈 stage 周期推进的时间。当 q 较大时，最慢 stage 决定持续处理速度；当 q 较小时，各 stage 时间之和也会明显影响 TTFT 和窗口完成时间。因此，本文直接最小化 $T_q(x)$ ：

$$x_q^* = \arg \min_{x \in \mathcal{X}_{\text{pp}}} T_q(x),$$

其中, $\mathcal{X}_{PP}$ 表示满足显存、设备连通性和推理运行时要求的连续 layer 分割集合。分割点只能设置在相邻的完整 layers 之间。 $T_q(x)$ 用于部署前筛选, 能够同时反映流水线瓶颈和首次填充开销; 真实服务中的排队、动态 batching 和运行时调度由后续端到端实验测量。当业务配置包含多种输入输出长度或不同 prefill/decode 比例时, 对每类代表性窗口分别计算完成时间, 并按请求分布汇总后确定分割点。下文用 x^* 表示依据目标请求分布得到的 PP 分割, x_q^* 是只包含单一代表窗口时的特例。

窗口完成时间用于部署前选择, 在线服务性能则通过请求轨迹重放评价: 对方案 x 以到达率 λ 持续提交具有相同到达时间、输入长度和输出长度的请求, 记长度为 T s 的统计窗口内完成的请求数为 $N_T(x, \lambda)$ 。P99 或 P90 TTFT、TPOT、E2E latency 均满足对应阈值, 且队列保持稳定、运行过程无异常时, 该到达率属于系统可稳定承载的范围。方案 x 的最大 SLA 请求处理速度定义为

$$R_{SLA}(x) = \max_{\lambda \in \Lambda_{SLA}(x)} \frac{N_T(x, \lambda)}{T},$$

其中, $\Lambda_{SLA}(x)$ 为满足上述条件的到达率集合, $R_{SLA}(x)$ 的单位为 requests/s。类似地, 以闭环客户端测试并发时, 满足同一组 SLA 条件的最大并发记为 $n_{SLA}(x)$ 。相对于多节点平均切分方案 x_{equal} , 最终需要验证

$$R_{SLA}(x^*) \geq 1.5R_{SLA}(x_{equal}), \quad n_{SLA}(x^*) \geq 1.5n_{SLA}(x_{equal}).$$

前一个条件要求满足 SLA 的最大请求处理速度提高至少 50%; 后一个条件要求基线在并发 n 下满足 SLA 时, Adaptive-PP 在并发 $1.5n$ 下仍满足相同要求。由此, 部署前最小化 $T_q(x)$ 用于确定 PP 分割点, 硬件实验中的 R_{SLA} 和 n_{SLA} 用于判断最终目标是否达到。

4.6.4 连续 layer 分割的动态规划求解

PP 执行方式和设备路径确定后, 连续 layer 分割位置直接决定各 stage 的负载。给低算力设备分配过多 layers 会形成流水线瓶颈, 忽略显存和边界 activation 大小又可能造成显存超限或较高的跨设备传输开销。动态规划因此沿设备顺序搜索可行的连续 layer 边界, 并选择窗口完成时间最短的分割方案。

EdgeShard 将边缘环境中的连续 layer 切分与 shard placement 建模为动态规划问题 [24]。本文沿用这种顺序分段思路, 将单个 stage 的代价定义为“一台设备执行一段连续 layers”的计算与输入传输时间。算法从算力拓扑中读取给定的连通设备路径, 再用前缀和预先计算任意连续 layer 区间的计算量、参数量、KV Cache 和显存需求; 当 layer 区间放到非首个 stage 时, 还需加入前一设备传入边界 activation 的时间。

图 13 展示了 Adaptive-PP 的整体计算过程。设备算力、显存和链路状态与模型逐层特征、请求负载共同形成区间代价; 动态规划随后按照设备顺序比较不同的前序分割点, 并排除显存超限或链

路不可用的分配。最终输出为一组非均匀 PP 边界，使算力不同的设备承担不同数量的连续 layers。

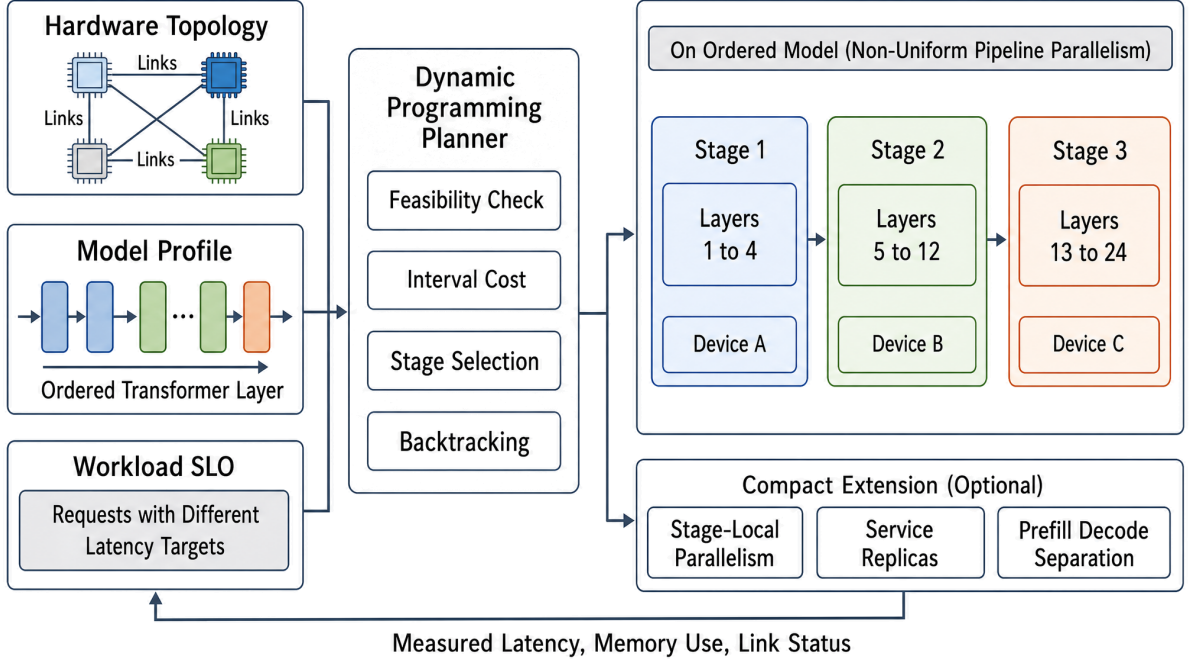


图 13 基于动态规划的非均匀 PP 分割过程

动态规划沿设备顺序分配 layers。对于包含 K 个 stages 的方案， $\mathcal{D}[k, j]$ 保存“前 j 个 layers 已分配给前 k 个 stages”的有效结果。每个结果记录当前最大 stage 时间 b 和所有 stage 时间之和 s 。若前一个分割点位于第 i 层之后，则第 k 台设备承担第 $i + 1$ 层至第 j 层，其计算和输入传输的总时间记为 $c_k(i, j)$ 。加入这一段后，状态更新为

$$(\max\{b, c_k(i, j)\}, s + c_k(i, j)).$$

同一个 $\mathcal{D}[k, j]$ 可能对应多个分割方法。若一种方法的 b 和 s 都不小于另一种方法，它在后续加入任何 stages 后都不可能更优，因此可以删除。显存超限、链路不可用或 layer 区间无法执行的转移也会直接排除。全部 layers 分配完成后，算法在 $\mathcal{D}[K, N]$ 中选择 $s + (q - 1)b$ 最小的结果，再沿前序分割点恢复各 stage 的 layer 数。在设备路径、区间代价、显存约束和 micro-batch 数 q 已经确定时，该搜索会遍历所有合法的连续 layer 边界，得到当前代价模型下的最优 PP 分割。进入验证环境后，逐层执行和边界传输的实测数据将用于校准代价模型并更新分割结果。

对固定设备路径，动态规划的计算量为 $O(KN^2P)$ ，其中 P 是每个状态保留的非劣结果数。当前验证环境最多使用四台设备，模型只有 28 或 48 个可切分文本 layers，显存和运行能力约束还会提前排除大量无效方案，因此完整搜索开销可控。

部署后，系统持续采集各 stage 的 prefill/decode latency、峰值显存和边界传输时间，并据此修正区间代价。当设备的长期可用算力、显存配额或链路状态发生明显变化时，规划器在下一次部署或

维护窗口重新运行动态规划。本文不假设运行时能够在单个请求执行过程中迁移 layers 或即时改变 PP 边界。

4.6.5 非均匀 PP 的收益估算

4.6.2 节的理论比较在当前参数下选择了纯 PP。并行方式确定后，本节进一步考察连续 layers 如何分配给能力不同的设备。具体做法是将动态规划得到的非均匀分割与 Equal-PP 对比，计算 layer 边界调整带来的处理速度变化，并根据已有 SLA 参考点估算并发提高至 $1.5n$ 后的 TTFT。该结果用于判断 PP 分割的优化空间，最终性能由下一节的硬件实验验证。

收益估算沿用 4.6.1 中的四类硬件环境：E1 和 E2 保持相同的四卡异构算力，仅改变跨机链路带宽；E3 使用两张受限算力设备和 HCCS；E4 同时包含 310P、半算力 910B4 与完整算力 910B4。这样既能观察设备能力不均衡造成的 stage 瓶颈，也能比较 25 Gbps、100 Gbps 和 HCCS 对 PP 边界通信的影响 [43]。

测试使用 CodeLlama34B 和 Qwen2-VL-7B-Instruct。

两者的两字节权重规模分别约为 67.5 GB 和 16.6 GB。CodeLlama34B 包含 48 个文本解码层，模型权重已超过单个 910B4 的 64 GB 设备内存。因此，PP 分割既影响部署可行性，也直接决定流水线性能。Qwen2-VL-7B-Instruct 包含 32 个视觉编码 blocks、模态投影模块和 28 个文本解码层。其计算负载还会随图像和文本输入变化，需要分别测量各类模块的执行时间与内存占用。

部署前先根据模型结构和代表性请求计算各 layer 或模块的工作量，再结合设备有效算力、链路带宽和显存容量建立性能画像。Equal-PP 按 layer 数量均分模型，Adaptive-PP 则用动态规划搜索满足资源约束的连续 layer 分割，并将两者代入相同的窗口完成时间模型。以下数值基于设备规格和模型结构计算；进入验证环境后，将以硬件逐层测量得到的性能画像重新计算。

收益估算中的计算量沿用 4.6.2 节的 P_i 和 $C(i, j)$ 。对于 CodeLlama34B，请求 r 的文本长度记为 S_r ，每个 Transformer layer 的计算量由 $P_{\text{layer}}(S_r)$ 得到；当 $S_r = 512$ 时，每层为 717.26 GFLOPs。对于 Qwen2-VL-7B-Instruct，记请求 r 的 visual token 数和 text token 数分别为 V_r 和 S_r ，将 $n_v = V_r$ 代入前面定义的 P_{vision} ，完整 prefill 计算量写为

$$P_{\text{Qwen}}(r) = P_{\text{vision}}(V_r) + \sum_{j=1}^{28} P_j^{\text{text}}(S_r, V_r).$$

其中， $P_{\text{vision}}(V_r)$ 已包含 patch embedding、32 个视觉编码 blocks 和模态合并模块，第二项为 28 个文本解码 layers。对于包含多种请求长度的测试轨迹，先计算每个请求在切分单元 i 上的 FLOPs，再取平均值

$$\bar{P}_i = |\mathcal{R}|^{-1} \sum_{r \in \mathcal{R}} P_i(r),$$

并以 $\bar{P}_i$ 代替 4.6.2 节中的 P_i 计算区间工作量 $C(i, j)$ 。这样, stage k 的计算时间仍由 $T_{\text{comp}}(l_{k-1}, l_k, g_k)$ 给出, 不再另设一套计算时间符号。视觉编码器和模态投影模块整体作为第一个不可拆分单元, 动态规划只改变后续文本 layers 的边界。

边界通信同样沿用 activation 大小 A_i 和输入传输时间 $C_k^{\text{in}}(i)$ 。BF16 文本层边界的元素宽度为 $e = 2$ byte, 因此

$$A_i = 2b S_i h_i.$$

CodeLlama34B 在 $S_i = 512$ 、 $b = 1$ 和 $h_i = 8192$ 时的边界 activation 为 8 MiB。Qwen2-VL-7B-Instruct 的视觉模块、模态投影和文本层具有不同的输出形状, 因此从模型执行图读取各边界的 $A_i(r)$, 再按请求分布求平均。相邻设备带宽 B_{g_{k-1}, g_k} 统一使用 Gbps, 固定时延 L_{g_{k-1}, g_k} 使用 ms, 并代入 4.6.3 节的 $C_k^{\text{in}}(i)$ 公式。由此得到区间代价 $c_k(i, j)$ 、stage 时间 $t_k(x)$ 和窗口完成时间 $T_q(x)$ 。Equal-PP 与 Adaptive-PP 使用完全相同的计算、通信和流水线公式, 两者只在 layer 分割点 x 上存在差异。

记 Equal-PP 的等层数分割为 x_{equal} , 动态规划得到的非均匀分割为 x^* , 则两者的流水线完成时间分别为 $T_{\text{equal}} = T_q(x_{\text{equal}})$ 和 $T_{\text{adaptive}} = T_q(x^*)$ 。当一次执行包含 q 个 micro-batches, 且完成时间以 ms 表示时, 对应的请求处理速度可估计为

$$\hat{R}_{\text{equal}} = \frac{qb \times 10^3}{T_{\text{equal}}}, \quad \hat{R}_{\text{adaptive}} = \frac{qb \times 10^3}{T_{\text{adaptive}}}.$$

在请求分布和调度方式保持一致时, 两者的比值反映 PP 分割变化带来的处理能力差异。为了在尚无排队曲线时初步判断 $1.5n$ 并发是否值得进入硬件测试, 本文采用局部线性近似: 假设参考运行点附近的 TTFT 随“并发请求数与处理能力之比”同比变化。若 Equal-PP 在并发 n 下的第 p 分位 TTFT 为 D_p , 并发提高到 $1.5n$, 而处理能力由 $\hat{R}_{\text{equal}}$ 提高到 $\hat{R}_{\text{adaptive}}$, 则

$$\hat{D}_{p, \text{adaptive}}(1.5n) = D_p \frac{1.5\hat{R}_{\text{equal}}}{\hat{R}_{\text{adaptive}}}.$$

表 10 暂以配置给出的 TTFT 阈值作为 D_p , 对应 Equal-PP 在并发 n 时位于 SLA 边界的估算条件; 获得基线实测值后, 将用实测 TTFT 更新该输入。该近似用于筛选实验场景, 固定执行时间、排队分布、batching 效率和运行时调度将在正式实验中统一测量。负载接近饱和、请求长度差异较大或队列增长呈明显非线性时, 估算误差可能增大, 因此最终 SLA 结论以逐级提高并发后测得的 P90/P99 TTFT 为准。

基于上述模型, CodeLlama34B 的估算取 $S = 512$ 、 $b = 1$ 和 $q = 8$, 并按照实际链路速率计入 activation 序列化时间。设备算力使用前述验证环境给出的有效值, 部署前暂不增加额外的算子效率修正, 即令 $\eta_g = 1$; 获得实测数据后再分别标定各设备的 η_g 。在 (230, 230, 115, 57.5) TFP16 的四

设备路径上，Equal-PP 的分割为 (12, 12, 12, 12)。100 Gbps 链路下，动态规划得到 (18, 18, 9, 3)，流水线完成时间由 1353.91 ms 降至 605.47 ms；按 $8 \times 10^3 / T$ 换算，处理速度由 5.91 requests/s 提高到 13.21 requests/s，相对提升为 $(13.21 / 5.91 - 1) \times 100\% = 123.6\%$ 。25 Gbps 链路下，最优切分变为 (19, 18, 9, 2)，完成时间由 1374.04 ms 降至 619.28 ms，处理速度提高 121.9%。受限两卡 HCCS 路径上，分割由 (24, 24) 调整为 (32, 16)，完成时间由 2545.91 ms 降至 1797.47 ms，处理速度提高 41.6%。在 (70, 115, 230) TFP16 的三设备路径上，分割由 (16, 16, 16) 调整为 (8, 13, 27)，25 Gbps 链路下的完成时间由 1466.62 ms 降至 860.81 ms，处理速度提高 70.4%。

Qwen2-VL-7B-Instruct 的估算同时计入视觉编码器、模态投影模块和 28 个文本解码层。代表性请求包含 640 个 visual tokens 和 256 个 text tokens，视觉编码器与模态投影模块整体放置在第一个 stage，动态规划在扣除该固定模块开销后搜索文本层的分割点。四卡 100 Gbps 和 25 Gbps 路径的最优文本层分割分别为 (4, 14, 7, 3) 和 (5, 14, 7, 2)，处理速度预计提高 71.0% 和 70.1%。三设备 25 Gbps 路径的最优分割为 (0, 1, 27)，第一个 stage 主要承担视觉编码与模态投影，预计提升为 87.5%。在 (115, 57.5) TFP16 的两卡 HCCS 路径上，最优文本层分割为 (16, 12)，完成时间仅由 926.94 ms 降至 870.59 ms，对应的处理速度提升为 6.5%。两卡路径中的可调整边界较少，异构算力带来的不均衡也相对有限，因此重新分配文本层所能获得的收益明显小于四卡和三设备路径。

表 9 汇总了动态规划得到的最优 layer 分割方案。搜索只允许在相邻的完整 layers 之间设置分割点，并检查设备顺序、链路和当前可获得的显存信息；获得完整运行时显存画像后，还需要再次检查目标并发下的 KV Cache、activation 和预留空间。表中结果给出通过当前结构与权重容量检查的 PP 边界及其部署前性能估算，硬件验证将在实测性能画像和完整运行时环境下进一步确认。对于已有参考并发和 TTFT 阈值的配置，再利用前述关系估算并发增至 $1.5n$ 后的 TTFT，结果见表 10。E4 的 SLA 参考点将在正式测试中获得：先运行 Equal-PP，测得其最大稳定并发 n 和最大 SLA 请求处理速度，再在相同条件下测试 Adaptive-PP。

表 9 动态规划得到的最优 layer 分割方案与请求处理速度估算

模型与场景	动态规划最优切分	Equal-PP / requests/s	Adaptive-PP / requests/s	相对提升 / %
CodeLlama34B / E1	(18, 18, 9, 3)	5.91	13.21	123.6%
CodeLlama34B / E2	(19, 18, 9, 2)	5.82	12.92	121.9%
CodeLlama34B / E3	(32, 16)	3.14	4.45	41.6%
CodeLlama34B / E4	(8, 13, 27)	5.45	9.29	70.4%
Qwen2-VL-7B-Instruct / E1	(4, 14, 7, 3)	16.19	27.68	71.0%
Qwen2-VL-7B-Instruct / E2	(5, 14, 7, 2)	15.70	26.70	70.1%
Qwen2-VL-7B-Instruct / E3	(16, 12)	8.63	9.19	6.5%
Qwen2-VL-7B-Instruct / E4	(0, 1, 27)	7.67	14.38	87.5%

表中的预计提升由动态规划得到的最优分割方案相对 Equal-PP 计算，直接对应当前性能模型中能够部署的 layer 分割。CodeLlama34B 四条路径的预计提升为 41.6%–123.6%；Qwen2-VL-7B-Instruct 的预计提升为 6.5%–87.5%。这些数值仍属于部署前估算，正式硬件结果还会受到算子效率、内存访问、链路协议和推理运行时调度开销影响。

表 10 基于 SLA 边界与局部线性近似的 $1.5n$ 并发 TTFT 初步估算

模型与场景	TTFT SLA / ms	并发请求数 $n \rightarrow 1.5n$	$1.5n$ 下预计 TTFT / ms
CodeLlama34B / E1	P99 150 ms	8 到 12	100.6 ms
CodeLlama34B / E1	P99 500 ms	32 到 48	335.4 ms
CodeLlama34B / E1	P90 100 ms	16 到 24	67.1 ms
CodeLlama34B / E1	P90 180 ms	32 到 48	120.7 ms
CodeLlama34B / E1	P90 500 ms	64 到 96	335.4 ms
CodeLlama34B / E2	P99 1000 ms	16 到 24	676.1 ms
CodeLlama34B / E2	P90 500 ms	16 到 24	338.0 ms
Qwen2-VL-7B-Instruct / E3	P99 520 ms	8 到 12	732.6 ms
Qwen2-VL-7B-Instruct / E3	P90 500 ms	16 到 24	704.4 ms
Qwen2-VL-7B-Instruct / E1	P99 610 ms	8 到 12	535.1 ms
Qwen2-VL-7B-Instruct / E1	P90 820 ms	64 到 96	719.3 ms

表 10 给出了并发提高到 $1.5n$ 后的初步筛选结果。按照前述近似, 当 $\hat{R}_{\text{adaptive}}/\hat{R}_{\text{equal}} > 1.5$ 时, 处理能力增幅超过并发增幅, 估算 TTFT 会低于 Equal-PP 在 n 并发下的参考值。CodeLlama34B 在 E1 和 E2 上满足这一条件, Qwen2-VL-7B-Instruct 在 E1 上也具有进一步测试的空间。两卡 E3 的处理速度提高 6.5%, 低于 50% 的并发增幅, 因此该场景应优先测试其它组网或并行方式。表中数值用于安排实验优先级; 获得逐层实测性能画像后, 将更新计算效率、传输时延和边界 tensor 大小, SLA 达标情况以完整推理运行时中的请求轨迹重放结果为准。

4.6.6 实验设计与评价指标

硬件实验从请求处理速度和并发承载能力两个方面验证所选方案。前者考察在相同 SLA 下, 单位时间内可稳定完成的请求数能否相对 Equal-PP 提高 50%; 后者考察 Equal-PP 在并发 n 下满足 SLA 时, 所选方案能否在并发提高至 $1.5n$ 后保持相同要求。记两种方案的最大稳定请求处理速度为 R_{equal} 和 $R_{\star}$, 最大 SLA 并发为 n_{equal} 和 $n_{\star}$, 则目标为

$$R_{\star} \geq 1.5R_{\text{equal}}, \quad n_{\star} \geq 1.5n_{\text{equal}}.$$

当前计算选择纯 PP, 其 layer 边界由 Adaptive-PP 给出。TP 和 PP+TP 在硬件实验中作为并行方式对照, 使用相同模型、设备路径和请求配置测量。

实验首先通过微基准获得两个模型在不同输入长度、batch size 和设备上的逐层执行时间, 并测量不同消息大小下的 CANN/HCCl 集合通信与点到点传输时间。利用这些数据更新区间代价后, 重新计算 TP、PP 和 PP+TP 的理论结果。随后将选出的方案接入完整推理服务, 在相同请求轨迹和 SLA 条件下进行端到端比较。

正式实验同时保留并行方式与 PP 分割算法两组对照。对于目标并发下能够单设备运行的模型, 先报告单设备结果作为容量与性能参考; 多设备并行方式对照包括纯 TP、PP+TP 和 Adaptive-PP, PP 内部对照包括 Equal-PP、Greedy-PP 和 Adaptive-PP。Equal-PP 按 layer 数均分; Greedy-PP 从 Equal-PP 出发, 每次尝试把当前瓶颈 stage 边界上的一个 layer 移交给相邻 stage, 选择使 T_q 下降最多且满足显

存与链路约束的移动，直到没有单步移动能够继续改进；Adaptive-PP 则在给定设备路径上用动态规划搜索全部合法的连续 layer 分割点。所有方法使用相同的模型、数值精度、可用设备资源、链路和请求序列。该结果用于比较逐层集合通信、混合并行通信、平均分割和局部搜索分别与 Adaptive-PP 相差多少。

工作负载覆盖不同的输入长度、输出长度和并发水平。对于 Qwen2-VL-7B-Instruct，额外改变图像分辨率和图像数量，以反映 visual token 数量变化对第一个 stage 的影响。每组配置至少使用五个随机种子重复测试，并报告均值、标准差和 95% 置信区间。

最大 SLA 请求处理速度通过固定到达率的请求轨迹重放测量。实验采用开环请求提交，并在相同请求分布和统计时长下逐级提高到达率。每个到达率经过预热后进入固定统计窗口；当 P99/P90 TTFT、TPOT 和 E2E latency 均处于相应阈值内，队列长度保持稳定，且未发生 OOM 或运行时异常时，记录该点的实际 rps。持续提高到达率后，分别得到 Equal-PP 和所选方案的最大稳定值 R_{equal} 与 $R_{\star}$ 。请求处理速度的相对变化按

$$G_{\text{SLA}} = \frac{R_{\star} - R_{\text{equal}}}{R_{\text{equal}}} \times 100\%$$

计算。

并发承载能力采用闭环客户端测量。Equal-PP 运行在参考并发 n ，所选方案将并发提高到 $1.5n$ ；每个客户端在前一个请求完成后再发送下一个请求。两种方案使用完全相同的请求属性序列，并保留所有进入系统的请求。实验记录 P99/P90 TTFT、TPOT、平均 E2E latency、峰值显存和运行异常，从而观察并发提高 50% 后各项 SLA 指标的变化。

最终结果同时报告 Equal-PP 的 n 和 R_{equal} 、所选方案在 $1.5n$ 并发下的实际时延和 $R_{\star}$ ，以及请求处理速度提升 G_{SLA} 。算法比较进一步报告 TP 集合通信、PP 边界通信、各 stage 的计算时间、流水线空泡时间、layer 分割点和峰值显存，用于解释 TP、PP 与 PP+TP 在不同组网中的性能差异。

4.7 小结

本章围绕“一次推理中首先被切分的对象”组织 CEC 下的资源自适应模型切分部署方法。Layer-wise / Vertical Partitioning 关注 layer、block 或 model shard 如何跨节点展开；Tensor / Operator-level Partitioning 关注同一层内部的 tensor、head、MLP 或通信 primitive 如何并行；Expert / Component-level Partitioning 关注 MoE expert 等组件如何驻留、换入和跨节点访问；Inference Phase Partitioning 关注 prefill、decode、verification、download 等阶段如何匹配异构资源；Semantic / Token-level Partitioning 则关注 draft、verify、semantic segment 或 token-wise responsibility 等生成职责如何拆分。该分类以系统优化对象为依据，不受各论文具体术语的限制。

这五类方法之间存在明显的互补关系。层级切分和张量/算子级切分主要解决模型容量、显存和并行度问题；专家/组件级切分更适合 MoE 等具有稀疏激活结构的模型；阶段切分用于缓解 prefill 与 decode 在计算和访存需求上的差异；语义/token 级切分则把协同推理推进到生成过程内部。CEC 场景需要综合模型结构、节点能力、链路状态、KV 位置、业务 SLO 和隐私边界选择切分粒度。

面向华为验证环境，本章使用统一代价模型比较 TP、PP 与 PP+TP。模型同时读取设备有效算力、显存、链路带宽、逐层计算量、集合通信和边界 activation 大小。代表性 prefill 窗口的部署前计算在四类组网中均选择 PP，Adaptive-PP 随后在给定设备路径上通过动态规划确定连续 layer 边界。100 Gbps 四卡路径上的 PP+TP 与 PP 较为接近，而 25 Gbps 路径上的逐层 TP 通信开销更明显。

正式实验将纯 TP、最佳 PP+TP、Equal-PP、Greedy-PP 和 Adaptive-PP 放在相同模型、设备和请求轨迹下比较。CodeLlama34B 按 48 个文本层计算，Qwen2-VL-7B-Instruct 同时计入视觉编码、模态投影和 28 个文本层。获得逐层执行和 CANN/HCCCL 通信数据后重新完成理论选择，再通过完整推理运行时检验所选方案能否使满足 SLA 的最大请求处理速度提高 50%，并在基线并发的 1.5 倍负载下继续满足相同 SLA。

五、CEC 下的业务感知 Batching 参数优化方法

LLM serving 中的 batching 不再只是传统 DNN 推理里的固定 batch size 选择。一次 LLM 请求通常先经历 prefill，再进入逐 token decode；请求到达时间、prompt 长度、输出长度、SLO、KV Cache 占用和上下文复用机会都会随时间变化。因此，batching 的核心问题可以定义为：在给定请求队列、实例负载、KV 状态和协作节点间链路条件下，动态决定哪些请求进入 batch、batch 在什么粒度更新、prefill 与 decode 是否混合、长 prompt 是否切块、不同阶段是否解耦，从而在 TTFT、ITL、TPOT、E2E latency、throughput、goodput 和资源利用率之间取得平衡。Orca 将生成式 Transformer serving 的调度粒度推进到 iteration level [1]；vLLM 通过 PagedAttention 为 continuous batching 提供了 KV 内存管理基础 [2]；Sarathi-Serve 代表 chunked prefill 路线 [3]，DistServe 则代表 prefill/decode disaggregation 路线 [4]。在 CEC 场景中，这个问题进一步复杂化。单个请求入口附近的节点或服务实例通常只有较小的请求池，单纯等待大 batch 会破坏 TTFT；边缘 GPU/NPU 显存有限，KV Cache 会限制可并发 decode 请求数；终端接入链路和边缘节点间链路状态会影响 prefill offloading、decode placement 和状态传输收益；不同业务还可能具有差异化 SLO，例如机器人控制、车联网辅助决策、工业现场问答和普通对话服务的最大等待时间并不相同。基于这些特点，本章以 batching 机制及 batch 更新粒度为分类主线，并将 SLO、KV Cache、多租户和推测验证等因素作为贯穿各类方法的调度约束。这些横向因素分别影响请求接纳、batch 组成、KV 管理和执行顺序。例如，SOLA/JITServe 根据 SLO 约束控制请求接纳，vLLM 为 KV-aware continuous batching 提供内存管理基础，S-LoRA/Punica 中的多租户 adapter serving 会改变 batch 的组成方式和执行算子，AdaServe 和 MuxServe 则分别引入推测验证与多模型时空复用条件下的批处理决策。本文将 CEC 下的业务感知 batching 参数优化划分为四类：

1. **Queue-level Dynamic / Admission Batching**：batch 在请求进入执行前根据到达情况、等待窗口、优先级、距离 deadline 的剩余时间或 admission threshold 动态组成。
2. **Continuous Batching / Iteration-level Batching**：以 decode iteration 或 token iteration 为边界动态维护 active batch，使完成请求退出、新请求或被抢占请求加入。
3. **Chunked Prefill / Blended Batching**：把长 prefill 切成 chunk，并与 decode token 或其他请求混合执行，缓解长 prompt 对 decode 的阻塞。
4. **Disaggregated Prefill-Decode Batching**：将 prefill 和 decode 分配给不同实例、资源池或协作节点，分别形成阶段 batch，并通过 KV/state transfer 衔接。

这四类方法从 batch 的形成与更新粒度描述不同的批处理组织方式，彼此之间可以组合使用。本文以固定参数、按到达顺序形成批次的 Static-FIFO 作为实验基线；四类主体方法则分别面向请求接纳与动态组批、迭代级 active batch、prefill chunk 混合和 prefill/decode 阶段解耦。Queue-level dynamic/admission batching 可以引入 SLO-aware priority；Continuous batching 可以结合抢占、KV-aware

admission 和 active set 调整, Orca [1] 和 FastServe [44] 分别体现了 iteration-level scheduling 与 preemptive scheduling 的代表路线; Chunked Prefill / Blended Batching 通常建立在 continuous batching runtime 之上, Sarathi-Serve [3] 是这一方向的典型系统; Disaggregated Prefill-Decode Batching 则将两个阶段的 batch 分配到不同资源池, DistServe [4] 是阶段解耦路线的代表。

表 11 汇总了与 batching 执行机制直接相关的代表工作以及会改变批处理决策的横向机制; 图 14 给出了四类方法在批次形成粒度上的分类主线。SLO-aware、multi-tenant、KV-aware、speculative verification、multi-model serving 和 inference/finetuning co-serving 贯穿上述分类, 并通过请求优先级、batch 组成、显存占用和资源竞争影响调度结果。LoRA serving、多模型 serving 和 inference/finetuning co-serving 的关键问题在于, 如何组织不同 adapter、模型或任务类型共享 batch 与 GPU 时间片, 并在 SLO、显存和阶段干扰约束下确定执行顺序, 因此也被纳入比较。

表 11 CEC 下业务感知 Batching 参数优化方法相关工作

序号	代表工作	发表在哪	主归属	相关机制	方法关键词
1	JITServe [45]	NSDI 2026	Queue-level Dynamic / Admission Batching	SLO-aware request admission	根据请求 SLO deadline、剩余时间预算和当前服务带宽动态决定接纳、排序和组批
2	SOLA [46]	MLSys 2025	Queue-level Dynamic / Admission Batching	SLO-aware scheduling; phase-aware scheduling	根据成本模型预测 TTFT/TPOT 相对 SLO 的紧迫程度, 对等待请求排序并决定谁优先进入 batch
3	QLM [47]	AIOps@ ASPLOS 2024	Queue-level Dynamic / Admission Batching	Queue management	以满足端到端延迟 SLO 为目标, 通过队列控制减少 head-of-line blocking 并提高 SLO attainment
4	TimelyLLM [48]	MobiSys 2026	Queue-level Dynamic / Admission Batching	time utility; iteration-level control	面向时间敏感应用, 将 deadline 和 time utility 纳入 batch 接纳与请求优先级
5	Preble [49]	ICLR 2025	Queue-level Dynamic / Admission Batching	KV-aware admission; distributed prompt scheduling	在分布式 serving 中把 prefix/KV cache 命中和负载均衡共同纳入请求分配, 使共享 prompt 请求更容易进入可复用状态的实例
6	Fairness in Serving Large Language Models [50]	OSDI 2024	Queue-level Dynamic / Admission Batching; Continuous / Iteration-level Batching	multi-tenant fairness; VTC	在 continuous batching 机制上引入 Virtual Token Counter, 用已服务 token 成本调整调度顺序, 减少租户之间的服务不公平
7	Orca [1]	OSDI 2022	Continuous / Iteration-level Batching	selective batching	提出 iteration-level scheduling, 新请求在 decode iteration 边界加入, 完成请求及时退出 active batch
8	FastServe [44]	NSDI 2026	Continuous / Iteration-level Batching	preemptive scheduling	在 continuous batching 基础上引入迭代级抢占, 缓解长请求占用和尾延迟问题
9	TokenFlow [51]	EuroSys 2026	Continuous / Iteration-level Batching	streaming token scheduling	以 token stream / scheduling interval 为控制粒度, 动态做 admission、preemption 和 resumption
10	vLLM [2]	SOSP 2023	Continuous / Iteration-level Batching	KV block memory management	PagedAttention 用逻辑/物理 KV block 映射降低显存碎片, 为更大的 active batch 提供内存基础
11	SGLang [5]	NeurIPS 2024	Continuous / Iteration-level Batching	RadixAttention; prefix reuse	通过 RadixAttention 自动匹配和复用共享前缀 KV, 使复杂多轮、RAG 和 agent workload 的 active batch 更少重复 prefill

12	Llumnix [52]	OSDI 2024	Continuous / Iteration-level Batching	runtime rescheduling; live migration	在多个 vLLM 实例之间迁移运行中请求及其内存状态，缓解 active batch 负载不均、优先级变化和尾延迟问题
13	Punica [53]	MLSys 2024	Continuous / Iteration-level Batching	multi-tenant LoRA batching	用 SGMV 等算子支持不同 LoRA adapter 请求共享同一基础模型 batch，降低多租户 LoRA serving 的显存和计算浪费
14	S-LoRA [54]	MLSys 2024	Continuous / Iteration-level Batching	heterogeneous adapter batching; unified paging	通过统一分页管理 adapter 权重和 KV cache，并用定制 LoRA 算子支持大量 adapter 请求在同一 serving 实例中批处理
15	NanoFlow [55]	OSDI 2025	Continuous / Iteration-level Batching; Chunked Prefill / Blended Batching	nano-batch scheduling	将执行拆成 nano-batches 并搜索数量、大小和顺序，同时对长 prefill 做 token 粒度分块
16	Sarathi-Serve [3]	OSDI 2024	Chunked Prefill / Blended Batching	chunked prefill; decode interleaving	将长 prefill 切成 chunk，并与 decode 混合交错执行，平滑两阶段负载
17	Libra [56]	NSDI 2026	Chunked Prefill / Blended Batching	phase quota control	在混合批次中调控 prefill 与 decode token 配额比例，降低两阶段算子干扰
18	A Hybrid Online and Offline Requests Inference Serving System for LLM in Private Computer Environment [57]	IEEE TC 2026	Chunked Prefill / Blended Batching	online/offline mixed batching	在同一 continuous batch 内组织在线 prefill/decode 与离线任务，在线优先、离线填充空闲资源
19	LLMStation [58]	USENIX ATC 2025	Chunked Prefill / Blended Batching	iteration-level multitasking; inference/tuning co-serving	将 fine-tuning 任务改造成可挂起流水线，并在 iteration 粒度批处理 inference 与 tuning 请求，以提高 GPU 利用率同时满足 serving SLO
20	FlexLLM [59]	NSDI 2026	Chunked Prefill / Blended Batching	token-level co-serving; SLO-aware scheduling	将 inference 和 PEFT finetuning 融合到 token 级执行，并优先保护 latency-sensitive inference 的 SLO，再利用空闲 token 预算推进 finetuning
21	AdaServe [60]	EuroSys 2026	横向机制: Speculative Verification Batching	multi-SLO speculative decoding	按请求 SLO 定制 speculation tree，并通过 speculate-select-verify pipeline 形成验证批处理；它主要补充 multi-SLO 场景下的验证调度，而不是 prefill chunking
22	DistServe [4]	OSDI 2024	Disaggregated Prefill-Decode Batching	phase-specific workers	将 compute-heavy prefill 与 memory/latency-sensitive decode 解耦到不同 worker pool
23	Splitwise [29]	ISCA 2024	Disaggregated Prefill-Decode Batching	prompt pool; token pool	将 prompt computation 和 token generation 放到不同机器池，并分别匹配硬件资源
24	Mooncake [13]	USENIX FAST 2025	Disaggregated Prefill-Decode Batching	distributed KV cache; global scheduling	在 prefill/decode 解耦基础上构建分布式 KVCache，并按 KV 本地性和 SLO 调度实例
25	MuxServe [61]	ICML 2024	横向机制: Multi-model Multiplexed Batching	adaptive batch scheduling; spatial-temporal multiplexing	面向多 LLM serving 联合做空间共置和时间复用，并利用 prefill/decode 阶段特征设计 adaptive batch scheduling，以提升多模型场景下的 SLO 吞吐

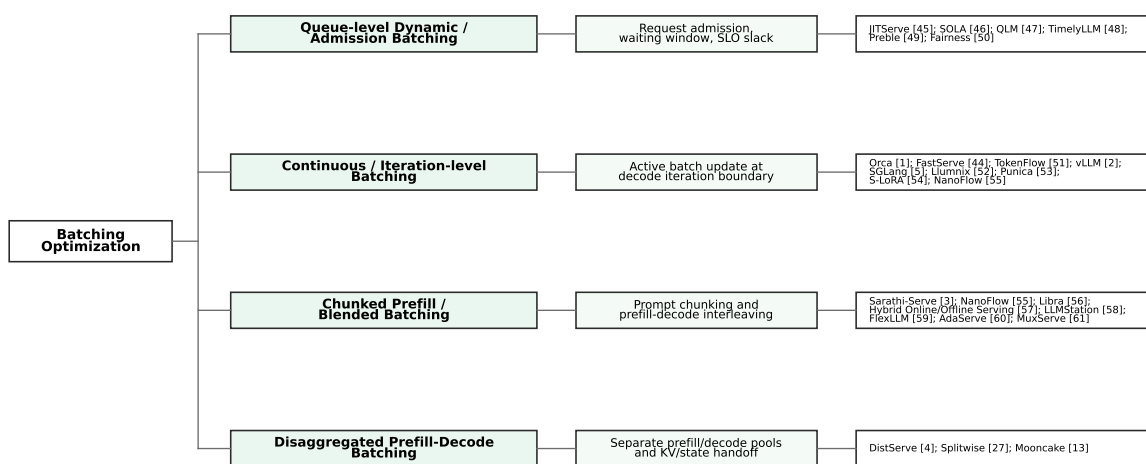


图 14 CEC 下业务感知 Batching 参数优化方法的分类树

5.1 Queue-level Dynamic / Admission Batching

Queue-level Dynamic / Admission Batching 要解决的是“固定等待固定 batch size 过于僵硬”的问题。真实 LLM 服务中，请求到达是随机的，prompt 长度、输出长度和 SLO 也不一致。如果系统仍按固定 batch size 或固定时间窗口组批，低负载时会等待过久，高负载时又可能把紧急请求排在队尾。因此，这类方法在请求进入执行前动态决定 batching window、batch membership、admission threshold 和请求优先级。JITServe 从 SLO-aware serving 角度建模请求服务带宽 [45]，SOLA 以 state-aware scheduling 提高 SLO attainment [46]，QLM 关注队列管理和 head-of-line blocking [47]，TimelyLLM 则面向时间敏感机器人应用组织请求调度 [48]。Preble 说明 prefix/KV locality 会改变请求分配 [49]；Fairness in Serving Large Language Models 则说明多租户公平性也会改变请求排序和 token 服务顺序 [50]。

这种队列级决策可以抽象成图 15 中的闭环：请求先等待，控制器再根据剩余时间预算决定立即执行还是继续合批。Incoming Requests 携带 SLO 和 priority 进入 Waiting Queue，Admission Controller 持续读取距离 deadline 的剩余时间、wait time、service estimate 和 queue length。临近 deadline 的请求进入 Urgent Batch，剩余时间较充足的请求进入 Throughput Batch 或继续等待；两个可执行 batch 随后进入 runtime。执行产生的 TTFT、E2E latency 和 SLO violation 由 Metrics 模块反馈给控制器，用于调整 batching window 与 admission threshold。Dynamic batching 会同时改变 batch size、请求组成和等待时间，并围绕各请求的剩余时间预算在线决策。

这类方法的核心思想是把组批从静态规则改为在线决策。调度器可以根据队列长度、请求等待时间、距离 deadline 的剩余时间、prompt 长度、预计输出长度、业务优先级和当前实例负载决定是否立即发起 batch，还是继续等待更多请求。对于剩余时间较充足的请求，可以适当等待以提高吞吐；对于即将违反 TTFT 或 deadline 的请求，应尽快进入执行队列。与 static batching 相比，dynamic batching

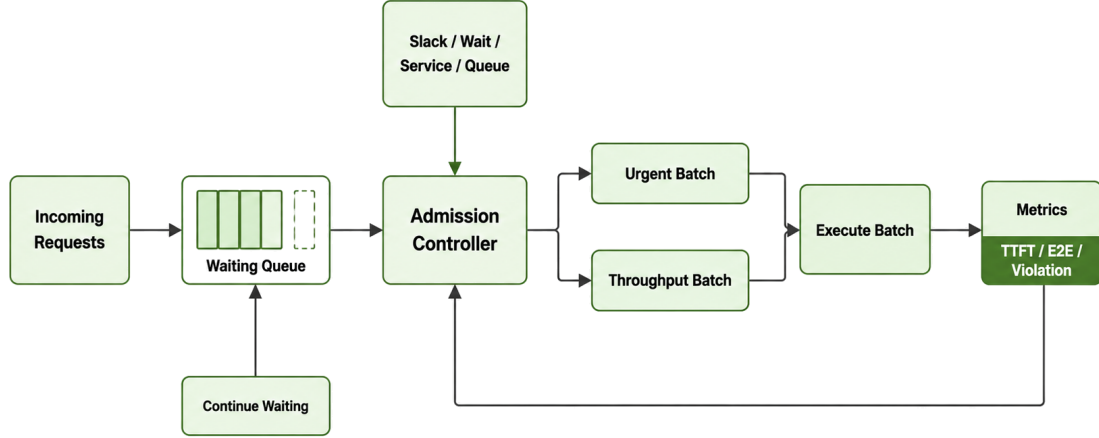


图 15 Queue-level Dynamic / Admission Batching 机制示意图

的 batch 仍然主要在执行前形成，但形成过程已经具备业务感知能力。

Queue-level Dynamic / Admission Batching 的关键变量包括 batching window、max wait time、admission threshold、request priority、距离 deadline 的剩余时间、estimated service time 和 batch size limit。很多 SLO-aware request batching 工作可以放在这一类中理解：它们并不一定改变 decode iteration 的 active batch 机制，而是先在请求队列和接纳阶段决定谁应该进入下一批执行 [45], [46]。

在 CEC 中，dynamic batching 需要把网络时延纳入剩余时间预算。靠近请求入口的实例通常 RTT 较低，但请求池较小；能够聚合多个入口的协作节点请求池更大，但转发距离和排队时间也可能增加。因此，同一个请求应在入口附近立即小批次执行，还是转发到其他协作节点等待更合适的 batch，取决于剩余 SLO、入口到实例链路、当前排队时间和模型能力。Dynamic batching 是 CEC 中自然的业务感知入口，因为它在请求尚未进入长期 decode 循环前就可以完成接纳、排序和路径选择。

Queue-level Dynamic / Admission Batching 的风险在于调度器需要可靠估计请求服务时间。如果输出长度预测错误、队列状态滞后或网络时延估计不准，batching window 和优先级选择可能反而放大尾延迟。因此，在 CEC 中应避免过长等待窗口，并将 admission control、优先级老化和 fallback 策略作为配套机制。

5.2 Continuous Batching / Iteration-level Batching

Continuous Batching 进一步打破了 batch 生命周期。它要解决的是 LLM decode 阶段中请求完成时间不同导致的空槽浪费问题。传统 static 或 dynamic batching 即使在执行前能组成合适 batch，一旦进入

自回归 decode，短请求会先完成，长请求还在继续生成。如果 batch 在整个生成过程中固定不变，已完成请求留下的 slot 无法被新请求利用，新到请求也必须等待当前 batch 全部结束。Orca 首先系统化提出 iteration-level scheduling 和 selective batching，成为后续 continuous batching 系统的基础 [1]。

当调度边界从请求级下降到 decode iteration 后，batch 的含义就变成图 16 中持续变化的 active set。图中每个 Iteration 表示一次 decode iteration，Active Batch 中的四个序列共同生成下一个 token。图中每个 Iteration 表示一次 decode iteration，Active Batch 中的四个序列共同生成下一个 token。

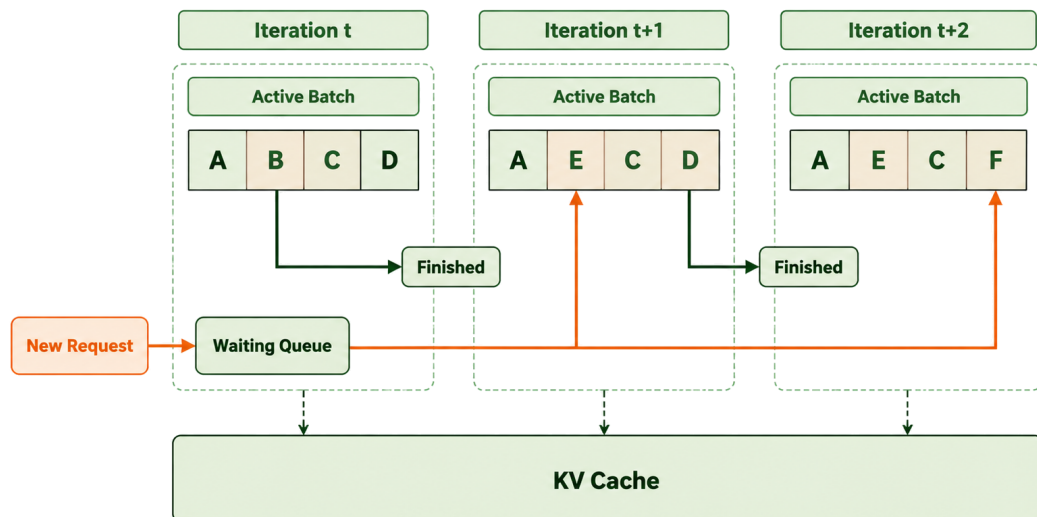


图 16 Continuous Batching / Iteration-level Batching 机制示意图

在 Iteration t 结束后，请求 B 完成并退出，请求 E 从 Waiting Queue 加入，因此下一轮的 active set 由 A、B、C、D 变为 A、E、C、D；到 Iteration $t+1$ 结束后，请求 D 退出、请求 F 加入，active set 进一步变为 A、E、C、F。A 和 C 连续保留在三个迭代中，其 KV Cache 也持续增长。该过程说明，continuous batching 会在迭代边界及时回收完成请求留下的 slot，并用等待请求补充空位，而不必等待整个 batch 结束。

Continuous Batching 的解决思路，是把 batch 的更新边界缩短到 decode iteration。每生成一轮 token 后，系统都可以移除已经完成的请求，加入完成 prefill 的新请求，或者恢复被抢占的请求。这样 batch 不再是一组固定请求，而是一个随 token 生成持续变化的 active set。Orca 将这一思想形式化为 iteration-level scheduling，使 LLM serving 从“按完整请求批处理”转向“按 token iteration 批处理” [1]；FastServe 在这一基础上引入迭代级抢占 [44]，TokenFlow 面向 streaming token 服务做 preemptive scheduling [51]，NanoFlow 则用 nano-batch execution 进一步细化执行流 [55]。

这类方法的核心变量包括 active sequence set、decode batch size、token iteration order、join policy、leave policy、preemption policy、resume policy 和 KV block allocation。由于每轮 decode 都要读取并追

加 KV Cache, continuous batching 必须和内存管理绑定: 请求能否加入 active batch, 不仅取决于是否有计算 slot, 也取决于是否有足够 KV block 和状态本地性。vLLM 的 PagedAttention 正是从 KV block 管理角度支撑更大的 active batch 和更低的显存碎片 [2]; SGLang 用 RadixAttention 复用共享前缀 [5]; Llmunix 则把运行中请求迁移纳入多实例调度 [52]。这些工作说明, continuous batching 的 active set 会进一步受到 prefix cache 和实例负载的共同约束。

多租户 LoRA serving 也可以从 continuous batching 角度理解。Punica [53] 和 S-LoRA [54] 让使用不同 adapter 的序列共享同一个基础模型执行批次, 并处理 adapter 权重分页、异构 adapter batching 和定制 LoRA 算子等问题。这些机制决定 batch 内请求能否合并执行, 以及合并后的显存和算子开销。

在 CEC 中, Continuous Batching 不能简单照搬云端大请求池策略。靠近请求入口的边缘实例并发通常较低, 短时间内能加入 active batch 的请求有限; 如果为等待更多 token stream 而延长调度周期, TTFT 会恶化。因此, 局部实例更适合短窗口、低等待的 micro-continuous batching, 能够聚合多个入口的协作节点则可以形成更稳定的 active batch。对于移动用户, 还需要考虑请求是否会在 decode 期间切换入口, 避免 active batch 中的 KV 状态和用户路径频繁迁移。

Continuous Batching 是现代 LLM serving 的基础能力, 但不是完整的业务感知调度。它提升 decode 阶段资源利用率, 却不天然感知 SLO、应用等级、KV 状态迁移、边缘链路和隐私约束。因此, 在 CEC 中应把 Continuous Batching 视为 runtime 基座, 再叠加 dynamic priority、phase-aware interleaving 和状态感知 admission。

5.3 Chunked Prefill / Blended Batching

Chunked Prefill / Blended Batching 要解决的是长 prefill 阻塞 decode 的问题。Prefill 处理完整 prompt, 矩阵规模大、并行度高, 通常更偏 compute-bound; decode 逐 token 生成, 频繁访问 KV Cache, 更偏 memory-bound 或 latency-bound。如果长 prompt prefill 连续占用 GPU, 已经在流式输出的请求会出现 ITL 抖动; 如果完全优先 decode, 新请求 TTFT 又会被推迟。两阶段混部需要确定 prefill 和 decode 在同一执行资源上的交错方式。Sarathi-Serve [3] 将长 prefill 切成 chunks 并与 decode 交错, 是这一类方法的代表性工作。

长 prompt 不能直接整段 prefill 的原因, 可以通过图 17 看到: 系统需要把 prefill chunk 和 decode token 交错安排, 在 TTFT 与 ITL 之间控制阶段干扰。图中 Long Prompt 被拆成三个 prefill chunks, 并与 Active Sequences 的 decode steps 交替进入六个连续的调度时隙。Phase Controller 通过 chunk size 和 prefill/decode ratio 控制每次执行的 prefill 工作量以及两类阶段的交错频率: chunk 较大时能够提高 prefill 效率, 但可能拉长 decode 间隔; chunk 较小时更有利于稳定 ITL, 却会增加调度和 kernel 启动开销。因此, 该机制通过有限粒度的切块与交错, 在保持 decode 流畅性的同时控制新请求的 TTFT。

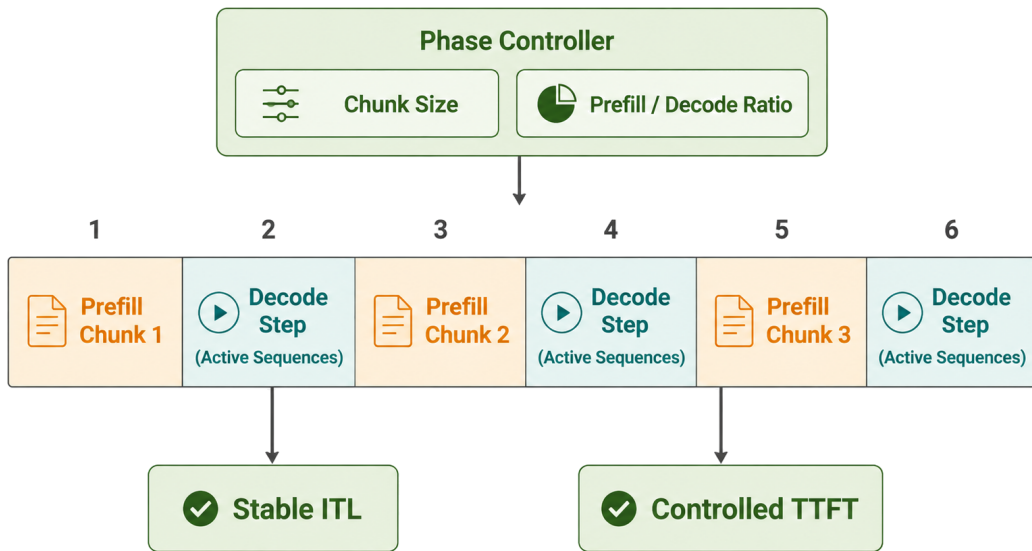


图 17 Chunked Prefill / Blended Batching 机制示意图

Chunked Prefill 的思想是把长 prompt 拆成多个 prefill chunk，而不是一次性执行完整 prefill。Blended Batching 则进一步把 prefill chunk 与 decode token 混合在同一个调度周期或同一个 batch 中，使 compute-heavy 的 prefill 和 memory/latency-sensitive 的 decode 交错执行。这样可以平滑每轮计算量，减少长 prefill 对 decode 的阻塞，也能避免 decode 完全占用调度周期导致新请求 TTFT 过高。

这类方法的核心变量包括 chunk size、prefill token budget、decode token budget、prefill-to-decode ratio、interleaving policy、phase quota、online/offline mixing ratio 和 stall control。Chunk 太大，会重新造成 prefill 阻塞；chunk 太小，会增加调度和 kernel overhead。Prefill/decode 比例太偏向 prefill，会伤害 ITL；太偏向 decode，会伤害 TTFT。因此，Chunked Prefill / Blended Batching 的本质是阶段内混合调度和阶段间干扰控制。Libra 可以理解为阶段配额控制的扩展 [56]，NanoFlow 代表更细粒度执行流优化 [55]，LLMStation 和 FlexLLM 则把混合对象扩展到 inference/finetuning co-serving 等更复杂 workload [58], [59]。这些工作本质上仍然需要在同一批执行资源上安排不同类型 token 或任务片段的执行顺序。MuxServe 同样涉及 batch scheduling，但其主问题是多模型 serving 的空间共置与时间复用，更适合作为 multi-model multiplexed batching 的横向机制，而不是 chunked prefill 的直接代表 [61]。

在 CEC 中，这类方法适合由不同能力的边缘节点协同执行。靠近请求入口的节点可以保持较短 decode 间隔，算力更强的协作节点处理较长 prefill chunk；当链路稳定且 KV 传输成本可控时，系统还可以把长 prompt prefill 分配到更强节点，再将可复用状态交给靠近用户的 decode 节点。若链路抖动大或边缘并发不足，过度切分 prefill 可能增加状态传输和调度开销，因此 chunk size 和 interleaving

ratio 必须纳入在线代价模型。

Chunked Prefill / Blended Batching 比 continuous batching 更直接处理 prefill/decode 干扰。它适合长 prompt、多轮对话、RAG 增强上下文和流式输出并存的场景，但需要准确控制 chunk size 和阶段配额。对于 CEC，关键是把网络传输和 KV 状态位置纳入阶段混合决策，否则 prefill 拆得越细，跨节点状态交接越可能成为新的瓶颈。

5.4 Disaggregated Prefill-Decode Batching

Disaggregated Prefill-Decode Batching 是对 phase-aware 思路的进一步扩展。Chunked Prefill / Blended Batching 仍然假设 prefill 和 decode 在同一个实例或同一类资源上交错执行；而 disaggregated batching 直接把两阶段拆到不同 worker pool、不同 GPU 实例、不同机器池或不同协作边缘节点。Prefill batch 和 decode batch 分别形成，分别扩缩容，并通过 KV Cache 或中间状态传输衔接。DistServe 从 goodput 优化角度论证阶段解耦 [4]，Splitwise 强调 prompt pool 与 token pool 的硬件匹配 [29]，Mooncake 则在 prefill/decode 解耦基础上构建 KVCache-centric serving 架构 [13]。

PD 分离后的 batching 可以按图 18 理解：prefill instance 内部形成 prompt batch，decode instance 内部维护 active sequence batch，两个池之间通过 KV/state handoff 连接。Global Scheduler 根据 SLO、

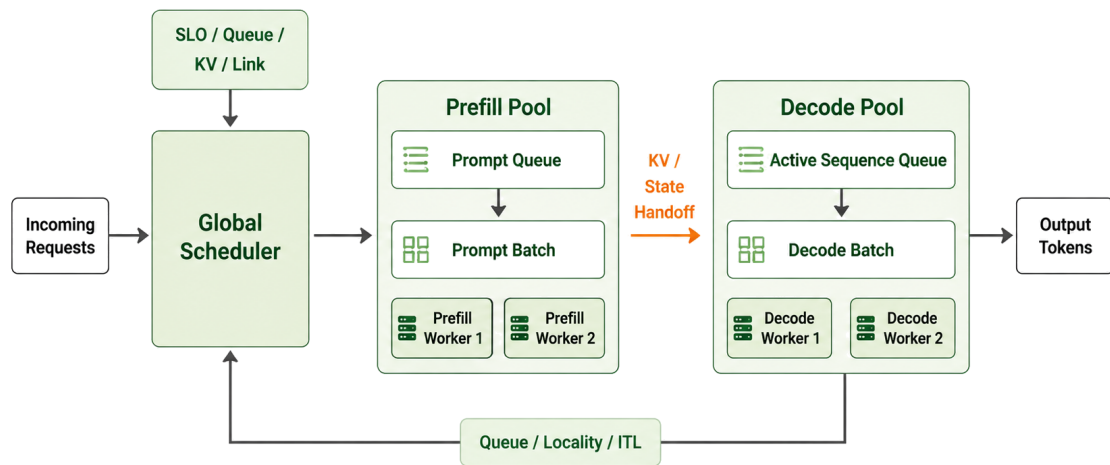


图 18 Disaggregated Prefill-Decode Batching 机制示意图

queue length、KV locality 和 link state 将请求送入 Prefill Pool；Prefill Worker 形成 prompt batches 并生成初始 KV blocks。随后 KV / State Handoff 将 KV blocks、state transfer 和 metadata 交给 Decode Pool，Decode Worker 再维护 active sequence batches 并输出 tokens。图中的反馈路径表示 queue length、KV

locality、ITL 等运行时指标回流到全局调度器，用于调整后续请求进入哪个 pool 以及是否需要扩缩容。因此，PD 分离后的 batching 实际上变成两个 pool 的独立组批与跨池状态协调问题。

这类方法的出发点是 prefill 和 decode 的计算与访存特征明显不同。Prefill 更偏 compute-bound，适合吞吐导向、更强算力或更大 batch 的资源池；decode 更偏 memory/latency-bound，需要稳定 ITL、低排队和快速状态访问。若两者混部在同一资源池中，长 prefill 可能阻塞 decode，decode 小步迭代又可能降低 prefill 吞吐。Disaggregation 通过物理或逻辑隔离减少两阶段干扰，并允许系统分别为 prefill 和 decode 选择 parallelism、batch size 和扩缩容策略。

Disaggregated Prefill-Decode Batching 的核心变量包括 prefill worker pool、decode worker pool、phase queue、KV transfer、state locality、worker scaling、prefill/decode routing 和 global scheduling。它的收益来自阶段资源匹配和干扰隔离，但代价集中在 KV 传输、状态连续性和跨节点调度。Prefill 完成后，decode 必须访问初始 KV Cache；如果 KV 传输慢、远程访问不稳定或用户入口发生变化，disaggregation 的收益会被状态成本抵消 [4], [13]。

本节关注 PD 分离之后的 batching：prefill pool 如何形成 prompt batch，decode pool 如何维护 active sequence batch，以及两个 pool 之间如何通过 KV/state handoff 协调。

在 CEC 中，disaggregated batching 可以映射到不同能力的协作节点：算力较强或能够聚合请求的节点承担 prefill-heavy 请求，靠近用户且持有会话状态的节点负责低时延 decode；容量较大的节点也可以维护 prefill pool 和 KV state service，其他节点只在需要较低 ITL 时承担 decode。对于移动用户，系统还需要判断 decode worker 是否应该跟随入口迁移，还是远程访问原有 KV。也就是说，CEC 下的 disaggregated batching 不能只看 GPU 利用率，还必须同时看链路稳定性、KV 迁移成本和服务连续性。

Disaggregated Prefill-Decode Batching 适合 prefill/decode 负载差异明显、阶段队列规模足够、状态传输成本可控的场景。若请求很短、边缘并发很低或链路抖动大，复杂阶段拆分可能不如本地 blended batching 稳定。因此，CEC 中应把 disaggregation 视为跨实例的资源组织方式，而不是默认打开的单点优化。

5.5 面向华为验证环境的方案建议

异构 Ascend 推理服务中的请求到达率、输入输出长度、KV Cache 占用和距离 deadline 的剩余时间不断变化，合适的 batch size、token budget、等待窗口和请求顺序也会随之改变。固定配置难以覆盖这些组合；普通强化学习虽然能够根据运行状态调整参数，但探索和随机输出可能造成显存越界、关键请求超时或运行时无法执行。为明确动态 batching 在不同请求特征下能够获得多大收益，本文先建立设备、网络、模型和请求的统一系统模型，分析请求长度与 batch size 变化时相对 Static-FIFO 的性能范围。该结果一方面刻画动态组批可利用的优化空间，另一方面为后续强化学习的状态、动

作和优化目标设计提供依据。在此基础上，本文提出带拉格朗日约束和运行时安全投影的 Safe-RL batching 方法。

评价时先检查 SLA 和运行安全，再比较性能。对同一模型、请求轨迹和硬件环境，只有 P90/P99 TTFT、TPOT 和 E2E latency 满足业务配置，显存保持在可用范围内，且受控实验中未发生 OOM、不可执行动作和已接纳关键请求硬超时，该运行点才进入完成请求速率与平均 E2E latency 的比较。实验目标是相对 Static-FIFO 将满足 SLA 的请求处理速度提高 20%，平均 E2E latency 下降 20%，并保持零硬违规 [43]。

为估计动态组批在验证环境中的性能空间，本文选取四卡 A800T-A2、100 Gbps 直连场景，CodeLlama34B 和 Qwen2-VL-7B-Instruct 沿用第四章得到的非均匀 PP 部署，并让输入和输出覆盖短、中、长三档。具体设备算力、layer 分配和请求长度设置将在后文给出。

理论分析中的 Static-FIFO 使用固定 batch size 和等待窗口，按请求到达顺序形成批次，并在当前批次完成后再处理下一批。批内输入按最长 prompt 补齐，生成过程持续到最长输出结束。极端情况下，一个 batch 包含 1 个最长请求和 63 个最短请求，短请求会承担大量额外计算。用 Static-FIFO 的总计算量除以所有请求各自所需计算量之和，再减 1，得到 **CodeLlama34B 和 Qwen2-VL-7B-Instruct 的计算侧提升上界分别为 1060.2% 和 353.9%**，该上界属于理论上界，实际部署中达不到。

为了在训练 Safe-RL 前先观察长度感知组批能够获得的大致收益，本文进一步采用一个长度相近组批算法：每次观察队首 24 个等待请求，选择距离 TTFT deadline 最近的请求作为基准；当紧迫程度相同时，优先选择到达时间最早的请求。算法再根据预计的 prefill 和 decode 计算量对其余请求排序，选择执行开销最接近的 7 个请求组成大小为 8 的 batch。**与 Static-FIFO 相比，该算法在 CodeLlama34B 和 Qwen2-VL-7B-Instruct 上的预估提升分别为 111.9% 和 90.1%**。这一结果用于判断动态调整请求组合是否具有足够的优化空间，最终收益仍需显存、请求到达顺序和 SLA 约束下通过 Safe-RL 实验验证 [43]。

5.5.1 系统模型

Batching 控制依赖请求、模型、部署资源和运行状态四类信息。为了统一描述算法能够观察和控制的对象，系统模型进一步给出由模型计算量、设备算力和链路开销推导 prefill、decode 服务时间的方法。图 19 左侧给出验证环境中的异构设备拓扑：E1 和 E2 使用同一组四台 A800T-A2，其中两台使用完整算力，另外两台分别使用一半和四分之一算力，并分别考察 100 Gbps 直连和 25 Gbps RDMA；E3 使用 56 GB/s HCCS 连接两台设备；E4 则通过 25 Gbps 链路连接 310P、半算力 910B4 和完整算力 910B4。中间部分将 PP stage、请求特征和运行状态映射到这些设备，并据此计算 prefill、decode、排队和显存开销。右侧在统一服务时间模型上分析不同请求长度与 batch size 下相对 Static-FIFO 的性能范围，并给出在线 batching 需要调节的参数及评价指标。这些信息随后作为 Safe-RL 的状态与

性能反馈。

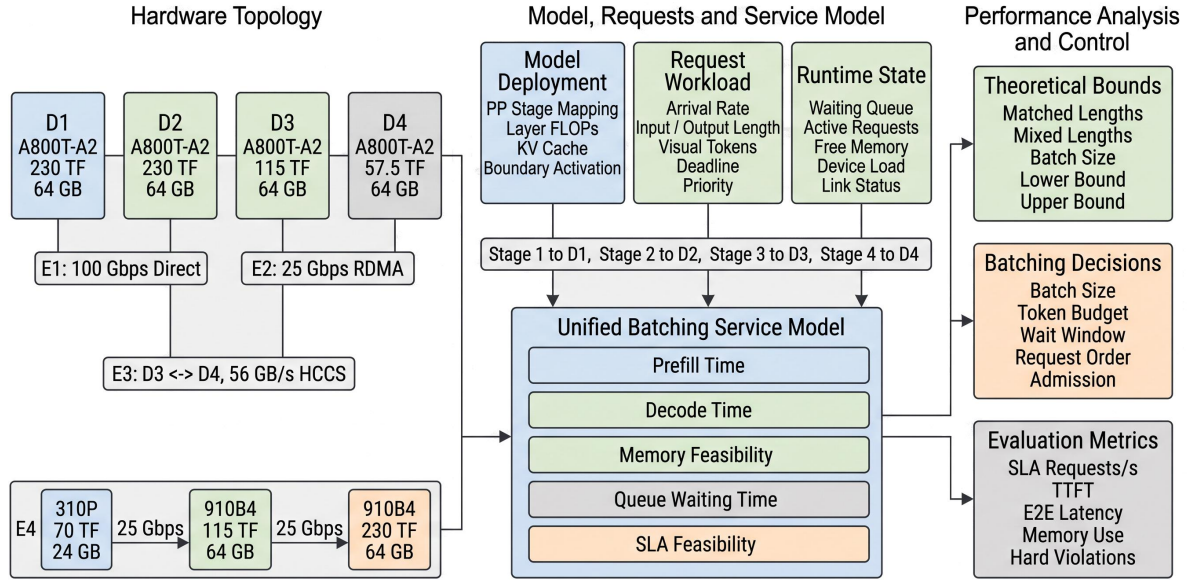


图 19 面向异构设备的业务感知 batching 系统模型

硬件环境表示为图 $\mathcal{H} = (\mathcal{V}, \mathcal{E})$ 。设备节点 $g \in \mathcal{V}$ 记录当前可用算力 F_g 、可用显存 M_g 和运行状态，链路 $(g, g') \in \mathcal{E}$ 记录有效带宽 $B_{g,g'}$ 与固定传输时延 $L_{g,g'}$ 。模型沿用第四章确定的非均匀 PP 部署方案 $\mathcal{D}$ ，其中包含各 stage 的放置设备、连续 layer 边界以及相邻 stage 之间的 activation 传输路径。对于 stage k ，其有效算力记为 $F_k^{\mathcal{D}}$ ，prefill 与 decode 的实测计算效率分别记为 η_k^{pre} 和 η_k^{dec} 。

到达系统的请求 r_i 由到达时间 a_i 、文本输入长度 P_i 、visual token 数 V_i 、预测输出长度 $\hat{O}_i$ 、实际输出长度 O_i 、请求 deadline d_i 和业务优先级 κ_i 描述。纯文本请求令 $V_i = 0$ 。实际输出长度 O_i 在请求完成后用于评价预测误差和服务结果，在线决策只能读取 $\hat{O}_i$ 。控制器在每个决策窗口读取等待队列、active requests、各请求的剩余时限、近期到达率、KV Cache 占用、剩余显存和设备利用率，并决定下一窗口的组批参数与请求顺序。

模型计算量直接由网络结构和请求长度得到。设一个文本 Transformer layer 的参数量为 N_{layer} ，hidden size 为 h 。长度为 P 的单请求 prefill 计算量和生成 O 个 tokens 的 decode 计算量分别为

$$C_{\text{pre}}(P) = \sum_{l=1}^L (2PN_l + 4P^2h_l),$$

$$C_{\text{dec}}(P, O) = \sum_{u=1}^O \sum_{l=1}^L (2N_l + 4(P + u)h_l).$$

第一式中的线性项来自模型参数参与的矩阵计算，二次项来自 prefill attention；第二式逐 token 累加 decode 计算。CodeLlama34B 使用 48 个文本 layers 的参数。Qwen2-VL-7B-Instruct 还要加入视觉编码

器和模态投影的计算量，并把文本 decoder 的上下文长度写为 $P + V$ 。因此，图像数量、分辨率和预处理后 visual token 数会同时影响 prefill 时间与后续 decode 的上下文长度。

对部署 $\mathcal{D}$ 中的 stage k ，batch 的 prefill 服务时间由该 stage 承担模块的 FLOPs、有效算力和通信共同计算：

$$T_k^{\text{pre}}(b, P, V) = \frac{C_k^{\text{pre}}(b, P, V)}{\eta_k^{\text{pre}} F_k^{\mathcal{D}}} \times 10^3 + T_k^{\text{link, pre}}.$$

单轮 decode 时间 $T_k^{\text{dec}}(b, P, V, u)$ 采用相同形式，其中计算量对应当前 token 步 u 。对单个请求，完整 prefill 时间和生成 O 个 tokens 的 decode 时间分别为

$$T_{\mathcal{D}}^{\text{pre}} = \sum_k T_k^{\text{pre}}, \quad T_{\mathcal{D}}^{\text{dec}} = \sum_{u=1}^O \sum_k T_k^{\text{dec}}(u).$$

各量均以 FLOPs、FLOPs/s 和 ms 为单位。第四章已经选择纯 PP，因此式中的通信项只包含相邻 stages 之间的 activation 传输。部署前令计算效率系数为 1，可得到由模型规格、设备有效算力和链路共同确定的时间参照；进入 Ascend 环境后，以不同 batch size 和长度下的逐层执行时间以及 CANN/HCCL 通信时间更新效率和通信项。由此形成的 prefill/decode 服务时间曲线直接用于 SLA 仿真和 Safe-RL 训练。

显存约束同时计入模型权重、activation、KV Cache 和运行时预留。设 batch 中 active requests 的当前上下文长度为 ℓ_i ，单 token KV Cache 大小为 m_{KV} ，则 KV 占用近似为 $m_{\text{KV}} \sum_i \ell_i$ 。该值与各 stage 已驻留权重和运行时空间共同决定当前能够接纳的请求数及 token budget。

5.5.2 性能理论分析

在统一系统模型下，本文以 Static-FIFO 为基线，先分析不同请求组合产生的长度补齐计算，再判断在线算法能够减少其中多少开销。当一个最长请求与其余最短请求进入同一 batch 时，Static-FIFO 的冗余计算最多，对应所考察请求范围内的计算侧提升上界。在此基础上，有限窗口长度感知算法给出受到观察范围和请求顺序限制时的可实现预估，进一步改变 batch size 还可以分析批内请求数量对两类结果的影响。这些结果为 Safe-RL 选择需要观察的请求特征、需要调节的 batching 参数以及合理的性能目标提供理论依据；SLA、排队、KV Cache 和控制开销的影响则在 5.5.6 节通过请求轨迹实验验证。

对于请求 r_i ，将完成其 prefill 和 decode 所需的模型计算量记为

$$C_i = C_{\text{pre}}(P_i, V_i) + C_{\text{dec}}(P_i, V_i, O_i).$$

下文用 $C(P, V, O)$ 表示具有给定输入、视觉和输出长度的请求计算量，因此 $C_i = C(P_i, V_i, O_i)$ 。设

一个静态 batch 包含 B 个请求。若执行过程按 batch 内最长输入和最长输出进行长度补齐 (padding), 记 $P_{\max}$ 、 $V_{\max}$ 和 $O_{\max}$ 为该 batch 的最大长度, 则总计算量近似为

$$W_{\text{pad}} = B C(P_{\max}, V_{\max}, O_{\max}).$$

当长度相近的请求被放在一起, 并且短请求结束后空出的执行位置能够及时回收时, 请求集合至少要完成的有效计算量为

$$W_{\text{use}} = \sum_{i=1}^B C_i.$$

两者的比值

$$S_{\text{len}} = \frac{W_{\text{pad}}}{W_{\text{use}}}$$

给出长度重排在该简化模型中的计算侧速度倍数上界。相对 Static-FIFO 的提升为 $(S_{\text{len}} - 1) \times 100\%$ 。 S_{len} 是无量纲的速度倍数; 实际 requests/s 由服务时间模型或硬件实测得到, 并进一步接受显存、排队和 deadline 检查。

表 12 给出代表性请求的模型计算量和部署前服务时间。CodeLlama34B 采用 512-token 输入和 256-token 输出; Qwen2-VL-7B-Instruct 采用 640 个 visual tokens、1024 个 text tokens 和 256-token 输出。时间计算使用第四章 E1 场景中选出的 PP 部署, CodeLlama34B 的文本层分配为 (18, 18, 9, 3), Qwen2-VL-7B-Instruct 为 (4, 14, 7, 3)。计算同时计入各 stage 的有效算力和 100 Gbps 链路上的边界 activation 传输, batch size 取 1, 计算效率系数取 1。

表 12 代表性请求的模型计算量与 E1 部署前服务时间

模型与请求配置	Prefill / TFLOPs	Decode / TFLOPs	Prefill / ms	Decode / ms
CodeLlama34B, 512 / 256 tokens	34.43	17.27	207.84	104.23
Qwen2-VL-7B-Instruct, 640 visual + 1024 text / 256 output	27.19	3.53	177.78	24.53

表中的时间用于把 FLOPs 计算与具体设备、链路联系起来。实际 batch size 增大后, 将批内所有请求的 FLOPs 和 activation 大小代入同一公式即可得到对应服务时间。Ascend kernel 的实际利用率通常随 batch size 和序列长度变化, 因此正式实验以逐层测量得到的 η_k^{pre} 、 η_k^{dec} 和通信时间替换部署前参照, 再判断 TTFT、TPOT 和 E2E latency 是否满足 SLA。

对任一 batching 配置 a , 将其 batch size、请求长度和执行顺序代入服务时间曲线, 可以得到完成该批请求所需的时间 $\hat{T}(a)$, 并进一步估计各请求的等待时间、TTFT 和 E2E latency。记 $\mathcal{A}_{\text{SLA}}$ 为同时满足显存限制和 SLA 阈值的配置集合, $B(a)$ 为配置 a 完成的请求数, 则部署前的 SLA 吞吐上界

写为

$$R_{\text{SLA}}^{\text{ub}} = \max_{a \in \mathcal{A}_{\text{SLA}}} \frac{B(a) \times 10^3}{\hat{T}(a)} \text{ requests/s.}$$

输入长度和 visual token 数主要改变 prefill 时间，输出长度改变 decode 轮数，batch size 同时影响计算效率、KV Cache 和等待时间，因此这些参数都会改变可行配置集合与吞吐上界。正式实验使用 Ascend 实测服务时间重新计算 $\hat{T}(a)$ ，并通过请求轨迹验证预测的 SLA 可行性。

理论上界用于界定请求重排的最大优化空间，可实现的部署前预估则由有限窗口长度感知组批得到。每次准备新 batch 时，调度器观察等待队列前 W 个请求，先选择剩余 TTFT 时间最短的请求作为本批的基准请求，以避免长度匹配导致紧急请求继续等待；随后利用 5.5.1 节的服务时间模型估计其余请求的 prefill 和 decode 执行开销，从中选择预计执行开销最接近的 $B - 1$ 个请求。请求加入后还要单独检查 KV Cache、显存和 SLA，检查失败时跳过该请求并继续考察下一个。部署前分析以 $B = 8$ 为参考 batch size，并令观察范围为其三倍，即 $W = 24$ ；后续再改变 W 和 B 检查结果的敏感性。

为了在相同计算口径下比较理论空间与有限窗口算法，记 Static-FIFO 形成的所有 batches 的长度补齐计算量为 W_{FIFO} ，全部请求自身所需的有效计算量为 W_{use} ，有限窗口长度感知算法形成的 batches 的长度补齐计算量为 W_{len} 。处理相同数量请求时，两种提升分别为

$$G_{\text{ub}} = \frac{W_{\text{FIFO}}}{W_{\text{use}}} - 1, \quad G_{\text{len}} = \frac{W_{\text{FIFO}}}{W_{\text{len}}} - 1.$$

其中， G_{ub} 表示完全消除额外长度补齐计算时的上界， G_{len} 表示在 $W = 24$ 、 $B = 8$ 条件下执行上述排序算法得到的预估提升。两者均使用 5.5.1 节的模型 FLOPs 计算，不再使用输入输出 token 数的线性加权值。

表 13 使用短、中、长三档请求进行计算。CodeLlama34B 的输入长度为 128、512 和 2048 tokens，Qwen2-VL-7B-Instruct 的文本输入长度为 256、1024 和 4096 tokens，并固定 640 个 visual tokens；输出长度均为 64、256 和 512 tokens。计算侧预估令同一观察窗口中的请求均已到达且 SLA 紧迫程度相同，因而只比较长度补齐计算量；请求到达和 deadline 的影响在随后请求轨迹中加入。

表 13 短、中、长三档随机请求下的纯计算上界与有限窗口排序预估

请求分布	CodeLlama34B		Qwen2-VL-7B-Instruct	
	计算上界	排序预估	计算上界	排序预估
输入输出长度一致	0.0%	0.0%	0.0%	0.0%
仅输入长度变化	135.5%	115.3%	103.1%	89.5%
仅输出长度变化	35.7%	32.8%	11.8%	11.0%
输入输出长度同时变化	173.3%	111.9%	115.7%	90.1%

输入长度变化带来的提升高于输出长度变化，主要原因是较长 prompt 会同时增加参数计算和

attention 计算，并把静态 batch 中其它短请求的 prefill 长度一并抬高。表中的有限窗口排序只能观察队首 24 个请求，因此预估结果低于可以在全部请求中自由匹配的计算上界；加入真实到达时间和 deadline 后，紧急请求优先还会进一步限制长度匹配空间。在输入输出长度同时变化时，CodeLlama34B 的纯计算上界为 173.3%，有限窗口排序预估为 111.9%；Qwen2-VL-7B-Instruct 的两项结果分别为 115.7% 和 90.1%。这些数值用于描述组批算法自身能够减少多少长度补齐计算，尚未计入请求等待、显存限制和 SLA 筛选。

将上述规则应用于带到达时间、输入输出长度和 deadline 的请求轨迹后，可以得到实际形成的 batch 序列。每个 batch 的完成时间由 $\hat{T}(a)$ 计算，请求等待时间由到达时刻与 batch 启动时刻之差得到。设轨迹持续时间为 T_{trace} ，单位为 s，其中满足 SLA 并完成的请求数为 N_{SLA} ，则该规则的预估请求处理速度为

$$\hat{R}_{\text{len}} = \frac{N_{\text{SLA}}}{T_{\text{trace}}} \text{ requests/s.}$$

在相同请求轨迹上运行 Static-FIFO，即可由 $\hat{R}_{\text{len}}/\hat{R}_{\text{FIFO}} - 1$ 得到长度感知组批的预估提升。该结果反映有限观察范围、请求到达顺序和 SLA 共同约束下能够实现的参考性能，并作为 Safe-RL 设计与实验结果之间的中间参照。

为了观察极端长度失配对计算上界的影响，进一步使用 $B = 64$ 的请求集合进行比较。CodeLlama34B 的输入长度取 128、512 和 2048 tokens，Qwen2-VL-7B-Instruct 取 256、1024 和 4096 text tokens，并固定 640 个 visual tokens；输出长度取 64、256 和 512 tokens。对 decode 而言，长度匹配最好的情况是 64 个请求具有相同的输出 token 数；同时计入 prefill 后，还要求输入长度一致，此时没有额外的长度补齐计算。输出失配场景由 1 个 512-token 输出和 63 个 64-token 输出组成，输入失配场景由 1 个最长输入和 63 个最短输入组成。本文将“1 个最长输入、最长输出请求与 63 个最短输入、最短输出请求”定义为组合失配的最差情况。表 14 给出这些极端组合下的纯计算上界。

表 14 64 个请求极端长度组合下的纯计算上界

请求组成	CodeLlama34B 上界	Qwen2-VL 上界	含义
输入输出长度完全一致	0.0%	0.0%	静态组批已无长度补齐损失
仅输出长度极端失配	76.1%	21.7%	长输出持续占用 batch 执行周期
仅输入长度极端失配	478.6%	280.8%	最长 prompt 放大整个 batch 的 prefill 计算
输入输出同时极端失配	1060.2%	353.9%	长输入与长输出的影响叠加

输入长度带来的范围明显大于输出长度，主要原因是 prefill attention 中包含随输入长度平方增长的计算项。Qwen2-VL 的视觉编码开销对每个请求都存在，因而长度失配占总计算量的比例低于 CodeLlama34B。组合失配中的大幅提升描述 1 个最长请求与 63 个最短请求被强制放入同一静态 batch 的极端情况，用于界定算法层面的最大空间；在线策略还要服从到达顺序、deadline、显存和等待时间限制。

batch size 越大，一个极长请求影响的其它请求越多。表 15 固定组合失配模式，比较不同 B 下的理论上界。

表 15 batch size 对组合失配理论上界的影响

Batch size / requests	CodeLlama34B 提升	Qwen2-VL 提升
8	432.7%	225.8%
16	671.0%	288.4%
32	893.1%	329.7%
64	1060.2%	353.9%

较大的 batch 为长度重排提供更多空间，也会增加 KV Cache、单轮执行时间和 TTFT 风险。表中的上界用于界定长度重排的优化空间，Safe-RL 的实际收益还受到可观察请求数、SLA 允许的等待时间、显存和运行时开销影响。算法在这些条件下联合调整 batch 容量、token budget 与请求顺序。进入硬件环境后，再使用 Ascend 实测的 prefill 和 decode 服务时间校准理论结果。

5.5.3 基于安全强化学习的 Batching 算法

前述理论分析表明，请求长度、batch size 和可观察请求范围都会改变 batching 的优化空间；在线系统还要同时处理到达率、KV Cache、deadline、prefill/decode 干扰和运行时能力。若为这些变量逐项设置阈值，规则数量会快速增加，并且一次调整对后续队列和显存的影响难以由局部规则评价。强化学习能够从连续运行反馈中学习多变量之间的关系，因此适合用于动态 batching 控制。

现有系统已经提供了必要的执行基础。Orca [1] 支持迭代级调度，vLLM [2] 提供灵活的 KV Cache 管理，Sarathi-Serve [3] 和 DistServe [4] 分别利用 chunked prefill 与 prefill/decode 分离缓解阶段干扰，JITServe [45] 和 SOLA [46] 进一步考虑 deadline。这些机制扩大了可调节的参数和执行方式，也使固定规则更难覆盖全部组合。

普通强化学习策略在训练探索、负载突变和状态估计误差下可能输出越界动作。本文采用 Safe-RL 将长期风险约束与当前窗口的确定性安全检查结合起来。Actor 根据请求、资源和 SLA 状态提出 batching 方案，Critic（价值评估网络）根据后续运行结果评价该方案的长期完成请求数与风险；拉格朗日约束用于限制多个窗口上的累计风险，安全投影在动作下发前检查显存、deadline 和运行时支持范围。无法修正的动作回退到最近的稳定配置。

图 20 给出了完整闭环。系统先采集队列、请求长度、KV Cache 和 SLA 状态，Actor 输出 batch 参数、请求排序和执行机制，安全投影生成可执行配置，推理运行时返回完成请求数、时延、资源占用和违规事件，反馈再用于更新 Actor、Critic 和约束模型。该方法的重点是在动态提高完成请求速度的同时，使受控压力实验中的 OOM、不可执行动作和已接纳关键请求硬超时均保持为 0。

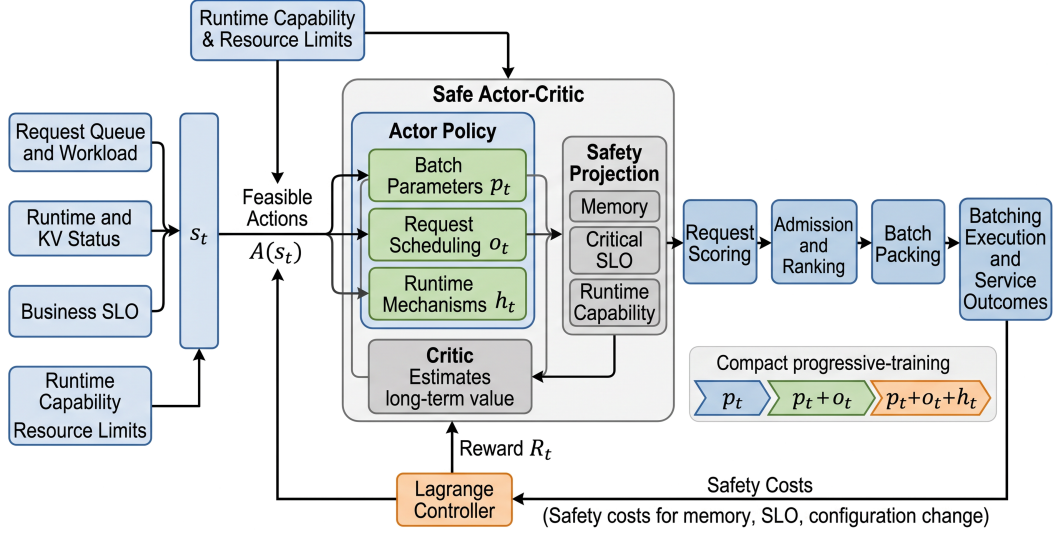


图 20 基于拉格朗日约束和安全投影的在线 batching 优化架构

5.5.4 CMDP 建模与决策生成

连续 batching 控制可以写成约束马尔可夫决策过程（Constrained Markov Decision Process, CMDP）。这一模型描述一次配置调整对后续队列、KV Cache 和请求时限的持续影响，并进一步规定 Actor 的输出如何转换为实际 batch，使各项决策都有明确的数据来源和运行时取值范围。

本文将推理过程划分为控制窗口 $t = 0, 1, \dots$ ，每个窗口依次完成状态采集、动作生成、运行执行和反馈更新。CMDP 表示为

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{P}, R, \{\xi^j\}, \gamma),$$

其中， $\mathcal{S}$ 和 $\mathcal{A}$ 分别为状态空间与动作空间， $\mathcal{P}$ 描述执行动作后的状态变化， R 表示服务收益， ξ^j 表示第 j 类运行风险， γ 用于降低较远反馈对当前决策的影响。

窗口开始时的状态记为 $s_t = (q_t, z_t, d_t)$ 。 q_t 包含等待队列、active requests、prefill/decode 请求数、近期到达率和输入输出长度分布； z_t 包含可用显存、剩余 KV blocks、设备利用率、近期服务时间、链路状态、上一窗口配置和运行时支持能力； d_t 包含请求优先级、deadline 及其剩余时间。模型结构、单 token KV Cache 大小和 5.5.1 节得到的 prefill/decode 服务时间曲线作为相对稳定的上下文输入。

动作 a_t 包含三类决策。第一类是 batch 容量参数，包括 active batch 上限 b_t 、token budget u_t 、最大等待窗口 w_t 、prefill chunk size c_t 和 prefill/decode 配额 ρ_t 。第二类是请求调度参数，包括准入阈值 τ_t 与请求排序权重 $\vec{\omega}_t$ 。第三类是运行时机制 h_t ，用于选择当前环境已经支持的 chunked prefill、长请求隔离或 PD 资源池路由。容量参数和调度参数在短周期更新，切换开销较高的机制在较长周期更新。

这些决策变量由运行状态逐步生成。系统先读取 Ascend 推理运行时支持的参数、取值范围和调

整粒度；再依据实例并发上限、剩余 KV blocks 和单 token KV 增量确定 b_t 与 u_t 的上界，依据 deadline、队列积压和到达率限制 w_t 与 τ_t ，并依据 token 粒度和当前 prefill/decode 积压确定 c_t 与 ρ_t 的可选范围。Actor 输出归一化数值后，将其映射到这些范围并按运行时粒度取整。对于离散机制，当前环境不支持的选项在 Actor 输出阶段直接屏蔽。

请求排序权重进一步转换为 batch 成员。等待请求 r_i 的得分为

$$S_i = \omega_1 f_{\text{urgency}}(r_i) + \omega_2 f_{\text{age}}(r_i) + \omega_3 f_{\text{priority}}(r_i) - \omega_4 f_{\text{cost}}(r_i) - \omega_5 f_{\text{KV}}(r_i).$$

距离 deadline 越近、等待越久或业务优先级越高，请求得分越高；预计 FLOPs、输出长度和 KV 增量越大，其资源成本越高。系统按得分选择满足准入阈值的请求，并同时检查请求数不超过 b_t 、总 token 数不超过 u_t 。等待时间项持续增长，使长请求和低优先级请求仍能获得执行机会。

窗口收益以完成请求数为主要正向项，并扣除归一化的排队时间和 E2E latency，使策略同时学习提高处理速度和缩短响应时间。风险包括 OOM 与不可执行配置、TTFT/TPOT/E2E 超限、长请求等待过久和相邻窗口配置变化过大。策略在这些风险上限内提高长期服务收益：

$$\max_{\pi} J_R(\pi), \quad J_j(\pi) \leq \epsilon_j.$$

其中， J_R 是多个控制窗口上的累计服务收益， J_j 是第 j 类累计风险， ϵ_j 是相应上限。硬系统事件和已接纳关键请求硬超时的上限取 0，分位 SLA、公平性和配置变化采用业务允许值。完成请求数、等待时间和 E2E latency 的权重在训练集上固定，并在全部对比实验中保持一致。Actor 输出的建议动作还要经过下一节的安全投影后才能下发。

5.5.5 策略学习与安全执行

Safe-RL 需要同时处理跨窗口的性能收益和当前窗口的执行安全。Actor 负责提出联合 batching 方案，Critic（价值评估网络）根据后续队列、显存和请求时延变化评价该方案。拉格朗日约束影响训练方向，安全投影负责动作下发前的最终检查，两者共同覆盖长期风险和单次执行安全。

Actor 策略记为 $\pi_{\theta}(a_t | s_t)$ 。训练时引入非负风险系数 μ_j ，将窗口收益调整为

$$\tilde{R}_t = R_t - \sum_j \mu_j \xi_t^j.$$

当某类风险接近上限时，对应系数增大，Critic 会降低高风险动作的长期评价；风险保持较低时，策略可以继续探索提高完成请求数的配置。Critic 依据实际执行后的状态与收益更新价值估计，Actor 再提高长期评价较好动作的选择概率。部署阶段使用策略均值或最高概率动作，减少随机输出造成

的配置波动。

拉格朗日约束根据历史轨迹控制多个窗口上的累计风险，实际系统还需要在每次动作下发前进行确定性检查。将 Actor 提出的动作记为 $\tilde{a}_t$ ，经过检查后实际下发的动作记为 $\hat{a}_t$ ，二者满足

$$\hat{a}_t = \Pi_{\mathcal{A}_{\text{safe}}(s_t)}(\tilde{a}_t),$$

其中， Π 表示把建议动作修正到安全范围内的确定性映射， $\mathcal{A}_{\text{safe}}(s_t)$ 是当前状态下同时满足显存、已接纳关键请求 deadline 和推理运行时能力要求的动作集合。轻量性能模型先预测建议动作对应的峰值显存、关键请求 TTFT、decode ITL 和单轮执行时间，再加入根据离线预测误差设置的安全余量。修正后的预测结果通过全部约束检查后，动作才会进入推理运行时。显存容量和运行时支持项可以直接检查，时延条件则依据带安全余量的预测上界判断；安全压力实验进一步统计实际执行中的 OOM、硬超时和不可执行动作，用于确认零硬违规目标。

具体来说，若预测显存超限，投影模块先缩小 token budget 或 active batch 上限，并重新选择 batch 成员；若已接纳的关键请求可能错过 deadline，则缩短等待窗口、提高紧迫度权重，并把执行配额转向该请求当前所处的 prefill 或 decode 阶段，同时减少与其无关的工作；当前推理运行时不支持的机制则直接关闭。连续参数被调整到最近的可执行值，离散机制只能从支持列表中选择。若多次修正后仍找不到可行动作，系统回退到上一稳定配置，并优先处理已经接纳的关键请求。只有当负载超过物理承载能力时，才在准入阶段对新请求限流，并将限流结果单独统计。

训练轨迹同时记录建议动作 $\tilde{a}_t$ 、实际动作 $\hat{a}_t$ 、修改幅度和运行结果。Critic 根据实际执行后的状态变化进行评价；如果 Actor 频繁提出需要大幅修正的动作，训练信号会加入额外惩罚。随着训练进行，策略应逐步减少对投影和回退的依赖，直接生成更接近安全范围的配置。

运行时采集请求长度、TTFT、ITL、E2E latency、KV Cache 使用量、违规和回退信息，并反馈给 Actor、Critic 与约束模块，形成图 20 所示的闭环。短周期负责更新 batch 参数和请求调度，长周期负责切换开销较高的执行机制。拉格朗日约束用于降低多个窗口上的累计风险，安全投影用于拦截当前窗口中预测会越界的动作，两个控制周期则避免高开销机制频繁切换。

为降低在线探索风险，训练分三步进行：首先固定 FIFO 和运行时机制，只训练 batch 容量参数；随后加入请求准入与排序；最后联合优化 batch 参数、请求调度与执行机制。

这种顺序逐步扩大动作空间，使 Critic 能够区分容量调整、请求排序和机制切换的影响，并减少训练初期配置反复变化的情况。

训练初期使用静态 batching、FIFO 和启发式策略生成离线训练数据。在线阶段先只计算建议配置并评估其可行性，不实际下发；确认性能模型误差可控后，再允许策略在安全范围内尝试小幅调整，并保留稳定配置作为回退。性能模型通过离线性能测量和在线数据持续更新。在线控制器以静

态 batching 和固定请求顺序作为参照，并以前文给出的性能与安全目标作为训练和验证依据；具体收益在下一节结合基础仿真和正式实验进行分析。

5.5.6 实验设计与评价指标

Safe-RL 的验证由服务时间标定、请求轨迹回放和安全压力测试组成。实验先用 5.5.1 节的 FLOP 模型确定需要测量的输入长度、输出长度和 batch size 组合，再以 Ascend 上的实测 prefill/decode 时间替代理论计算中的效率系数。所有策略使用相同模型部署、请求到达时间、请求属性和随机种子，并按照“先满足 SLA 与硬安全要求，再比较完成请求速率和平均 E2E latency”的顺序评价。

性能标定覆盖 CodeLlama34B 和 Qwen2-VL-7B-Instruct，并沿用第四章最终选择的模型部署。对每个模型分别改变输入长度、输出长度、visual token 数、active batch、token budget、KV Cache 占用和 prefill/decode 配额，测量各 stage 的 prefill 时间、单轮 decode 时间、通信时间和峰值显存。这些数据形成 T_D^{pre} 与 T_D^{dec} 服务时间曲线，用于离线训练、SLA 仿真和安全投影。

主负载采用 4 到配置上限之间随机变化的闭环并发。每隔 500 ms 更新目标并发；并发升高时增加客户端，并发降低时等待已有请求完成后停止补充。请求输入和输出均覆盖短、中、长三档，Qwen2-VL-7B-Instruct 额外改变图像数量、分辨率和 visual token 数。业务优先级按高、普通和低三档混合，并为高优先级请求设置更严格的 deadline。正式测试覆盖给定的 11 组模型、组网、P90/P99 TTFT 和并发配置 [43]，各方法使用相同的到达轨迹、请求属性和随机种子。

设统计窗口长度为 T s，窗口内提交并最终完成的请求数为 N_T^{done} ，则完成请求速率为

$$\hat{R}_{\text{thr}} = \frac{N_T^{\text{done}}}{T}.$$

平均 E2E latency 为

$$\bar{L}_{\text{E2E}} = \frac{1}{N_T^{\text{done}}} \sum_{i \in \mathcal{R}_T} (f_i - a_i).$$

其中， $\hat{R}_{\text{thr}}$ 的单位为 requests/s； $\mathcal{R}_T$ 是统计窗口内提交并最终完成的请求集合， a_i 和 f_i 分别为请求到达与完成时间，单位为 ms。窗口结束后停止提交新请求并排空队列，使平均时延覆盖窗口内提交的全部请求。未通过 SLA 或安全运行的运行点保留失败原因和时延记录，其完成请求速率不参与可行策略之间的性能比较。

主对比包括两类基线和三种 RL 方法。基线为 Static-FIFO、Ascend 推理运行时默认配置和 SLA-Heuristic；RL 方法为 Unconstrained-RL、Lagrangian-RL 和完整 Safe-RL。Static-FIFO 使用固定 batch size、等待窗口和 token budget，按到达顺序形成批次，并在当前批次完成后再处理下一批；SLA-Heuristic 按 deadline 和长度设置人工规则；Unconstrained-RL 只优化性能；Lagrangian-RL 加入长期风险约束；Safe-RL 进一步加入当前窗口的安全投影和回退。六种方法使用相同的硬件资源、请求范

围和运行时能力。

实验分别改变输入长度分布、输出长度分布、batch size、可观察请求数、最大等待时间、token budget、prefill chunk size、prefill/decode 配额、输出长度预测误差、并发变化速度和 SLA 严格程度。低并发场景用于观察调度空间不足时的控制开销，中高并发和长度混合场景用于检验算法能否利用 5.5.2 节给出的理论空间。固定并发扫描用于确定各方法满足 SLA 的最大稳定并发，随机并发轨迹用于比较动态负载下的完成请求速率和平均 E2E latency。

实验需要验证三项结论：Safe-RL 能否在满足 SLA 的前提下提高完成请求速率并降低平均 E2E latency；拉格朗日约束和安全投影能否实现零硬违规；batch 容量、请求排序和运行时机制分别贡献多少收益。除随机并发主实验外，还设置稳定负载、阶跃负载、短时突发、长 prompt 集中到达、不同 prefix cache 命中率以及 prefill-heavy、decode-heavy 场景。训练与测试使用不同的请求序列和随机种子。

第一组实验回放上述随机并发轨迹。每种策略使用相同的请求到达时间、输入输出长度、优先级和随机种子。校准阶段在相同的 batch size、观察范围、最大等待时间、token budget、prefill chunk size 和 prefill/decode 配额范围内调节方法：Static-FIFO 与启发式方法在训练轨迹上选择满足 SLA 的最佳固定配置，Safe-RL、Lagrangian-RL 和 Unconstrained-RL 则把同一范围作为动作边界进行训练；测试阶段固定各方法在训练阶段得到的参数或策略。每个测试运行点先检查 P90/P99 TTFT、TPOT/E2E latency、显存使用、队列稳定性和硬安全事件；未通过检查的配置记录失败原因，通过检查的配置再比较完成请求速率、平均 E2E latency、P95/P99 latency、最长等待时间、ITL 和设备利用率。

设 Static-FIFO 和 Safe-RL 在同一条请求轨迹上均满足 SLA，其完成请求速率和平均 E2E latency 分别为 R_0, L_0 和 $R_{\text{safe}}, L_{\text{safe}}$ 。其中， R_0 与 R_{safe} 的单位为 requests/s， L_0 与 L_{safe} 的单位为 ms，相应的性能目标为

$$R_{\text{safe}} \geq 1.2R_0, \quad L_{\text{safe}} \leq 0.8L_0.$$

两个目标分别表示完成请求速率至少提高 20% 和平均 E2E latency 至少下降 20%，均依据同一配置、同一并发轨迹下的实测值计算。测试窗口结束后停止提交新请求并排空已有队列，平均 E2E latency 对窗口内提交的全部请求计算；同时报告到达量、完成量、拒绝量和窗口结束时的积压量。除混合请求负载的整体结果外，还分别改变输入长度分布、输出长度分布、并发水平和 SLA 严格程度，报告不同长度请求的完成请求速率、时延和最大等待时间，以检查长请求饥饿和短请求阻塞。主动限流和拒绝只在超过系统承载能力的安全压力测试中启用，并与性能主实验分开统计。

随机并发实验用于计算满足 SLA 时的完成请求速率和平均 E2E latency 改善；固定并发扫描用于补充确定各方法的最大稳定并发。与第四章方法联合使用时，以第四章理论比较最终选定的部署方案运行 Static-FIFO 所得到的实测结果作为本章基线，再计算 batching 带来的增量收益。

第二组实验验证业务感知调度。固定请求量和长度分布，改变高优先级请求比例与 deadline 紧迫程度，分别统计各优先级的 SLA 违规率、等待时间和完成率，并报告最大等待时间和 Jain 公平性指数。

第三组实验为安全压力测试。实验延长最高并发的持续时间、集中注入长 prompt，并通过预占显存或减少可用 KV block 模拟资源突然收缩。比较 Safe-RL、Lagrangian-RL 和 Unconstrained-RL 的峰值显存、OOM、不可执行动作、已接纳关键请求硬 SLA 违规、建议动作被修改的次数、回退和限流情况。零硬违规直接按三类事件的总数定义：

$$N_{\text{hard}} = N_{\text{OOM}} + N_{\text{invalid}} + N_{\text{deadline}} = 0.$$

其中， N_{deadline} 只统计系统已经接纳的关键请求。Safe-RL 需要在所有受控压力场景中保持 $N_{\text{hard}} = 0$ 。同时记录 Actor 建议被安全模块修改的比例和参数修改幅度；若这些数值随训练下降，说明 Actor 正在逐步学会直接生成更接近安全范围的配置。超过系统物理处理能力的新请求单独统计限流和接纳结果。

第四组实验进行消融：依次比较仅调整 batch 容量参数、进一步加入请求排序、以及同时加入运行时机制选择的结果。对于 chunked prefill、长请求隔离和 PD 路由，额外记录启用次数、持续时间和切换开销。

每个实验点预热后至少完成 1000 个请求，并使用不少于五个随机种子重复。时延报告平均值、P95 和 P99，完成请求速率报告均值和 95% 置信区间，同时测量状态采集、Actor 推理、安全投影和 batch 重组的控制开销。达到本节目标需要同时满足：Static-FIFO 与 Safe-RL 在相同硬件、请求轨迹和 SLA 检查下运行，Safe-RL 的完成请求速率提高不少于 20%，统计窗口内提交请求的平均 E2E latency 下降不少于 20%，且 $N_{\text{hard}} = 0$ 。

5.6 小结

本章按 batch 形成和更新粒度，将业务感知 batching 参数优化划分为四类：Queue-level Dynamic / Admission Batching、Continuous Batching / Iteration-level Batching、Chunked Prefill / Blended Batching 和 Disaggregated Prefill-Decode Batching。Queue-level 方法决定请求进入执行前如何等待、排序和接纳；Continuous Batching 将 batch 更新边界推进到 decode iteration；Chunked Prefill / Blended Batching 通过切分长 prompt 并与 decode 混合执行来控制阶段干扰；Disaggregated Prefill-Decode Batching 则把 prefill 和 decode 放入不同实例或资源池中分别组批。

这四类方法对应不同的 batching 控制层次，可以在同一 runtime 中组合使用。本章将 SLO-aware、KV-aware、multi-tenant 和 speculative verification 作为横向约束或附加机制：SLO 影响 admission 和排

序，KV 状态影响 active batch 容量和迁移成本，多租户影响队列公平性，speculative verification 则引入额外的验证 batch。以 batch 更新粒度为主线，可以清楚地区分“请求进不进 batch”“active set 何时更新”“prefill 是否切块混合”“两阶段是否分池组批”这几类核心问题。

CEC 场景改变了 batching 的优化重心。云端 serving 通常依赖大请求池和高速互联来扩大 batch，而单个请求入口附近的实例往往请求池较小、链路波动更强、SLO 更紧、KV 显存更有限。因此，靠近入口的实例更适合短等待窗口、低延迟 continuous batching 和保守的 chunked prefill，能够聚合多个入口或具备更强算力的协作节点则适合处理 prefill-heavy 请求、长上下文和较大的 batch。业务感知 batching 需要同时读取请求长度、SLO、优先级、KV 占用、实例负载和链路状态，并据此联合调节多个控制量。

面向验证环境，本章先用设备拓扑、第四章选定的模型部署、模型 FLOPs 和请求属性建立系统模型，由此计算不同 batch size 与输入输出长度下的 prefill/decode 服务时间。理论分析比较静态长度补齐与每个请求有效计算量，给出长度完全一致、输入失配、输出失配和组合失配下的性能范围。该分析解释了动态组批可以利用的空间，也为后续 Safe-RL 结果提供与模型和硬件条件一致的参照。

在线控制被建模为 CMDP，并采用带拉格朗日约束的 Actor-Critic 求解。Actor 联合生成 batch 容量、请求排序和运行时机制，Critic（价值评估网络）评价多个窗口上的完成请求数、响应时间与风险；安全投影在动作下发前检查显存、deadline 和运行时支持范围，无法修正时回退到稳定配置。正式实验使用 Ascend 实测服务时间曲线替代理论计算中的效率系数，并在相同 SLA 下比较 Static-FIFO、规则方法、普通 RL、Lagrangian-RL 与 Safe-RL 的完成请求速率、平均 E2E latency 和硬违规事件。

六、CEC 中的 LLM Inference Task Offloading

传统边缘计算卸载通常将任务建模为具有给定输入数据量、计算量、deadline 和能耗约束的计算 job，其核心问题是在本地设备、邻居设备、边缘服务器及可选远程服务之间选择执行位置，并联合优化通信资源和计算资源分配。在这一范式下，任务卸载的主要决策对象通常是完整任务、子任务、任务链断点或 DAG 节点，优化目标也多集中在时延、能耗、系统成本和队列稳定性等方面。

LLM inference task 改变了这一问题形态。与普通计算任务不同，LLM 推理的执行时间不仅由输入 prompt 决定，还受到输出长度、decode step 数、batch 状态、模型队列和上下文长度影响；一次请求也不一定只由单个模型独立完成，而可能需要多个模型、多个 agent、多个工具或多个 verifier 协作完成；多轮对话和长上下文服务则进一步引入 KV cache、prefix cache、session state、agent memory 和 user context 等运行时状态。因此，在 CEC 场景下，LLM inference task offloading 不再只是决定“任务是否离开请求入口节点”，而是需要决定完整请求如何在协作节点间路由、多个模型或 agent 如何编排，以及支撑后续推理的 context/KV/runtime state 如何迁移、复用或重新计算。

基于上述特性，本章从一次 LLM 服务请求在 CEC 系统中的执行过程出发组织相关工作。具体而言，当一个 LLM request 到达系统后，首先需要决定该请求或会话应由哪个协作节点、模型实例或可选外部服务处理；对于复杂请求，系统可能进一步需要编排多个模型、agent 或工具形成协作式工作流；在请求执行和多轮交互过程中，context、KV cache、prefix cache、session state 和 agent memory 等运行时状态还需要在不同节点之间保留、迁移、复用或重新计算。因此，本章将现有方法划分为三类：request/session-level routing and offloading、multi-model/multi-agent workflow orchestration，以及 context/KV/runtime state offloading and migration，三类代表性工作分别在表 17、表 18 和表 19 中展开。

代表工作总览。表 16 汇总了 LLM inference task offloading 相关工作的主归属、相关机制和方法关键词。前三类对应本章 6.1–6.3 的文献脉络，最后一类面向项目场景，将 request routing、KV/state management、SLA/memory 约束和长期 session cost 放入同一决策闭环。表中部分工作采用 cloud-edge 或 edge-cloud 实验环境，本文借鉴其路由与状态管理机制，并将其映射到协作边缘节点之间的调度问题。

表 16 CEC 下 LLM Inference Task Offloading 代表工作总览

序号	代表工作	发表在哪	主归属	相关机制	方法关键词
1	QoS-LLM-Router [62]	TMC 2025	Request / Session-level Routing and Offloading	QoS-aware routing; state-aware scheduling	结合请求质量、响应长度、GPU memory 和队列状态，将 edge LLM expert routing 建模为长期 QoS 优化问题
2	CES-CB [63]	TSC 2025	Request / Session-level Routing and Offloading	request cost prediction; combinatorial bandit	在其原始 edge/cloud 环境中，利用 prompt embedding 和历史执行数据预测请求成本，选择可提升吞吐并缓解长请求阻塞的请求集合
3	RouteLLM [64]	ICLR 2025	Request / Session-level Routing and Offloading	strong/weak model routing	基于 query 难度和偏好数据判断是否调用强模型，在保持质量的同时降低强模型调用成本

4	Training-Free Router [65]	NeurIPS 2025	Request / Session-level Routing and Offloading	budget-aware multi-LLM routing	通过近邻检索估计不同 LLM 的质量和成本，并用预算影子价格实现低训练成本的在线模型路由
5	ActiveInfer-Offload [66]	TMC 2024	Request / Session-level Routing and Offloading	offloading; resource allocation	使用 active inference 在不确定 cloud-edge 环境中联合决定 LLM inference task offloading 和资源分配
6	SLM-LLM-Collab [67]	TMC 2026	Request / Session-level Routing and Offloading	SLM caching; D2D / edge collaboration	在本地 SLM、邻居 SLM 和 edge LLM 之间做短时卸载决策，并将长期 SLM caching 作为请求路由前置条件
7	MGCO [68]	TSC 2026	Request / Session-level Routing and Offloading	mobility-aware offloading	用 Transformer MHA encoder 建模历史决策和未来移动状态，使 generative computation offloading 避免短视路由
8	MasRouter [69]	ACL 2025	Multi-model / Multi-agent Workflow Orchestration	multi-agent routing	联合选择协作模式、agent 数量、角色分工和底层 LLM，使 multi-agent system configuration 匹配 query 难度
9	Conductor [70]	ICLR 2026	Multi-model / Multi-agent Workflow Orchestration	natural-language workflow control	训练元控制器用自然语言生成多模型 workflow，控制 subtask、model id 和上下文访问范围
10	AFLOW [71]	ICLR 2025	Multi-model / Multi-agent Workflow Orchestration	automatic agentic workflow search	将 agentic workflow 表示为代码形式的 LLM 调用图，通过搜索和执行反馈自动优化复杂任务 workflow
11	DT-MADTO [72]	TMC 2025	Multi-model / Multi-agent Workflow Orchestration	mobility-aware DAG offloading	用 digital twin 预测移动和资源状态，联合决策 DAG 子任务卸载、带宽分配和中间数据等待时间
12	FuseSpill [73]	TPDS 2026	Context / KV / Runtime State Offloading and Migration	KV spillover; auxiliary GPU scheduling	在显存受限 GPU 上按代价模型选择 KV cache 溢出策略，并将溢出序列调度到辅助 GPU 继续 decode
13	Weaver [74]	USENIX ATC 2025	Context / KV / Runtime State Offloading and Migration	attention offloading	通过 attention offloading 缓解 multi-LLM serving 中的资源压力，提高多模型服务吞吐和资源利用率
14	Mooncake [13]	USENIX FAST 2025	Context / KV / Runtime State Offloading and Migration	distributed KV cache; global scheduling	将 CPU/DRAM/SSD/RDMA 组织为全局 KVCache Store，并按缓存命中、节点负载和 TTFT/TBT SLO 调度请求
15	CA-AIGC-Migration [75]	TSC 2026	Context / KV / Runtime State Offloading and Migration	context migration; mobility-aware service migration	将模型迁移转化为 context migration，在 reward 中联合考虑 context relevance、AoI、计算延迟和传输成本
16	长期成本感知请求卸载方案 本报告		Project-oriented Long-term Cost-aware Offloading	Markov 人口转移; 长 度/会话持续性预测; action masking; 增量 KV placement	联合选择计算节点与 KV reuse/sync/recompute 方式，并通过后台增量预放置降低后续请求的按需 KV 同步压力

6.1 Request / Session-level Routing and Offloading

Request/session-level routing 是最接近传统 task offloading 的 LLM inference offloading 形式。它将完整 LLM 请求、query、conversation turn 或 session 作为基本决策对象，决定其应该由本地设备、邻近边缘节点、其他协作边缘实例还是可选外部服务处理。与普通任务不同，LLM request 的处理时间往往难以在执行前准确确定，因为它不仅取决于 prompt length，还受到 output length、decode step 数、batch 状态和模型当前队列影响。对于多个 edge LLM experts、多个 SLM/LLM 服务或分布式边缘网络，系

统还需要根据请求复杂度、模型能力、响应质量、时延约束、GPU memory、队列状态和调用成本进行联合决策。

这一类问题的核心矛盾是：靠近用户的节点具有低时延、低回传开销和数据本地性优势，但计算资源和显存有限，长输出请求可能占据 batch 并造成 head-of-line blocking；能力更强或能够聚合请求的远端节点可提供更大的模型与请求池，却会增加 RTT、带宽开销和数据出域风险。对于 CEC 系统而言，请求路由需要回答“由哪个 edge server 执行”“调用哪个 LLM expert”“是否先由 SLM 处理”以及“何时升级到更强的协作节点或外部模型服务”等问题。表 17 汇总了该方向的代表性工作。

表 17 Request / Session-level Routing and Offloading 代表性工作

代表工作	发表在哪 & 时间	问题表述	核心方法	优化目标
QoS-LLM-Router [62]	TMC, 2025	多个 edge LLM experts 在生成质量、响应长度、GPU memory 和队列状态上异构；动态到达的请求需要实时选择 expert，以同时保证 response quality 和 latency requirement。	将请求路由建模为 infinite-horizon MDP；使用 DistilBERT-based predictors 预测 response quality 和 length；使用 heterogeneous graph attention network 抽象系统状态，并用 SAC 学习 QoS-aware routing policy。	最大化长期 accumulated QoS，同时考虑 response quality 和 latency constraint。
CES-CB [63]	TSC, 2025	LLM request 的 output length 和处理时间不可预测；如果长请求留在 edge，会阻塞后续短请求并降低 edge throughput。	将 prompt embedding 作为 context，将每轮 edge batch selection 建模为 contextual combinatorial semi-bandit；使用 neural prediction 与 UCB 机制平衡 exploration/exploitation。	最大化 edge throughput，减少因长请求导致的 head-of-line blocking。
RouteLLM [64]	ICLR, 2025	给定一个 query，如何在推理前决定调用强模型还是弱模型，使系统在满足质量目标的同时尽量少调用强模型。	使用偏好数据训练 router，预测强模型相比弱模型是否具有明显优势；根据阈值决定调用强模型还是弱模型。	在保持回复质量的同时降低强模型调用比例和推理成本。
Training-Free Router [65]	NeurIPS, 2025	在高流量、多 LLM 在线服务中，如何在不知道未来请求、预算有限且不能频繁训练 router 的情况下，快速把每个 query 分配给最合适的 LLM，以最大化整体回答质量并控制成本。	先用历史 query 的近邻检索 (ANNS) 快速估计当前 query 在各个 LLM 上的质量和成本，再用最开始一小部分在线 query 做一次轻量优化，学习每个 LLM 的预算“影子价格” γ 。之后每个新 query 只需按“估计质量收益 - 预算成本惩罚”打分，选择得分最高的 LLM，从而实现无需训练、低延迟、预算感知的在线路由。	在每个 LLM 的 token 预算约束下，为在线到达的 queries 选择路由模型，使所有已处理 queries 的总体回答质量/性能最大化。
ActiveInfer-Offload [66]	TMC, 2024	Cloud-edge LLM inference 需要同时考虑任务卸载和资源分配；传统 DRL 方法数据效率低，且难以适应动态任务负载和 latency requirement。	使用 active inference 框架建模 LLM inference task offloading 和 resource allocation，在不确定环境下进行在线决策。	优化 cloud-edge LLM inference 的服务性能、资源利用和时延表现。
SLM-LLM-Collab [67]	TMC, 2026	在给定 SLM/LLM 服务能力和缓存状态下，每个请求需要决定本地 SLM 执行、D2D 卸载给邻居 SLM，还是卸载到 edge LLM 与 cached SLM plugins 协同推理。	在短时间尺度上使用 factor graph 和 belief propagation 进行分布式 inference offloading 决策；长期 SLM caching 部分作为请求路由的前置条件，而非本节重点。	在时延和显存约束下最大化长期平均推理准确率。
MGCO [68]	TSC, 2026	移动场景下，generative computation offloading action 会受到历史决策和未来移动状态影响；短程决策容易短视，且传统 RNN/LSTM 对长序列建模能力有限。	使用 Transformer MHA encoder 建模长程移动和历史决策信息；通过候选移动区域约束缩减 action space，提高收敛效率。	在移动 edge-cloud 系统中实现更高效的 generative computation offloading，降低长期服务开销。

从 CEC 角度看，上述工作共同说明，LLM request routing 不能简单沿用传统 task offloading 中“输入数据量 + CPU cycles + deadline”的建模方式。一方面，请求执行时间受到输出长度和 batch 状态影响，具有更强的不确定性；另一方面，请求质量取决于模型能力、prompt 语义和模型选择，而不是单纯取决于计算资源是否充足。因此，CEC 中的 request-level offloading 需要同时感知 semantic complexity、model capability、network condition 和 queue state。

多用户和移动性会进一步放大这一复杂性。用户位置变化会改变请求入口、链路质量、候选服务节点和区域请求热点；请求执行过程中可能发生 handover，多轮会话的 context 或 KV cache 也可能留在原服务节点。因此，request/session routing 不能只根据当前时隙的队列和链路状态做短视选择，而需要同时考虑未来位置变化、会话连续性、状态位置和模型可用性。未来可以进一步研究 mobility-aware session routing，使请求路径选择同时感知用户移动趋势、后续多轮会话、模型缓存位置和 context/KV 可复用性。

6.2 Multi-model / Multi-agent Workflow Orchestration

随着 LLM 从单轮问答走向复杂推理、代码生成、工具调用、科学问答和长期任务执行，一个 request 到来后，系统不一定只调用一个模型完成回答。更一般的形式是：系统根据请求复杂度和任务类型，动态选择多个模型、多个 agent 或多个工具，并决定它们之间的角色分工、通信拓扑、执行顺序、上下文可见性和结果聚合方式。这一类问题可以被称为 multi-model 或 multi-agent workflow orchestration。

这一类问题与传统 stage-level offloading 或 splittable-request offloading 不同。Stage-level offloading 关注 prefill、decode、verification、diffusion steps 等底层执行阶段如何分布到不同计算节点；而 multi-agent workflow orchestration 关注的是语义层和任务层的协作结构，例如是否需要 planner、solver、critic、verifier、researcher、programmer 等角色，是否采用 Chain-of-Thought、Self-Consistency、LLM Debate、chain topology 或 complete graph 等协作模式，以及每个 agent 背后调用哪个 LLM 或 SLM。换言之，这里的决策对象不是 token、模型层或推理阶段，而是一个 request 的 problem-solving workflow。

这一类问题的核心矛盾是：多 agent 协作可以提升复杂任务的推理能力、鲁棒性和可解释性，但也会带来额外模型调用成本、通信开销、上下文传输开销和调度复杂度。对于 CEC 系统而言，agent 背后的模型、工具、数据库、memory 和 verifier 可能分布在用户设备、不同边缘服务器和外部服务上。因此，multi-agent workflow orchestration 不仅是算法层的协作模式选择问题，也是计算、通信、状态和隐私的联合编排问题。表 18 列出了相关代表性工作。

从 LLM serving 角度看，RouteLLM 可以被视为 multi-agent routing 的基础形式：它只在强模型和弱模型之间做 query-level routing，尚未涉及 agent 数量、角色分配或协作拓扑。MasRouter 则进一步把 routing 扩展到 multi-agent system configuration，即根据 query 决定采用哪种协作模式、需要多少个 agent、每个 agent 扮演什么角色，以及每个角色调用哪个底层 LLM。Conductor 更进一步，不再只在预定义配置集合中选择，而是训练一个控制器用自然语言生成 workflow，包括任务拆解、模型调用和上下文访问控制。AFLOW 则是将 workflow 表示为代码形式，通过搜索方法，实时根据当前任务执行情况自适应生成新的 workflow。

从 CEC 角度看，multi-agent workflow orchestration 需要被进一步物理化。现有 multi-agent orchestration 工作大多默认所有 worker LLM 都可以通过 API 调用，较少考虑模型、工具和 memory 的实际

表 18 Multi-model / Multi-agent Workflow Orchestration 代表性工作

代表工作	发表在哪 & 时间	问题表述	核心方法	优化目标
RouteLLM [64]	ICLR, 2025	现有 LLM serving 中，所有 query 都调用强模型会带来高成本，而固定使用弱模型又可能损害回答质量；需要根据 query 难度选择合适模型。	使用偏好数据训练一个 query-level router，预测强模型是否会显著优于弱模型；根据阈值进行强/弱模型选择。	在保持目标回答质量的同时减少强模型调用比例，降低推理成本。
MasRouter [69]	ACL, 2025	现有 LLM routing 多解决“给单个 agent 选哪个 LLM”的问题，但在多智能体系统中，真正需要 routing 的还包括协作模式、agent 数量、角色分工以及每个角色对应的底层 LLM。	通过将 query 映射到 latent space 选择合作模式和 agent 数量；使用 structured probabilistic cascade 为 agent 分配角色；再用多项分布为每个角色选择底层 LLM；训练时使用 policy gradient 优化。	为每个 query 选择合适的 multi-agent system configuration，使回复质量和准确率尽可能高，同时控制推理成本。
Conductor [70]	ICLR, 2026	如何训练一个元控制器，让它用自然语言自动编排多个已有 LLM，使整体系统超过任一单个 worker。	使用一个 SLM 作为 Conductor，根据 query 生成自然语言 workflow；每个 step 包含 subtask、model id 和 access list；训练阶段采样多个 candidate workflows，根据最终答案正确性给 reward，并用 GRPO 更新 Conductor。	让 Conductor 学会任务拆解、模型选择、上下文可见性控制和 workflow 生成，使多模型协作后的最终答案尽可能正确。
AFLOW [71]	ICLR, 2025	如何在尽量减少人工设计的情况下，自动生成并优化由多次 LLM 调用组成的 agentic workflow，从而提升 LLM 在问答、代码生成和数学推理等复杂任务上的表现。	AFLOW 将 agentic workflow 表示为代码形式的 LLM 调用图/流程，并把 workflow 优化转化为一个搜索问题。选择已有高分 workflow，让 LLM 修改 prompt 或代码结构生成新 workflow，再执行评估，并把成功/失败经验回传到搜索树中，从而逐步找到更优工作流。	给定任务 T 和评价函数 G，在所有可能的 workflow 中找到使任务表现 G(W,T) 最大的工作流 W
DT-MADTO [72]	TMC, 2025	移动用户产生具有 DAG 依赖关系的复杂任务；设备移动导致链路状态和中间数据流传输代价动态变化，同时多设备并发卸载会造成 flow congestion。	构建 device、physical CEC、digital twin 三层架构；使用 digital twin 预测未来移动和资源状态，并用 DDPG / actor-critic 联合决策 subtask offloading、bandwidth allocation 和 flow waiting time。	最小化任务完成时间和能耗的加权 QoS 指标。

部署位置。但在 CEC 系统中，不同 agent 可能对应不同物理节点：本地设备上的 SLM 可以承担隐私敏感的初步理解任务，邻近边缘节点上的 LLM 可以承担低时延交互任务，能力更强的协作节点可以承担复杂推理和 verification，外部工具或数据库 agent 也可能位于不同网络位置。因此，agent workflow 的拓扑会直接影响端到端时延、带宽消耗、隐私风险和状态迁移成本。

多用户移动性进一步使 multi-agent workflow orchestration 变得更复杂。用户移动会改变可访问的 agent、工具服务和 memory 节点；workflow 中间状态可能在多个 agent 之间传递，如果用户在执行过程中发生 handover，后续 agent 是否继续在原节点执行、是否迁移到新的协作节点、是否远程访问原有 context，都会影响服务质量。虽然 DT-MADTO [72] 并不是 LLM agent workflow 工作，但其对 DAG 依赖关系、中间数据流和移动预测的建模，为 CEC 中的 multi-agent workflow offloading 提供了可借鉴思路。未来可以进一步研究 mobility-aware agent workflow orchestration，在 request 到达时联合决定协作模式、agent 位置、模型选择、上下文可见性和中间状态传输路径。

6.3 Context / KV / Runtime State Offloading and Migration

LLM inference 与普通计算任务的另一个关键区别在于运行时状态会持续影响后续请求。多轮对话、长上下文服务、prefix reuse 和 agent workflow 会产生 KV cache、prefix cache、session state、agent memory、tool outputs 和 intermediate reasoning traces。这些状态的位置会直接影响后续 TTFT、decode latency、重复计算成本、跨节点传输开销、隐私边界和服务连续性。因此，context/KV/runtime state

offloading 关注的不是计算任务本身去哪执行，而是支撑后续推理的状态应该保留在哪里、何时迁移、是否远程访问、是否复用或是否重新计算。

这一类问题的核心矛盾是：状态迁移可以保持多轮服务连续性、减少重复 prefill 计算并提升生成质量；但迁移或远程访问状态会消耗带宽、显存和存储资源，也可能增加一致性管理和隐私风险。对于长上下文 chatbot，KV cache 和 prefix cache 可能比单次请求本身更有价值；对于 multi-agent workflow，中间结果、工具调用结果和 agent memory 也可能在后续步骤或后续会话中被反复使用。因此，LLM inference offloading 的优化目标正在从 computation-aware 转向 computation-state co-optimization。表 19 汇总了 context、KV 和 runtime state 卸载迁移方向的代表性工作。

表 19 Context / KV / Runtime State Offloading and Migration 代表性工作

代表工作	发表在哪 & 时间	问题表述	核心方法	优化目标
FuseSpill [73]	TPDS, 2026	在显存受限 GPU 上进行 LLM 推理时，KV cache 容易溢出，现有 swap/recompute 策略处理低效，导致吞吐下降和溢出请求延迟增加。	FuseSpill 通过代价模型自适应选择 KV cache 溢出策略，并把溢出或主动卸载的序列调度到辅助 GPU 继续 decode，从而释放主 GPU 显存、提高 prefill/decode 利用率。	在显存受限 GPU 上提高 LLM 推理吞吐量，同时降低发生 KV cache spillover 的请求延迟。
Weaver [74]	USENIX ATC, 2025	多 LLM serving 中 attention computation 和模型实例管理会造成资源压力；attention 和相关状态如何 offload 会影响多模型服务效率。	通过 attention offloading 提升 multi-LLM serving 的资源效率，并缓解多模型服务中的资源压力。	提高多模型服务吞吐和资源利用率。
Mooncake [13]	USENIX FAST, 2025	在长上下文 LLM 聊天服务中，如何用全局分布式 KV cache 复用历史前缀计算，减少昂贵的 prefill GPU 计算，从而在满足 TTFT/TBT 延迟要求的同时提升系统吞吐。	把 LLM 推理拆成 prefill 和 decoding 两个独立资源池，并把集群中 CPU/DRAM/SSD/RDMA 组织成全局分布式 KVCache Store；调度器根据缓存命中、节点负载和 TTFT/TBT SLO，把请求路由到合适的 prefill/decoding 节点。	用更多存储换取更少重复计算，提高长上下文 LLM serving 效率。
DistServe [4]	USENIX OSDI, 2024	Prefill 和 decode 混部执行会造成阶段间干扰，并耦合两阶段资源分配；两阶段分离后，KV handoff 和状态传输成为连接 prefill 与 decode 的关键问题。	将 prefill 和 decoding 分离到不同 worker/resource pool，并根据 TTFT/TPOT 需求分别优化资源分配和并行策略。	最大化满足 TTFT 和 TPOT 约束下的 goodput。
T2DRL [76]	TON, 2026	Mobile edge 难以支撑 long-context LLM agents 的多轮交互，因为 context window 会同时影响准确率、延迟和资源消耗；模型缓存和请求执行策略需要随上下文和使用模式变化。	提出 T2DRL 框架，在训练和测试阶段共同学习 cached models 与服务请求的管理策略，并结合 double Dutch auction 做资源匹配。	降低系统成本，同时保证 long-context LLM agents 在感知和推理任务中的服务性能。
CA-AIGC-Migration [75]	TSC, 2026	用户移动到新边缘节点后，旧节点上的 long context 是有价值但迁移成本较高的状态资产；传统 service migration 迁移完整模型成本过高。	利用 LLM in-context learning 特性，将 model migration 转化为 context migration；在 reward 中联合考虑 context relevance、AoI、计算延迟和传输成本，并用 Transformer DRL 学习迁移策略。	在保持 AIGC 回复质量和准确性的同时降低 context migration 成本和服务时延。

从系统角度看，KV cache、prefix cache 和 context 不再只是 LLM inference 的内部实现细节，而是 CEC 中可以被 offload、cache、migrate 和 schedule 的重要资源。Mooncake 说明了长上下文服务中 KV cache 可以作为全局分布式存储资源被管理；DistServe 说明了 prefill/decode disaggregation 会使 KV handoff 成为连接不同资源池的关键状态传输；FuseSpill [73] 和 Weaver [74] 则从显存约束和 attention offloading 角度说明，runtime state 的位置会直接影响吞吐和时延。

在 CEC 中，context/KV/runtime state offloading 需要进一步考虑边缘节点之间的异构资源和网络差异。对于多轮会话，如果每轮请求都重新 prefill 长上下文，会产生大量重复计算；如果跨节点远

程访问 KV cache，则会引入额外 RTT 和带宽开销；如果主动迁移 context 或 KV，则可能占用边缘链路和存储资源。因此，系统需要在远程访问、状态迁移、状态压缩、状态丢弃和重新计算之间进行选择。

多用户移动性使这一问题更加突出。用户移动后，计算路径可以重新选择，但 long context、KV cache、agent memory 或 session state 可能仍保留在原服务节点。此时系统需要在四种选择之间权衡：迁移状态到新的服务节点、远程访问原有状态、重新计算状态，或丢弃部分状态并降低上下文质量。不同选择会影响后续 decode latency、生成质量、带宽消耗和隐私边界。CA-AIGC-Migration [75] 已经开始从 context migration 的角度研究这一问题，但对 KV cache placement、remote KV access、prefix cache sharing 和 agent memory migration 的移动 CEC 场景研究仍然不足。未来可以进一步研究 mobility-aware KV and context management，将用户轨迹预测、session continuation probability、KV reuse probability、edge storage pressure 和链路状态联合纳入决策。

6.4 面向华为验证环境的方案建议

前文 6.1–6.3 分别从 request/session routing、multi-model/multi-agent workflow orchestration 和 context/KV/runtime state management 三个角度，梳理了 CEC 中 LLM inference task offloading 的相关工作。总体来看，现有研究已经开始关注请求级路由、模型选择、工作流编排和 KV/context 管理，但在多基站 CEC 场景下，用户移动性、输出长度不确定性和多轮 session state 的联合作用仍未被充分建模。对于 LLM request 而言，调度动作不仅决定当前请求在哪个节点执行，也会决定最新 KV cache 增长在哪里、后续请求能否复用历史状态，以及用户移动后是否需要同步、重算或远程访问状态。

基于上述问题，本节给出一个面向用户移动性的长期会话感知卸载方案。该方案以“计算节点 + KV 获取方式”作为联合动作，在单请求层面枚举 reuse、sync 和 recompute 等可行方式，在跨请求层面通过长期状态价值评估当前动作对后续 KV 分布和服务成本的影响，并通过 KV manager 在请求间隙执行后台增量预放置。系统采用中心化 Router/agent 进行任务卸载决策，中心状态管理模块持续同步节点负载、链路状态、显存余量和版本化 KV 目录，使中心化 Router 能直接基于最新全局状态完成过滤和决策。

本节最后通过三类实验给出当前实现的初步结果。端到端模拟表明，在 1.5× 并发下，单请求穷举与完整 KV manager 相比同并发就近执行，将 CodeLlama34B 和 Qwen2-VL 的平均 E2E latency 分别降低 44.89% 和 26.02%；若按承载能力口径，与基础并发下的就近执行相比，平均 E2E latency 分别降低 19.65% 和 23.37%。中心状态同步实验表明，在三节点高频状态更新配置下，1.5× 并发产生约 3.01 Mbps 的节点间状态上报流量，最高单链路约 1.60 Mbps，占 100 Gbps 链路约 0.0016%。三节点原型路由网络的 inference 微基准表明，两层 128 hidden units 的 Dueling Double DQN 在 Apple M4 CPU 单线程 NumPy 实现下单次前向 P99 为 0.016 ms。需要说明的是，本轮端到端实验验证的是单

请求穷举与完整 KV manager 的组合效果，长期 Dueling Double DQN 的跨请求增量收益仍需在后继对照实验中单独验证。

6.4.1 问题背景与挑战

CEC 任务卸载通常面向普通计算任务：系统根据输入数据量、计算量、链路状态、队列负载和 deadline 选择执行位置。这类任务的工作量一般可在调度前估计，任务完成后对后续请求的状态影响也较弱。LLM inference request 则具有明显不同的运行特征：服务以多轮 session 持续进行，每轮 request 依赖历史上下文和 KV cache，同时又生成新的 token 与 KV state；decode step 数在调度时未知，使真实执行时间、返回流量、显存占用和新增 KV 规模都具有不确定性。因此，CEC 中的 LLM request offloading 不只是普通任务卸载的直接扩展，而是一个同时包含执行不确定性、状态依赖和长期状态演化的调度问题。

- **LLM 推理任务的卸载挑战。**输出长度是主要不确定来源。调度器在 request 到达时只能观测 prompt、上下文和业务 SLA，无法预知模型最终生成多少 token。输出长度决定 decode 时间、E2E latency、返回流量和峰值显存压力，也决定本轮新增 KV blocks 的规模。低估输出长度可能导致 SLA violation 或显存不足；过度保守则会降低并发和资源利用率。同时，LLM session 具有强历史依赖，后续 request 的 prefill 成本、状态准备成本和可行动作集合都与此前保留的 KV cache 有关。
- **用户移动性下的状态位置挑战。**现有 CEC 移动任务卸载研究多面向普通计算任务，主要处理接入链路变化、任务迁移和结果回传路径；已有 LLM 推理调度工作则更多关注静态或低移动性的服务集群，尚未充分讨论多基站 CEC 中用户移动、session 连续性和 KV 状态位置之间的耦合关系。对于 LLM 推理任务，移动性不仅改变后续 request 的入口节点，还会改变历史 KV cache 与未来请求入口之间的相对位置：当用户在多轮 session 中从一个基站覆盖区移动到另一个覆盖区时，历史 KV cache 可能仍保留在旧计算节点。系统因此需要同时决定当前计算节点和 KV 获取方式，即就地复用、跨节点同步或在目标节点重算。

6.4.2 方案概览与模块关系

上述两个挑战相互交织，使得调度策略制定更加困难。移动性改变未来 request 的入口位置，输出长度决定当前 request 的执行时间和新增 KV state，二者共同影响后续多轮中的状态同步、重算、显存占用和 SLA 风险。若仅优化当前 request，系统可能得到较低即时成本，却在后续多轮中产生额外状态同步或重算开销。因此，本节将 CEC LLM request offloading 建模为面向 session 的长期成本优化问题。

围绕上述问题，方案由以下三个递进的设计环节构成。

设计一：单请求穷举。传统就近计算策略将请求固定在当前入口节点执行，其主要依据是缩短请求到计算节点的网络距离，但没有同时比较候选节点的排队负载、计算资源、链路状态和历史 KV cache 位置。当入口节点负载较高或完整 KV 位于其他节点时，就近执行可能产生较长排队，或者需要在入口节点同步重量级 KV、重新计算历史上下文。单请求穷举将请求转发、节点排队、KV 同步或重算、prefill、decode 和结果返回等开销纳入统一且可测量的即时成本，并枚举满足 SLA、显存、节点可用性和 KV 一致性约束的“计算节点 + KV 获取方式”，从中选择当前请求预计 E2E latency 最小的动作。该策略能够主动利用远端空闲资源，并在转发轻量请求与恢复重量级状态之间进行即时比较。

设计二：面向 session 的长期状态价值评估。单请求穷举虽然能够降低当前请求的服务成本，但 LLM session 具有持续的状态演化：每轮 request 不仅消费历史 KV，还会在实际执行节点生成新的 KV blocks。因此，本轮执行节点会决定最新 KV 的增长位置，并进一步改变后续请求的状态本地性、同步量和可行动作集合。仅优化即时成本可能为了降低当前排队而切换执行节点，却使后续多轮持续承担远程转发或状态同步；也可能在入口变化时反复改变最新 KV 的位置，产生路由与状态位置振荡。为此，本文在即时成本之外进一步评估当前动作形成的下一 KV 分布、节点负载和未来入口变化所带来的累计服务成本，使 Router 同时判断“当前在哪里执行成本最低”和“当前执行位置是否有利于后续请求”，从而形成面向 session 的 Long-term Router。

设计三：概率与会话活跃度感知的后台增量 KV 预放置。即使 Long-term Router 能够识别未来更合适的执行节点，它仍受到前台 KV 恢复成本的限制。与通常为 KB 或 MB 级的请求负载相比，长上下文产生的 KV cache 可达到 GB 级；如果仅在请求到达后同步 KV，大规模状态传输可能主导请求关键路径，使 Router 即使判断某个候选节点具有更低的未来计算或排队成本，也因当前 residual KV 过大而继续将请求转发到原 KV 节点。为降低这一切换门槛，本文在请求完成后，根据候选入口概率、session 剩余活跃度和后台资源余量，逐步将缺失的连续 KV prefix blocks 复制到未来可能使用的节点。后续请求到达时，目标节点可以直接复用已经准备的 prefix，或仅同步少量 residual KV，从而将部分重量级状态传输移出请求关键路径。

三个设计环节的递进关系。单请求穷举解决就近计算忽略系统负载和 KV 位置的问题；长期状态价值评估进一步解决单请求穷举无法感知当前动作对未来 KV 分布影响的问题；后台增量 KV 预放置则降低长期路由切换时的前台状态恢复门槛。三项设计从当前请求的即时选择、跨请求的状态演化和请求间隙的后台状态准备三个层次，共同构成长短时间尺度结合的请求卸载方案。

在此基础上，本文采用“状态预测-显式可行性过滤-长期请求路由-后台增量 KV 预放置”的双时间尺度框架。在线 Router 在请求到达时选择计算节点与 KV 获取方式；KV manager 在请求完成后利用后台带宽逐步扩展候选节点上的连续 KV prefix。二者通过分布式 block directory 共享 KV 位置和版本状态，后台放置形成的新状态在下次请求到达时由 Router 直接观测和利用。具体包括以下三个模块：

- **不确定性感知的状态预测。**入口转移模型以 Markov 矩阵描述请求在节点间的转移概率；output length predictor 根据 prompt、上下文长度、任务类型和会话历史预测输出长度分布；session continuation predictor 根据当前请求序号、session 持续时间、历史请求间隔和上下文增长情况估计剩余活跃度。三类信息分别描述未来入口、KV 增量以及后台预放置可被后续请求利用的机会。
- **约束感知的长期路由决策。**对于包含 N 个计算节点的部署，调度器将“计算节点 + KV 获取方式”作为联合动作，构造包含 $3N$ 个动作的固定离散动作空间。系统根据 E2E SLA、显存余量、节点可用性和 KV 版本一致性生成 action mask，将违反服务约束或状态约束的动作提前屏蔽；Dueling Double DQN 一次输出全部 $3N$ 个动作的长期成本估计，并只在未被 mask 的动作中进行比较和选择。
- **概率与活跃度感知的 KV 管理。**KV manager 维护版本化 block directory，将 Router 选择的 reuse、sync 和 recompute 落实为 block 级查询与缺失状态恢复；请求完成后，KV manager 再根据入口转移概率、session 剩余活跃度和后台资源余量，按比例向候选节点复制缺失的连续 KV blocks，以降低后续请求的按需 KV 同步量。

6.4.3 系统场景与状态同步框架

本文面向多基站协同的边缘推理场景：每个基站部署边缘服务器，边缘服务器承载大模型推理实例，并通过边缘网络互联。一次 Router 训练与部署周期内，参与协同的计算节点集合记为 $\mathcal{N}$ ，受监测物理链路集合记为 $\mathcal{E}$ ，节点数 $N = |\mathcal{N}|$ 和链路数 $E = |\mathcal{E}|$ 保持固定；运行过程中，节点负载、可用显存、链路状态以及节点是否可用可以动态变化，其中暂时不可用的节点通过 action mask 排除。用户在基站覆盖区域之间移动时，后续 request 的入口会发生变化，而同一 session 的历史 KV cache 可能仍保留在先前的计算节点。对于一个 LLM session，实际 request 数量和每次输出长度在在线服务时通常未知。

在该场景下，当前 request 的执行位置与 KV 获取方式不仅决定本次请求的转发、排队和状态恢复开销，还会改变后续请求可利用的 KV 分布。随着候选节点增加，联合动作数量和跨请求状态组合迅速增长，难以通过固定规则完整刻画。因此，本文采用 Dueling Double DQN 估计“候选计算节点 + KV 获取方式”联合动作的长期成本，并结合 E2E latency SLA、显存、节点可用性和 KV 一致性约束筛选可行动作。中心化 Router 最终输出目标计算节点；对于具有历史上下文的 request，同时输出 reuse、sync 或 recompute 三种 KV 获取方式之一。具体实验拓扑和链路参数在 6.4.8 中给出。

为了支撑中心化路由，系统设置中心状态管理模块，统一保存 Router 决策所需的三类运行信息：

- **节点状态：**各推理节点周期性上报 prefill、recompute 和 decode 的待处理工作量、可用于 KV cache 的显存比例，以及节点和模型是否可用。
- **链路状态：**网络监测模块周期性上报每条物理链路的可用带宽、RTT 和可用性。

- **KV 状态：**请求生成新 KV 或后台放置完成后，KV manager 上报当前 session 已经完整保存的连续 KV block 区间、版本和所在节点。只有完整传输并通过版本检查的 KV blocks 才写入中心目录。

各上报方只发送自身负责的状态或变化量，中心状态管理模块汇总后形成中心化 Router 所需的全局状态。每个 request 到达入口节点后，入口节点将 session 标识、当前入口、请求规模和 SLA 等路由特征发送给中心化 Router；中心化 Router 读取本地维护的全局状态快照，构造输入并检查候选动作是否可执行，随后返回计算节点与 KV 获取方式。

中心状态管理会引入节点与链路状态上报以及 KV 目录更新等通信。6.4.8 对上述通信过程进行模拟，并进一步给出不同负载和节点规模下的带宽占用结果。

主要符号。

符号	含义
k, K	当前请求编号和实际但在线未知的 session 请求总数
$\mathcal{N}, N, \mathcal{E}, E$	部署节点集合及其节点数、受监测物理链路集合及其链路数
i, j, n, s	入口节点、目标/计算节点、一般节点和 KV 同步源节点索引
$U_k, J_k, \Delta\tau_k$	当前入口、所选计算节点和相邻请求间隔
$P_{ij}^{(k)}$	第 k 个请求后从节点 i 到节点 j 的入口转移概率
$x_k, x_k^{\text{req}}, m_k, Y_k, Y_k^{\text{obs}}, \rho_k(y), \hat{Y}_k^{(p)}$	长度预测特征、Router 请求特征、移动阶段标志、实际/观测输出长度、长度预测分布和第 p 分位预测
$\mathbf{f}_k^{\text{sess}}, \sigma_k(h), H_c, H_{\text{ref}}, \hat{N}_k^{\text{remain}}, \eta_k$	Session 历史特征、继续至少 h 轮的概率、预测窗口、活跃度参考长度、预计剩余请求数和归一化活跃度
$Q_k, \mathcal{B}_k, \mathcal{B}_{k,n}$	当前 KV block 总数、完整 KV block 集合和节点 n 已有 block 集合
$v_{k,n}, r_{k,n}, \mathcal{C}_k$	节点 n 的 KV 版本、最高连续 block 编号和版本化分布状态
$\mathcal{M}_{k,j}, q_b, \Delta\mathcal{B}_k$	节点 j 缺失 blocks、block b 的大小和本轮新增 blocks
$\alpha_0, \alpha_{\text{max}}, \alpha_{k,j}$	KV placement 基础系数、单轮比例上限和节点 j 的动态放置比例
$D_{k,j}^{\text{missing}}, D_{k,j}^{\text{plan}}, D_{k,j}^{\text{actual}}$	节点 j 缺失、计划放置和实际完成的 KV 字节数
$b_{k,j}^{\text{alloc}}, B_{i,j}^{\text{bg}}, \Delta t_k^{\text{bg}}, \mathcal{T}_{i,j}(t)$	Placement 任务分配带宽、链路后台带宽预算、调度时间片和链路上的在途任务集合
$M_{k,j}^{\text{place,free}}$	节点 j 当前可用于后台 KV placement 的剩余容量
Φ_k^{place}, H	第 k 轮实际完成的后台 KV 放置结果和完整系统状态转移函数
$a_k = (J_k, Z_k)$	计算节点与 KV 获取方式组成的联合动作
a, a', y, u'	当前候选动作、下一候选动作、输出长度取值和下一入口节点取值
Z_k	KV 获取方式：reuse、sync 或 recompute
$\mathcal{A}_k^F(s_k)$	满足 SLA、显存、版本和节点可用性约束的动作集合
$\mathcal{N}_k^{\text{avail}}, \chi_k^{\text{KV}}(a)$	当前可用节点集合和动作 a 的 KV 版本/block 可用性指示函数

符号	含义
$D_k^{\text{input}}, D_k^{\text{request}}$	输入数据量和跨节点请求/响应总字节数
$D_k^{\text{KV}}, BW_{i,j}^{\text{eff}}$	KV 同步字节数和节点间实测有效带宽
$L_k^{\text{new}}, L_k^{\text{ctx}}, L_{k,j}^{\text{miss}}$	当前轮新输入长度、历史上下文长度和节点 j 缺失的连续历史后缀长度
$\tau_j^{\text{prefill}}(L_0, \Delta L)$	节点 j 在已有 L_0 个连续 prefix tokens 时增量处理 ΔL 个 tokens 的实测时间
$T_k^{\text{comm}}, T_k^{\text{state}}, T_k^{\text{prefill}}$	请求通信、KV 状态准备和 prefill 时间
$T_k^{\text{queue}}, T_k^{\text{decode}}, T_k^{\text{return}}$	排队、decode 和结果返回时间
T_k^{E2E}	从请求发出到全部输出返回的端到端时延
$w_1, w_2, w_3, \bar{c}_k, c_k$	时延与传输成本权重、条件即时成本和当前期望成本
$\hat{T}_k^{\text{E2E}}, \Delta_k(a), SLA_k$	预测端到端时延、估计误差安全裕量和请求 SLA 上限
$\widehat{M}_k, M_k^{\text{reserve}}, M_{J_k}^{\text{free}}$	预测显存需求、安全预留显存和计算节点剩余显存
$s_k, s_{k+1}^{a,y,u'}, s_{k+1}^{\text{obs}}$	当前系统状态、模型化下一状态和观测下一状态
$\mathbf{L}_k, \mathbf{M}_k, \mathbf{W}_k$	系统状态中的负载、显存和链路状态向量
$\pi, \gamma, \mathcal{Q}^*$	路由策略、折扣因子和最优状态-动作价值
$\mathbf{o}_k, d_{\text{in}}(N, E), \mathcal{Q}_\theta, \mathcal{Q}_{\bar{\theta}}, V_\theta, A_\theta$	DQN 输入向量及其维度、在线网络、目标网络、状态价值分支和动作优势分支
$a_k^{\text{enum}}, a_k^*, a_{k+1}^{\text{sel}}$	单请求穷举动作、在线执行动作和下一状态动作选择
V_{k+1}, z_k	目标网络估值和 Bellman 训练目标

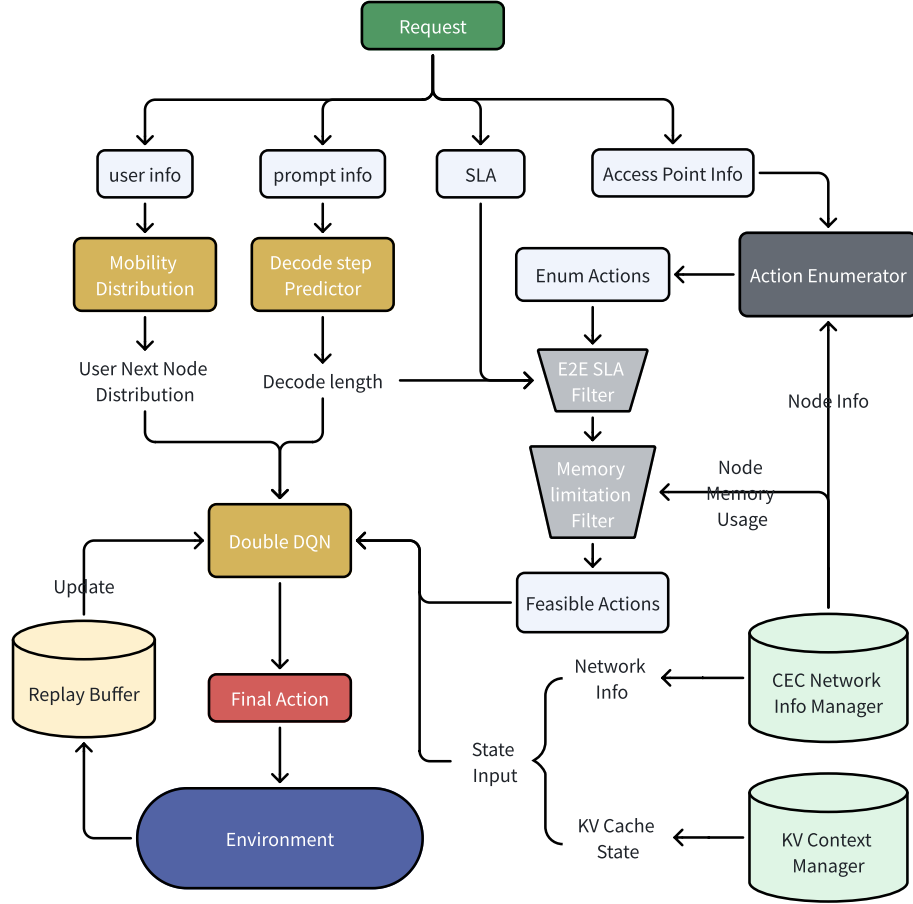


图 21 单个请求到达后的长期成本感知卸载决策流程。Request 被拆分为用户信息、prompt 信息和 SLA 信息：用户信息进入 mobility predictor 生成未来入口分布，prompt 信息进入 output length predictor 估计 decode 长度，中心状态管理模块与 KV context manager 分别提供节点负载、链路带宽、剩余显存以及 KV block 位置/版本等运行状态。State builder 将移动预测、长度预测、SLA、系统遥测和 KV 状态汇聚为 feature bundle；action enumerator 枚举“计算节点 + KV 获取方式”候选，per-action estimator 对每个候选动作估计请求传输、KV 同步/重算、prefill、decode 和显存占用。E2E SLA filter 与 Memory/KV filter 先处理时延、显存和 KV 可用性等硬约束，得到 feasible actions 作为 Dueling Double DQN 的 action mask；Dueling Double DQN router 一次输出固定 $3N$ 个动作的长期成本估计，并通过 action mask 只在可行动作中比较和输出 final action。环境执行后的实际 E2E 时延、输出长度、KV 增量、传输量和显存变化写入 replay buffer，用于回传更新预测器、估计器和路由策略。

6.4.4 入口转移、输出长度、会话持续性与 KV 状态

令 $U_k \in \mathcal{N}$ 为第 k 个请求的入口节点，入口转移矩阵给出

$$P_{ij}^{(k)} = \Pr(U_{k+1} = j \mid U_k = i).$$

矩阵可由历史入口序列、移动预测器或给定的工作负载模型获得，并允许随请求阶段更新；本轮模拟采用的具体转移过程集中列于 6.4.8。

输出 token 数 Y_k 在决策时未知，长度预测器根据请求特征 x_k 给出回复长度分布

$$\rho_k(y) = \Pr(Y_k = y \mid x_k), \quad \hat{Y}_k^{(p)} = \inf \left\{ y : \sum_{\ell \leq y} \rho_k(\ell) \geq p \right\}.$$

其中 $\hat{Y}_k^{(p)}$ 是第 p 分位长度预测，用于 SLA 与显存等硬约束过滤；完整分布 ρ_k 则用于长期价值估计。类似的长度预测已用于不可预测请求调度 [63]。长度预测不仅用于当前请求的 E2E latency 和显存需求估计，也进入动作价值评估：长输出会增加 decode 时间、返回流量、KV 增量和后续显存压力，从而改变 Dueling Double DQN 对不同计算节点与 KV 获取方式的偏好。

由于 session 的实际请求总数 K 在线未知，session continuation predictor 根据历史特征 $\mathbf{f}_k^{\text{sess}}$ 估计当前 session 在未来至少继续 h 个请求的概率：

$$\sigma_k(h) = \Pr(K \geq k + h \mid \mathbf{f}_k^{\text{sess}}).$$

特征 $\mathbf{f}_k^{\text{sess}}$ 可包含当前请求编号、session 已持续时间、历史请求间隔、任务类型和上下文增长速度。给定预测窗口 H_c ，预计剩余请求数和归一化活跃度分别为

$$\hat{N}_k^{\text{remain}} = \sum_{h=1}^{H_c} \sigma_k(h), \quad \eta_k = \min \left\{ 1, \frac{\hat{N}_k^{\text{remain}}}{H_{\text{ref}}} \right\},$$

其中 H_{ref} 是活跃度归一化参数。该预测使仍有较多后续请求的 session 获得更多增量放置机会，并降低临近结束的 session 对后台资源的占用。

KV 状态表示为版本化的节点分布：

$$\mathcal{C}_k = \{(n, v_{k,n}, r_{k,n}, \mathcal{B}_{k,n}) \mid n \in \mathcal{N}\},$$

其中 $v_{k,n}$ 、 $r_{k,n}$ 和 $\mathcal{B}_{k,n}$ 分别表示节点 n 的 KV 版本、最高连续 block 编号和已有 block 集合；节点 j 缺失的 blocks 为 $\mathcal{M}_{k,j} = \mathcal{B}_k \setminus \mathcal{B}_{k,j}$ 。Router 使用版本和连续 prefix 信息判断 reuse、sync 与 recompute 的可行性，KV manager 则从 $r_{k,n}$ 之后开始执行增量放置。

6.4.5 联合动作、成本与约束

动作定义为

$$a_k = (J_k, Z_k), \quad Z_k \in \{\text{reuse}, \text{sync}, \text{recompute}\},$$

其中 J_k 是计算节点， Z_k 是该节点获取 KV 的方式。若 $U_k \neq J_k$ ，请求自动转发；存在多个副本时，系统自动选择同步时间最短的源节点。由于动作价值同时考虑当前入口、预测未来入口和 KV 状态位置，策略可以选择并非当前入口最近的计算节点，以减少后续多轮 request 的状态同步或重算成本。reuse 仅在目标节点 J_k 已经持有当前请求所需的连续 KV prefix 或 blocks 时可行；若目标节点缺失必要 block，则该节点只能选择 sync 或 recompute。

对具有历史 KV 的请求，若节点 j 已经持有全部版本一致的最近历史 blocks，则该节点的可行

方式集合固定为 $\{\text{reuse}\}$ ；否则 reuse 不可行，系统再检查 sync 和 recompute 的条件。该规则允许多个完整副本节点同时开放 reuse ，但不为同一完整副本保留冗余的 sync 或 recompute 动作。Session 首请求没有历史状态，直接在可用节点执行 fresh prefill 。

请求通信、KV 状态准备和 prefill 时间分别为

$$T_k^{\text{comm}}(a_k) = \mathbf{1}\{U_k \neq J_k\} \frac{D_k^{\text{input}}}{BW_{U_k, J_k}^{\text{eff}}},$$

$$T_k^{\text{state}}(a_k) = \begin{cases} \frac{\sum_{b \in \mathcal{M}_{k, J_k}} q_b}{BW_{s, J_k}^{\text{eff}}}, & Z_k = \text{sync}, \\ 0, & Z_k \in \{\text{reuse}, \text{recompute}\}, \end{cases}$$

$$T_k^{\text{prefill}}(a_k) = \begin{cases} \tau_{J_k}^{\text{prefill}}(L_k^{\text{ctx}}, L_k^{\text{new}}), & Z_k \in \{\text{reuse}, \text{sync}\}, \\ \tau_{J_k}^{\text{prefill}}(L_k^{\text{ctx}} - L_{k, J_k}^{\text{miss}}, L_{k, J_k}^{\text{miss}}) + \tau_{J_k}^{\text{prefill}}(L_k^{\text{ctx}}, L_k^{\text{new}}), & Z_k = \text{recompute}. \end{cases}$$

其中 $\tau_{J_k}^{\text{prefill}}(L_0, \Delta L)$ 表示在已有 L_0 个连续 prefix tokens 的条件下，对后续 ΔL 个 tokens 执行增量 prefill 的时间。目标节点已有的连续 KV prefix 可被 recompute 直接复用，因此 recompute 只重建缺失的历史后缀；该后缀仍需对已有 prefix 执行 attention ，其计算量不是从零处理同等数量 tokens 的代价。 reuse 、 sync 和 recompute 随后均需处理当前轮新输入。完整请求时延为

$$T_k^{\text{E2E}} = T_k^{\text{comm}} + T_k^{\text{queue}} + T_k^{\text{state}} + T_k^{\text{prefill}} + T_k^{\text{decode}}(Y_k) + T_k^{\text{return}}(Y_k),$$

它已经包含首 token 响应阶段。给定输出长度 y 时，动作 a_k 的条件即时成本定义为

$$\bar{c}_k(s_k, a_k, y) = w_1 T_k^{\text{E2E}}(s_k, a_k, y) + w_2 D_k^{\text{request}}(a_k, y) + w_3 D_k^{\text{KV}}(a_k).$$

由于输出长度未知，决策时的当前期望成本为

$$c_k(s_k, a_k) = \mathbb{E}_{Y_k \sim \rho_k} [\bar{c}_k(s_k, a_k, Y_k)].$$

其中 T_k^{E2E} 衡量完整请求时延； D_k^{request} 是入口与计算节点不同时传输的输入及输出字节数； D_k^{KV} 是为当前动作按需同步的 KV 字节数。权重 w_1, w_2, w_3 用于完成单位归一化并反映时延、请求转发流量和前台 KV 同步流量的重要性。后台 KV placement 受独立的带宽和容量预算约束，不计入当前请求的关键路径成本；其完成结果通过下一状态中的 KV 分布影响后续动作价值。请求完成后，系统用实际 Y_k 和实测开销生成 replay 样本。

系统使用输出长度高分位预测 $\hat{Y}_k^{(p)}$ 构造可行动作集合

$$\mathcal{A}_k^F(s_k) = \left\{ a \left| \begin{array}{l} \hat{T}_k^{\text{E2E}}(a, \hat{Y}_k^{(p)}) + \Delta_k(a) \leq SLA_k, \\ \hat{M}_k(a, \hat{Y}_k^{(p)}) + M^{\text{reserve}} \leq M_{J_k}^{\text{free}}, \\ J_k \in \mathcal{N}_k^{\text{avail}}, \quad \chi_k^{\text{KV}}(a) = 1 \end{array} \right. \right\},$$

其中 $\mathcal{N}_k^{\text{avail}}$ 表示当前可用且部署了匹配模型/adapter 版本的节点集合, $\chi_k^{\text{KV}}(a)$ 表示 KV 获取方式的版本与 block 可用性检查: reuse 要求 J_k 已有所需连续 prefix/blocks, sync 要求至少存在可同步的版本一致副本且链路可用, recompute 要求目标节点可访问完整上下文并有足够计算与显存余量。本文对输出长度采用分层处理: 可行动作过滤使用高分位预测 $\hat{Y}_k^{(p)}$, 以降低输出长度低估造成的 SLA violation 与显存不足; 长期价值估计则保留长度分布 ρ_k , 对不同输出长度造成的即时成本和下一状态变化取期望, 避免策略过度保守。

单请求穷举仅在可行动中选择当前成本最低者:

$$a_k^{\text{enum}} = \arg \min_{a \in \mathcal{A}_k^F(s_k)} c_k(s_k, a).$$

它能够响应当前负载和链路状态, 但不考虑本次同步或重算对后续 KV 复用的影响, 因此作为本文长期策略的短视对照, 也作为 RL 异常时的安全回退。

6.4.6 长期优化与动作掩码 Dueling Double DQN

将第 k 个请求到达时的状态写为

$$s_k = (x_k^{\text{req}}, m_k, \rho_k, \sigma_k, \mathcal{C}_k, \mathbf{L}_k, \mathbf{M}_k, \mathbf{W}_k),$$

其中 x_k^{req} 汇总当前入口、输入与上下文规模、请求字节数和 SLA, m_k 是静止/移动阶段标志; 其余各项分别描述输出长度分布、session 持续性、KV 状态、负载、显存和链路。固定入口转移矩阵作为环境参数刻画下一请求入口的条件分布。有限时域优化问题为

$$\begin{aligned} \min_{\pi} \quad & \mathbb{E}_{\pi, U, Y} \left[\sum_{k=1}^K \gamma^{k-1} c_k(s_k, a_k) \right] \\ \text{s.t.} \quad & a_k = \pi(s_k), \quad a_k \in \mathcal{A}_k^F(s_k), \\ & U_{k+1} \sim P^{(k)}(\cdot | U_k), \\ & Y_k \sim \rho_k(\cdot | x_k). \end{aligned}$$

其中 $c_k(s_k, a_k)$ 是对当前输出长度分布取期望后的即时成本。为避免在优化问题中展开完整系统动力学，记

$$s_{k+1}^{a,y,u'} = H(s_k, a_k, y, u', \Delta\mathcal{B}_k(a_k, y), \Phi_k^{\text{place}})$$

为在当前动作 a_k 、实际输出长度 y 、下一入口 u' 、KV 增量和后台放置结果共同作用后的下一状态。请求完成后，KV manager 根据本轮更新后的 KV 分布生成后台任务， Φ_k^{place} 只记录在下一请求到达前已经完整传输并通过版本检查的 blocks。该转移函数同时更新入口节点、负载、显存、链路状态和版本化 KV 状态；后台 placement 由 KV manager 独立执行，其已完成结果通过下一状态影响 Router。对应的理论最优状态-动作价值满足

$$\begin{aligned} \mathcal{Q}^*(s_k, a_k) = & \sum_y \rho_k(y) [\bar{c}_k(s_k, a_k, y) \\ & + \gamma \sum_{u' \in \mathcal{N}} P_{U_k u'}^{(k)} \min_{a' \in \mathcal{A}_{k+1}^F(s_{k+1}^{a,y,u'})} \mathcal{Q}^*(s_{k+1}^{a,y,u'}, a')] \end{aligned}$$

入口变化既影响当前转发成本，也通过 $P_{U_k u'}^{(k)}$ 影响未来价值；输出长度不仅影响当前 decode 和显存，还通过 $\Delta\mathcal{B}_k(a_k, Y_k)$ 改变后续 KV 规模；后台增量放置则通过 Φ_k^{place} 改变候选节点的连续 prefix，使 Router 在后续请求中重新比较转发、reuse、sync 和 recompute。这延续了 QoS-LLM-Router 的长期 QoS 优化 [62] 和 MGCO 的非短视移动卸载思想 [68]，并将 CA-AIGC-Migration 的移动 context 管理 [75] 细化到版本化 KV blocks。

上式定义了希望逼近的 $\mathcal{Q}^*$ 。对短 session、离散状态且已知真实轨迹的小规模实验，可用有限时域动态规划或搜索得到离线 oracle；但是一般场景中同一 session 的 request 数量 K 不确定，且存在动作序列和状态爆炸，因此采用 Dueling Double DQN 近似求解。其中，Double DQN 使用在线网络选择动作、目标网络评价动作，Dueling 结构分别估计状态价值和各动作的相对优势。对于部署时确定的 N 个计算节点，每个节点对应 reuse、sync 和 recompute 三种 KV 获取方式，因此网络一次输出 $3N$ 个 Q 值。在执行 $\arg \min$ 或构造下一状态目标时，将 $a \notin \mathcal{A}_k^F(s_k)$ 的 Q 值置为 $+\infty$ ，使不可行动作不参与选择。

Replay buffer 中记录的是执行后的实际转移 s_{k+1}^{obs} ，其中已经包含本轮真实 Y_k 、KV 增量、后台 placement 完成量、显存变化和下一入口。Dueling Double DQN 在该观测下一状态上先用在线网络选择动作，再用目标网络评价：

$$\begin{aligned} a_{k+1}^{\text{sel}} &= \arg \min_{a' \in \mathcal{A}_{k+1}^F(s_{k+1}^{\text{obs}})} \mathcal{Q}_\theta(s_{k+1}^{\text{obs}}, a'), \\ V_{k+1} &= \mathcal{Q}_{\bar{\theta}}(s_{k+1}^{\text{obs}}, a_{k+1}^{\text{sel}}), \\ z_k &= \bar{c}_k(s_k, a_k, Y_k^{\text{obs}}) + \gamma V_{k+1}. \end{aligned}$$

即 replay 训练目标等于“观测输出长度下的即时成本 + 折扣后的观测下一状态价值”。模型化 roll-out 版本按照 Bellman 式对输出长度 y 和下一入口 u' 共同取期望，并计算对应的 $\bar{c}_k(s_k, a_k, y)$ 与下一状态价值。在线动作则为 $a_k^* = \arg \min_{a \in \mathcal{A}_k^F(s_k)} Q_\theta(s_k, a)$ 。若模型异常则回退至可行动作中的单请求穷举方案；若集合为空，则排队、降级或拒绝。

Dueling Double DQN 的输入、网络结构与动作输出。 上述长期成本模型给出了 Dueling Double DQN 需要学习的状态-动作价值及其训练目标。对于包含 N 个计算节点和 E 条受监测物理链路的部署，State Builder 将状态 s_k 转换为输入向量 $\mathbf{o}_k \in \mathbb{R}^{d_{\text{in}}(N, E)}$ ，其维度为

$$d_{\text{in}}(N, E) = (N + 16)_{\text{request}} + 1_{\text{phase}} + (9N + 3E)_{\text{central}} = 10N + 3E + 17.$$

具体输入包括：

- **请求与预测信息 ($N + 16$ 维)。** 当前入口节点的 one-hot 编码为 N 维；输入 token 数、历史上下文长度、请求字节数和 SLA 共 4 维；输出长度分布使用 8 个区间概率表示；session 在未来 1、2、4、8 轮继续的概率占 4 维。
- **请求阶段 (1 维)。** 表示当前请求所处的静止或移动阶段。
- **中心状态信息 ($9N + 3E$ 维)。** 每个计算节点包含 5 个负载、显存和可用性字段，以及 4 个当前 session 的 KV 状态字段，共 $9N$ 维；每条受监测物理链路包含可用带宽、RTT 和可用性 3 个字段，共 $3E$ 维。

输入编码。 State Builder 将全部字段转换为 float32，并按照字段含义归一化：

- **概率、比例和布尔信息：** 概率与比例保持在 $[0, 1]$ 区间，布尔信息使用 0 或 1 表示。
- **请求规模和待处理工作量：** token 数、数据量和各阶段待处理工作量采用对数归一化。
- **带宽和显存：** 按照系统配置的带宽或显存上限归一化。
- **时延信息：** RTT、预计恢复时间等时延字段按照当前请求的 SLA 归一化。

网络结构与动作输出。

- **共享状态编码。** 共享全连接网络从 $\mathbf{o}_k$ 中提取状态特征，供价值分支和优势分支共同使用。
- **Dueling 输出。** 价值分支输出一个状态价值 $V_\theta(s_k)$ ，优势分支一次输出固定动作空间中 $3N$ 个动作的优势 $A_\theta(s_k, a)$ 。将动作记为 $a_1, \dots, a_{3N}$ ，则

$$Q_\theta(s_k, a_\ell) = V_\theta(s_k) + A_\theta(s_k, a_\ell) - \frac{1}{3N} \sum_{r=1}^{3N} A_\theta(s_k, a_r).$$

- **动作过滤与选择。** 一次网络前向输出 $3N$ 个 Q 值，分别对应在各计算节点上采用 reuse、sync 或 recompute 的长期成本。系统为这些输出生成 action mask；节点暂时不可用、模型不可用或 KV 获取条件不满足时，将相应动作的 Q 值置为 $+\infty$ ，最终只在可行动作中选择 Q 值最小者。

每个请求到达后，入口节点将路由特征交给中心化 Router；Router 读取中心状态管理模块维护的全局状态快照，再运行 Dueling Double DQN 选择执行节点和 KV 获取方式。该过程的请求关键路径开销由路由特征传递、状态编码、预测器执行和路由模型前向计算组成，其中路由模型前向计算的测量结果见 6.4.8。

6.4.7 分布式 KV 管理与后台增量预放置

KV manager 维护跨节点 block directory，记录 model/adaptor 版本、prefix hash、token 区间、block 版本、大小和所在节点。Router 输出计算节点与 KV 获取方式后，KV manager 根据目录完成连续 prefix 查询和版本检查；对于 sync，系统从持有一致版本的节点中选择预计传输时间最短的源节点，并只传输目标节点缺失的 blocks。该目录同时记录后台预放置已经完成的 blocks，使后续 Router 可以在最新 KV 分布上评估 reuse、按需 sync 和 recompute。该设计借鉴了 Mooncake 的 KVCache-centric 全局存储思想 [13] 以及 prefill/decode 分离系统中的 KV handoff 机制 [4]。

为降低用户移动后的按需 KV 同步压力，系统在每个请求执行完成并写入新增 KV blocks 后触发后台增量预放置。设本轮完成后 session 共包含 Q_k 个 KV blocks，节点 j 已持有前 $r_{k,j}$ 个连续 blocks，则其缺失连续 KV 的总字节数为

$$D_{k,j}^{\text{missing}} = \sum_{\ell=r_{k,j}+1}^{Q_k} q_{b_\ell}.$$

系统根据入口转移概率和 session 活跃度确定节点 j 的动态放置比例：

$$\alpha_{k,j} = \text{clip} \left(\alpha_0 P_{U_{k,j}}^{(k)} \eta_k, 0, \alpha_{\max} \right), \quad D_{k,j}^{\text{plan}} = \alpha_{k,j} D_{k,j}^{\text{missing}},$$

其中 α_0 控制后台放置的整体强度， $\alpha_{\max}$ 限制单轮最大复制比例。入口转移概率为零的节点不接收主动副本；其余候选节点按转移概率和 session 活跃度获得相应的计划复制比例。

后台执行器在调度时间片 Δt_k^{bg} 内，根据分配带宽和目标节点可用于 placement 的剩余容量确定实际复制量：

$$D_{k,j}^{\text{actual}} = \min \left\{ D_{k,j}^{\text{plan}}, b_{k,j}^{\text{alloc}} \Delta t_k^{\text{bg}}, M_{k,j}^{\text{place,free}} \right\}.$$

链路上的全部并发 placement 任务共同满足

$$\sum_{m \in \mathcal{T}_{i,j}(t)} b_m(t) \leq B_{i,j}^{\text{bg}}(t).$$

因此，相同的计划复制比例不会强制异构链路完成相同数据量，实际完成量仍由各自后台预算决定。执行器将 $D_{k,j}^{\text{actual}}$ 向下对齐到完整 block，并从 $r_{k,j} + 1$ 开始复制连续 blocks。只有完整传输且通过版

本检查的 blocks 才提交至 Φ_k^{place} 和 block directory；后续请求以目录中已提交的最长连续 prefix 为状态恢复起点，前台 sync 或 recompute 补齐仍然缺失的历史后缀。

该机制使 session continuation predictor 输出较高的活跃 session 获得更大的单轮复制比例，并通过多次请求完成事件逐步扩展候选节点的连续 KV prefix；预计剩余请求较少时， η_k 降低，后台传输量随之收缩。Router 在下一请求到达时读取更新后的目录，自主比较小体量请求转发与本地 reuse、剩余 KV sync、缺失历史后缀 recompute 的成本，从而利用后台 placement 已经减少的状态准备开销。

6.4.8 实验设置、端到端结果与系统开销

本小节围绕三个问题展开评估：第一，单请求穷举与 KV manager 相比就近执行能否降低请求时延和 SLA 违约率；第二，中心状态同步会产生多大的通信开销；第三，Dueling Double DQN 的单次前向计算是否会显著增加请求路由时间。实验首先统一说明模拟环境、工作负载、对比策略和统计口径，随后依次给出端到端性能、中心状态上报开销和 DQN 前向计算时间。

1. 评价范围。本轮端到端模拟采用单请求穷举和完整 KV manager，尚未实现 6.4.6 中的长期状态价值估计，也未训练 Dueling Double DQN。因此，端到端结果用于建立未利用跨请求长期收益时的性能基线；长期路由相对单请求穷举的增量收益不在本轮评价范围内。Dueling Double DQN 仅在三节点原型配置下进行网络前向计算微基准，用于估计 9 个动作参与比较时模型接入 Router 后的单次计算开销。

2. 实验设置。实验设置依次说明计算资源与时间近似、请求轨迹与移动过程、工作负载分布，以及对比策略和统计方法。

计算资源与时间近似。每个节点在模拟中配置 8 张 64 GB NPU AI 加速卡。单卡使用 BF16 精度时的峰值计算能力取 376 TFLOPS，片上高带宽内存（HBM）的峰值读写带宽取 1.6 TB/s；计算效率系数 0.4 和带宽效率系数 0.6 分别用于将理论峰值折算为模拟器采用的有效计算能力和有效内存带宽。Prefill 按每批 4 个请求估计服务时间。Decode 采用固定批量为 32 的吞吐近似：模拟器估计一批请求完成一次 token 生成迭代的总成本，再除以 32 得到单请求平均 decode service time。该处理用于近似模型权重被一批请求共同复用时的平均服务能力，不模拟请求动态加入或退出 decode batch 的逐 token 执行过程，因此结果用于同一成本模型下的策略相对比较，不等同于真实系统中单个请求的逐 token wall-clock latency。

模拟器在路由前拒绝即使处于无排队、无转发和无 KV 恢复开销的理想条件下仍无法满足 SLA 的请求，其余已接纳请求在节点内部按照 SLA 剩余裕量进行非抢占式排队。

请求轨迹与移动过程。每组实验持续 60 s，并在 5 个随机种子上重复。Session 在前 80% 测试时间内启动；测试前 50% 的请求保持原入口，后 50% 在每个 request 到达时以 0.8 概率保持当前入口，并

分别以 0.1 概率转移至另外两个入口，即

$$P^{(k)} = I_3 \quad \text{或} \quad P^{(k)} = \begin{bmatrix} 0.8 & 0.1 & 0.1 \\ 0.1 & 0.8 & 0.1 \\ 0.1 & 0.1 & 0.8 \end{bmatrix}.$$

三个入口的 session 数按 1:1:2 分配。基线总并发分别为 CodeLlama34B 高优 24、CodeLlama34B 普通 96 和 Qwen2-VL-7B-Instruct 24；150% 负载下对应增加至 36、144 和 36，即实际入口并发分别由 [6, 6, 12]、[24, 24, 48]、[6, 6, 12] 增加为 [9, 9, 18]、[36, 36, 72]、[9, 9, 18]。

工作负载分布。记 $\text{LN}_T(\mu, \sigma; [l, u])$ 为按均值 μ 、标准差 σ 参数化，并将采样值限制在 $[l, u]$ 内的截断 lognormal 分布。公共 system/coding prompt 在三个同模型节点上预计算并常驻，不计入各 session 独有的历史上下文。历史保留系数取 1，表示某个 session 先前各轮的输入和输出 tokens 全部进入下一轮的历史 prefix，不做窗口截断或摘要压缩。工作负载配置如下。

配置项	CodeLlama 高优	CodeLlama 普通	Qwen2-VL 普通
单轮输入	$\text{LN}_T(512, 512; [32, 4096])$ 纯文本 tokens	$\text{LN}_T(512, 512; [32, 4096])$ 纯文本 tokens	$\text{LN}_T(256, 256; [8, 2048])$ 纯文本 tokens，加 4 帧 1024×768 图像
输出 tokens	$\text{LN}_T(384, 384; [16, 4096])$	$\text{LN}_T(384, 384; [16, 4096])$	$\text{LN}_T(256, 256; [4, 2048])$
Session requests	$\text{LN}_T(9, 6; [2, 24])$	$\text{LN}_T(13, 10; [2, 32])$	$\text{LN}_T(16, 6; [2, 24])$
平均轮间隔	2 s	4 s	5 s
SLA	150 ms	500 ms	500 ms

Qwen2-VL 的每轮 4 帧图像在模型侧转换为 4144 个 visual tokens，这些 tokens 参与 prefill、decode 和 KV cache 大小估计；在网络侧，图像按每个 visual token 对应 512 B 的压缩媒体数据估算，得到约 2.1 MB/request 的跨节点输入负载。两种表示分别用于估计模型计算量与请求转发流量，不能将 visual-token 数直接理解为网络传输字节数。由于 60 s 观测窗口会截断较晚启动且尚未结束的 session，窗口内观测到的 request/session 均值可能低于生成分布的设定均值；表中参数描述完整 session 的生成过程，结果则只统计观测窗口内实际到达的请求。

对比策略、评价指标与统计方法。两种路由与 KV 配置分别在基础并发和 $1.5\times$ 并发下运行：

- **就近执行**。每个请求始终在当前入口节点执行，不根据其他节点的排队状态进行负载转移。
- **单请求穷举 +KV**。Router 枚举候选计算节点以及 reuse、sync 和 recompute 三种可行的 KV 获取方式，选择当前请求预计 E2E latency 最小的动作。KV manager 使用 block-level residual sync，仅传输目标节点缺失的 KV blocks，并在请求完成后利用后台带宽增量预放置连续 KV blocks。

Router 在本轮模拟中直接使用请求轨迹中已生成的输入长度、输出长度和 session 剩余轮数，以排除预测误差的影响。主要评价指标为平均 E2E latency、P50 TTFT 和 SLA 违约率；同时检查各业务的

P99 TTFT。每个配置在 5 个随机种子上重复，结果采用 $\bar{x}^{+(x_{\max}-\bar{x})}_{-(\bar{x}-x_{\min})}$ 表示，其中 $\bar{x}$ 、 $x_{\min}$ 和 $x_{\max}$ 分别为五次实验的均值、最小值和最大值；上下标表示五次实验的观测范围，不表示置信区间。

3. 端到端性能。表 22 比较两种策略在基础并发和 1.5× 并发下的服务性能。表中“同并发提升”表示单请求穷举 +KV 相对相同并发下就近执行的指标降幅；“跨配置提升”表示单请求穷举 +KV 在 1.5× 并发下，相对基础并发就近执行的指标降幅。

表 22 单请求穷举与 KV manager 的端到端性能

模型	并发设置	指标	就近执行	单请求穷举 +KV	同并发提升	跨配置提升
CodeLlama34B	基础并发	平均 E2E	241.32 ^{+14.98} _{-12.74}	188.02 ^{+8.83} _{-9.04}	22.09%	—
		P50 TTFT	42.24 ^{+1.12} _{-1.22}	27.77 ^{+0.91} _{-0.70}	34.27%	—
		SLA 违约率	5.95 ^{+1.79} _{-2.12}	0.55 ^{+0.31} _{-0.44}	90.81%	—
	1.5× 并发	平均 E2E	351.85 ^{+241.99} _{-77.36}	193.90 ^{+7.61} _{-5.24}	44.89%	19.65%
		P50 TTFT	61.51 ^{+17.43} _{-5.81}	30.94 ^{+0.83} _{-0.81}	49.69%	26.74%
		SLA 违约率	14.25 ^{+12.35} _{-5.38}	0.61 ^{+0.22} _{-0.18}	95.75%	89.83%
Qwen2-VL-7B	基础并发	平均 E2E	252.78 ^{+28.28} _{-24.01}	194.40 ^{+13.13} _{-15.50}	23.09%	—
		P50 TTFT	132.13 ^{+9.83} _{-15.58}	117.96 ^{+8.17} _{-6.49}	10.73%	—
		SLA 违约率	4.99 ^{+2.27} _{-2.04}	0.09 ^{+0.38} _{-0.09}	98.10%	—
	1.5× 并发	平均 E2E	261.84 ^{+29.26} _{-21.78}	193.70 ^{+15.40} _{-17.68}	26.02%	23.37%
		P50 TTFT	141.23 ^{+16.55} _{-6.11}	118.78 ^{+9.30} _{-7.91}	15.90%	10.10%
		SLA 违约率	6.44 ^{+1.07} _{-1.37}	0.07 ^{+0.29} _{-0.07}	98.88%	98.56%

在相同并发条件下,单请求穷举 +KV 均优于就近执行;在 1.5× 并发下,CodeLlama34B 和 Qwen2-VL 的平均 E2E latency 分别降低 44.89% 和 26.02%。作为项目承载能力的补充口径，再将 1.5× 并发下的单请求穷举 +KV 与基础并发下的就近执行比较，两类模型的平均 E2E latency 分别降低 19.65% 和 23.37%，P50 TTFT 分别降低 26.74% 和 10.10%，SLA 违约率分别降低 89.83% 和 98.56%。该跨配置结果同时包含策略变化和负载变化，不作为同等条件下的算法效果比较。分业务的 P99 TTFT 分别为 CodeLlama 高优 181.67 ms、CodeLlama 普通 201.39 ms 和 Qwen2-VL 318.49 ms；这些数值用于观察首 token 尾时延，完整 SLA 违约率仍按请求 E2E latency 统计。

这些结果表明，单请求穷举与完整 KV manager 已能显著降低当前请求的排队和状态恢复开销，但尚不能说明跨请求长期状态价值的收益。后者需要在相同工作负载和 KV manager 配置下，进一步比较单请求穷举与训练后的长期 Router。

4. 中心状态上报开销。在中心化调度架构中，完整强化学习状态由中心状态管理模块维护，并由中心化 Router 在控制平面内读取。状态同步通信主要来自推理节点、网络监测模块和 KV manager 向中心状态管理模块上报状态；入口节点只向 Router 提交 session 标识、当前入口、请求规模和 SLA 等请求级路由特征。以下先计算状态上报的带宽占用，再说明中心化 Router 使用的状态字段，并分析节点规模扩展趋势。

4.1 状态上报机制与带宽占用。推理节点和网络监测模块分别周期性上报节点状态和链路状态；请求执行或后台 KV 放置完成后，KV manager 将同一节点、同一 session 新增的连续 KV block 合并为一条区间更新。表 23 给出三类上报消息的内容与传输大小。每条消息均计入 128 B 的 RPC、传输协议和数据对齐开销。

表 23 中心状态上报消息的内容、触发方式与传输大小

来源	上报内容	触发方式	消息内容	传输大小
推理节点	Prefill、recompute 和 decode 的待处理工作量，可用 KV 显存比例，以及节点可用性	每 1 ms	48 B	176 B
网络监测模块	可用带宽、RTT 和链路可用性	每 10 ms	32 B	160 B
KV manager	Session、KV 版本、节点、连续 block 区间和对应字节数	请求执行或后台放置完成后	80 B	208 B

实验时长为 60 s，包含 3 个推理节点和 3 条受监测物理链路。因此，每条轨迹固定产生 180,000 条节点状态消息和 18,000 条链路状态消息。KV 区间更新数依据请求轨迹估算：每个请求对应一条执行更新；对于移动阶段内尚未结束的 session，两个候选节点各计一条合并后的后台预放置更新。该方法用于估算 KV 状态上报量，不模拟后台任务队列的执行顺序。实验将中心状态管理模块部署在节点 B，表 24 给出 5 个随机种子上的平均消息数，以及实际经过节点间物理链路的上报数据量。

表 24 60 s 模拟中的中心状态上报开销

负载	平均节点上报数	平均链路上报数	平均 KV 更新数	60 s 数据量 (MB)	平均带宽 (Mbps)
基础并发	180,000	18,000	2,277.6	22.402	2.987
1.5× 并发	180,000	18,000	3,475.4	22.576	3.010

4.2 中心状态字段与输入维度。设系统包含 N 个节点和 E 条受监测物理链路，中心状态管理模块为 Router 维护的全局状态包含每节点 5 个运行状态字段、每节点 4 个当前 session 的 KV 状态字段，以及每条链路 3 个网络状态字段，总维度为

$$d_{\text{central}}(N, E) = 9N + 3E.$$

在当前三节点、三链路系统中，中心状态管理模块维护 36 个中心状态字段。表 25 按信息类别列出生字段含义、数据类型和原始大小。

表 25 中心化 Router 使用的中心状态字段（3 个节点、3 条链路）

信息类别	字段含义	字段数量与类型	原始大小
节点运行状态	每个节点的 prefill、recompute 和 decode 待处理工作量，可用 KV 显存比例，以及节点可用性	12 个 float32，3 个 uint8	51 B
当前 session 的 KV 状态	每个节点是否具有完整且版本一致的 KV、连续 prefix 比例、剩余同步时间和缺失历史后缀重算时间	9 个 float32，3 个 uint8	39 B
网络状态	每条物理链路的可用带宽、RTT 和可用性	6 个 float32，3 个 uint8	27 B
合计	当前三节点系统维护的全部中心状态	36 个字段	117 B

每节点 9 个字段由 5 个节点运行状态字段和 4 个当前 session 的 KV 状态字段组成。后 4 个字段由中心状态管理模块根据 KV 目录以及节点、链路状态生成。中心化 Router 在控制平面内直接读取并编码这 36 个字段；若统一编码为 float32，中心状态部分占 144 B。本轮实验将这些字段与 19 个请求及预测字段、1 个实验阶段字段拼接，构成 Dueling Double DQN 的 56 维输入。

4.3 当前三节点系统的总体状态上报带宽占用。表 26 给出两种负载下的实际节点间状态上报流量及最高单链路带宽。

表 26 当前三节点系统的中心状态上报通信开销

负载	状态上报 (Mbps)	最高单链路 (Mbps)	最高单链路占比 (%)
基础并发	2.987	1.575	0.0016
1.5× 并发	3.010	1.596	0.0016

三个节点按照 1 ms 周期合计每秒产生 3,000 条节点状态消息，构成中心状态同步流量的主体。即使采用每节点每秒 1,000 次的高频状态更新，1.5× 并发下的节点间状态上报流量仍约为 3.01 Mbps，最高单链路约为 1.60 Mbps，占对应 100 Gbps 链路的约 0.0016%。

4.4 节点规模扩展。节点数量增加时，即使不考虑请求完成后触发的 KV 目录更新，仅节点负载、显存余量、节点可用性、链路带宽和 RTT 等定期状态上报也会随系统规模增加。表 27 使用全连接拓扑 $E = N(N - 1)/2$ 估算不同节点规模下的节点与链路定期状态上报流量。该表只用于观察固定状态上报随节点数增加的趋势；为使结果不依赖中心状态管理模块的具体部署位置，表中假设这些定期上报均经过节点间链路。表中结果不包含随请求完成和后台 KV placement 变化而触发的 KV 区间更新。

表 27 节点数量增加时的节点与链路定期状态上报开销估算

节点数 N	链路数 E	定期状态上报 (Mbps)
3	3	4.608
10	45	19.840
25	300	73.600
50	1225	227.200
100	4950	774.400

在表 27 中，三节点三链路的 4.608 Mbps 来自 $3 \times 1000 \times 176\text{B}$ 的节点定期状态上报和 $3 \times 100 \times 160\text{B}$ 的链路定期状态上报全部跨节点传输后的带宽；表 26 中的 3.010 Mbps 则是在中心状态管理模块部署于节点 B 时，加入实际 KV 区间更新并按实际经过物理链路的数据量统计得到。因此，表 27 不是完整控制面状态同步开销，也不是其上限，而是单独估算节点和链路固定状态上报随规模增长的部分。在稀疏拓扑中，若 $E = O(N)$ ，状态字段数和通信量随节点数近似线性增长；若中心状态管理模块维护全连接拓扑中的全部链路状态，则 $E = O(N^2)$ ，链路状态上报量将随节点数呈二次增

长。三节点原型的 56 维输入和 9 维动作输出由本轮实验拓扑确定；部署到其他固定节点规模时，需要相应调整状态编码和动作输出层，并重新训练 Router。

5. 在线路由决策开销。

每个请求正式转发或执行之前，中心化 Router 基于请求特征和中心状态快照运行 Dueling Double DQN 选择动作。本轮模拟采用三个计算节点，路由模型的具体配置如下：

- **模型输入。**将 $N = 3$ 、 $E = 3$ 代入 6.4.6 的输入维度公式，得到 19 个请求与预测字段、1 个实验阶段字段和表 25 所列的 36 个中心状态字段，共 56 维。
- **动作输出。**三个计算节点分别对应 reuse、sync 和 recompute，优势分支输出 9 个动作优势，网络据此得到 9 个 Q 值。
- **网络参数与测量设置。**共享 MLP 包含两层 128 单元全连接层和一维价值分支，共 25,098 个 float32 参数；使用 Apple M4 CPU 单线程 NumPy，预热 2,000 次后测量 20,000 次。

表 28 给出该配置的单次前向计算时间。

表 28 三节点原型 Dueling Double DQN 的单次前向计算时间

实现	参数量	P50 (ms)	P95 (ms)	P99 (ms)
float32 CPU，单线程	25,098	0.010	0.012	0.016

三节点原型 Dueling Double DQN 前向计算的 P99 为 0.016 ms。该结果为独立微基准，尚未加入前述端到端结果表。完整实现还需要测量中心化 Router 的状态编码、输出长度预测器和 session continuation predictor 的执行时间；这些耗时与 DQN 前向计算共同计入请求 E2E latency。

6. 本轮结论与实验边界。本轮实验验证了单请求穷举与完整 KV manager 的组合效果，并分别估算了中心状态同步和 Dueling Double DQN 前向计算的开销。在当前三节点中心化 agent 场景中，高频状态上报产生的跨节点流量相对链路容量较小，三节点原型 DQN 前向计算的已测 P99 为 0.016 ms。端到端性能表尚未计入完整在线决策开销，也未实现长期 Dueling Double DQN；后续实验需要补充状态编码、预测器执行与长期 Router 的完整在线时延，单请求穷举与长期 Router 的对照，以及 residual sync 与后台预放置的消融。

6.5 小结

总体来看，CEC 中的 LLM inference task offloading 已超出传统“数据量、CPU cycles 与 deadline”驱动节点选择问题。系统既要路由完整请求和连续 session，也要编排多模型/多 agent workflow，并管理 context、prefix cache、KV cache 和 agent memory。用户移动进一步改变请求入口、网络路径和历史状态位置；输出长度不确定性则影响 decode 时间、返回流量、显存占用和新增 KV 规模。二者

共同使一次卸载动作持续影响后续多轮服务。

现有研究呈现三条主线：request/session routing 从就近执行转向质量、SLA、队列和显存联合感知；multi-model/multi-agent orchestration 从单模型选择扩展到角色、模型、工具和中间状态的跨节点编排；runtime-state management 则将 KV 和 context 视为可驻留、复制、同步、驱逐或重算的系统资源。三者共同表明，LLM 卸载需要从 computation-aware 走向 computation-state co-optimization。

面向项目场景，本章构建“状态同步-入口转移与长度/会话持续性建模-显式可行性过滤-在线请求路由-后台增量 KV 预放置-运行反馈”的闭环。中心状态管理模块为 Router 提供节点负载、链路、显存和 KV 目录信息；Router 在满足 SLA、显存、节点可用性和 KV 版本一致性约束的候选动作中，联合选择计算节点与 reuse、sync 或 recompute 方式。进一步地，长期状态价值模型刻画当前执行位置对后续 KV 分布、状态恢复和跨轮服务成本的影响，为从单请求穷举扩展到 session 级优化提供统一建模基础。

KV manager 将 KV 获取动作落实为 block directory 查询、连续 prefix 检查和缺失 blocks 恢复，并在请求完成后依据入口转移概率、session continuation predictor 输出和后台资源余量动态调整增量复制比例。初步模拟结果表明，在 $1.5\times$ 并发下，单请求穷举与完整 KV manager 相对同并发就近执行，将 CodeLlama34B 和 Qwen2-VL 的平均 E2E latency 分别降低 44.89% 和 26.02%。若按照项目承载力口径，与基础并发下的就近执行比较，两类模型的平均 E2E latency 分别降低 19.65% 和 23.37%。上述结果反映单请求穷举与完整 KV manager 的组合效果；路由、block-level residual sync 和后台增量预放置的独立贡献仍需通过消融实验区分。

中心状态通信估算表明，在当前三节点高频状态更新配置下， $1.5\times$ 并发产生约 3.01 Mbps 节点间状态上报流量，最高单链路约为 1.60 Mbps。中心化 Router 读取中心状态管理模块维护的全局状态，并结合入口节点提交的请求级路由特征进行决策；三节点原型 Dueling Double DQN 的前向计算 P99 为 0.016 ms。

当前模拟建立了单请求穷举与完整 KV manager 组合下的性能基线；长期状态价值的收益将在相同工作负载和 KV manager 配置下，通过单请求穷举与长期路由器的独立对照实验进一步验证。后续还需研究请求执行期间 handover、多 session 的全局 placement 资源分配和 prefill/decode 分离，由此逐步将 CEC 中的 LLM inference offloading 从单次执行位置选择扩展为面向连续 session 和版本化运行时状态的长期协同优化。

七、三类算法的协同优化

前文分别从模型切分部署、SLO 感知 batching 和 LLM inference task offloading 三个角度讨论了 CEC-LLM 推理系统的关键算法。真实系统中，这三类算法并不是彼此独立的模块：模型切分部署决定模型组件、服务实例和推理阶段的空间分布；业务感知 batching 决定请求、prefill chunk 和 decode token 在实例内部或不同阶段资源池中的时间组织；task offloading 决定完整请求、复杂 workflow 以及 context/KV/runtime state 在协作边缘节点之间的路由、迁移和复用。三者共享模型位置、队列长度、KV cache 位置、链路状态、用户移动趋势和 SLO slack 等同一组系统状态，因此任何一类算法的局部优化都可能改变另外两类算法的可行域和收益。

本章不再重新划分第四、五、六章的所有单点方法，而是围绕“三类算法之间怎样协同”组织已有工作。具体而言，协同优化关注的不是某个模型如何单独切分、某个 batch size 如何单独选择，或某个请求如何单独卸载，而是这些决策在不同时间尺度、多个协作节点和多种运行时状态下如何形成闭环。已有工作已经从局部角度揭示了这种耦合关系：model caching 会影响 request offloading 的可选路径，prefill/decode disaggregation 会改变 batching 和 KV handoff，KVCache-centric serving 会把状态位置纳入 request scheduling，移动场景下的 context migration 又会反过来影响长期 session routing。

本文将 CEC-LLM 推理中的三类算法协同关系划分为四类：

1. **Model / Service Placement – Request Offloading 协同**：低频模型缓存、服务部署和高频请求路由之间的双时间尺度闭环。
2. **Model Partitioning – Batching / Scheduling 协同**：模型切分、阶段解耦、continuous batching 和 serving scheduling 之间的容量匹配与时延控制。
3. **Request / Workflow Routing – Runtime State Management 协同**：请求路径、agent workflow、KV cache、prefix cache、session state 和 context migration 之间的状态感知路由。
4. **Mobility-aware Hierarchical Control**：用户移动、负载热点变化、handover 和状态位置变化共同驱动的跨层闭环控制。

这四类关系不是新的孤立算法类别，而是前文三类算法在系统运行时的交叉界面。第一类主要连接第四章的模型/服务部署和第六章的请求卸载；第二类连接第四章的模型切分和第五章的 batching；第三类连接第五章的状态感知调度和第六章的请求/workflow offloading；第四类则把移动 CEC 中的外部扰动纳入统一控制框架。表 29 汇总了与这些协同关系直接相关的代表工作、主归属和方法关键词。

表 29 CEC-LLM 三类算法协同优化相关工作总览

序号	代表工作	发表在哪	主归属	相关机制	方法关键词
----	------	------	-----	------	-------

1	SLM-LLM-Collab [67]	TMC 2026	Model / Service Placement – Request Offloading	SLM caching; D2D / edge inference offloading	长期缓存 SLM 或插件模型，短期在 local、D2D 和 edge LLM 之间选择卸载路径，体现模型缓存与请求卸载的双时间尺度协同
2	T2DRL [76]	TON 2026	Model / Service Placement – Request Offloading; Mobility-aware Control	long-context model caching; request management	在 mobile edge 中联合管理 cached models、service requests 和资源匹配，使 long-context LLM agent 服务同时感知模型缓存、上下文长度和边缘资源
3	H-MEC-Placement [77]	TMC 2025	Model / Service Placement – Request Offloading	service placement; task scheduling; resource allocation	面向 general MEC 说明服务部署、任务调度和资源分配在规划周期内存在强耦合，可作为 LLM 服务布局协同的基础参考
4	LVM-Offload [78]	INFOCOM 2026	Model / Service Placement – Request Offloading	foundation model selection; step offloading; resource allocation	在 foundation model 服务中联合考虑模型选择、推理步数、任务卸载和资源分配，说明模型能力选择会改变后续卸载路径
5	DistServe [4]	OSDI 2024	Model Partitioning – Batching / Scheduling	prefill/decode disaggregation; phase-specific workers	将 prefill 和 decode 分离到不同 worker pool，并分别优化并行策略、batching 和资源分配，收益取决于 KV handoff 与阶段队列协调
6	Splitwise [29]	ISCA 2024	Model Partitioning – Batching / Scheduling	prompt pool; token pool; KV transfer	把 prompt computation 和 token generation 放到不同机器池，利用阶段资源画像差异，同时需要处理阶段间 KV transfer 和调度边界
7	Orca [1]	OSDI 2022	Model Partitioning – Batching / Scheduling	iteration-level scheduling; selective batching	在 decode iteration 边界更新 active batch，说明 serving 调度粒度会直接影响模型实例利用率和请求等待时间
8	vLLM [2]	SOSP 2023	Model Partitioning – Batching / Scheduling; Runtime State Management	PagedAttention; KV block management	通过 KV block 管理支撑 continuous batching，使 batching 容量不只取决于计算资源，也取决于 KV 显存碎片和状态管理能力
9	Llumnix [52]	OSDI 2024	Model Partitioning – Batching / Scheduling; Runtime State Management	live request migration; instance rescheduling	在多个 LLM serving 实例之间迁移运行中请求及其状态，体现实例调度、batch 负载均衡和状态迁移之间的耦合
10	Mooncake [13]	USENIX FAST 2025	Request / Workflow Routing – Runtime State Management	distributed KV cache; global scheduling; PD disaggregation	将 KV cache 作为全局状态层管理，调度器按 KV 本地性、节点负载和 SLO 选择 prefill/decode 路径
11	Preble [49]	ICLR 2025	Request / Workflow Routing – Runtime State Management	distributed prompt scheduling; prefix/KV locality	把 prefix/KV cache 命中和负载均衡共同纳入请求分配，使 routing action 与后续 prefill 复用和 batch 组成相关
12	CachedAttention [79]	USENIX ATC 2024	Request / Workflow Routing – Runtime State Management	attention / prefix cache reuse	说明 attention/prefix cache 的复用机会会改变请求服务路径和重复计算成本，是状态感知调度的重要依据
13	FuseSpill [73]	TPDS 2026	Request / Workflow Routing – Runtime State Management	KV spillover; auxiliary GPU scheduling	显存受限时根据代价模型选择 KV cache spillover，并把溢出序列调度到辅助 GPU，改变原有 decode 执行位置
14	QoS-LLM-Router [62]	TMC 2025	Request / Workflow Routing – Runtime State Management	QoS-aware routing; queue / memory state	将 edge LLM expert routing 建模为长期 QoS 优化，路由决策同时感知质量、响应长度、队列状态和 GPU memory
15	CES-CB [63]	TSC 2025	Request / Workflow Routing – Runtime State Management	request cost prediction; edge batch selection	在其原始 edge/cloud 环境中利用 prompt embedding 和历史执行数据预测请求成本，说明 routing 会影响队列阻塞和 batch 组成

16	MasRouter [69]	ACL 2025	Request / Workflow Routing – Runtime State Management	multi-agent routing; model / role selection	根据 query 决定 multi-agent 协作模式、agent 数量、角色和底层 LLM；在 CEC 中需要进一步映射为 workflow offloading
17	Conductor [70]	ICLR 2026	Request / Workflow Routing – Runtime State Management	natural-language workflow control	训练控制器生成多模型 workflow、subtask 和上下文访问范围，提示 agent workflow 需要同时考虑模型选择和状态可见性
18	AFLOW [71]	ICLR 2025	Request / Workflow Routing – Runtime State Management	automatic agentic workflow search	将 agentic workflow 表示为代码形式的 LLM 调用图，并用执行反馈搜索更优 workflow；在 CEC 中还需加入物理节点和状态路径
19	MGCO [68]	TSC 2026	Mobility-aware Hierarchical Control	mobility-aware generative computation offloading	用 Transformer MHA encoder 建模历史决策和未来移动状态，说明请求卸载不能只根据当前时隙状态短视决策
20	DT-MADTO [72]	TMC 2025	Mobility-aware Hierarchical Control	digital twin; DAG offloading; mobility prediction	通过 digital twin 预测移动和资源状态，联合决策 DAG 子任务卸载、带宽分配和中间数据等待时间
21	CA-AIGC-Migration [75]	TSC 2026	Mobility-aware Hierarchical Control; Runtime State Management	context migration; mobility-aware service migration	将完整模型迁移转化为 long context migration，在回复质量、AoI、计算延迟和传输成本之间做权衡

7.1 三类算法的基本耦合关系

模型切分、batching 和 task offloading 共享同一组系统状态，包括模型副本位置、模型切分模板、prefill/decode worker 资源、batching window、实例队列长度、KV cache 位置、prefix cache 命中情况、session state、链路质量和用户移动趋势。模型切分如果改变 prefill 或 decode 的执行位置，就会改变 batching 的阶段容量和 KV handoff 成本；batching 如果改变请求等待时间和 token 生成节奏，就会影响 request routing 观察到的实例时延和 admission 边界；request/session routing 如果改变流量入口和服务实例选择，又会反过来改变模型实例负载、prefix cache 命中率和后续 context/KV 迁移成本。

因此，三类算法之间不是简单的前后级联关系，而是多时间尺度上的闭环关系。低频部署层决定模型切分、模型副本、SLM/LLM 插件和服务能力布局；中频 serving 层根据 workload 调节 batching、admission、prefill/decode 比例和实例负载均衡；高频 offloading 层根据请求复杂度、用户位置、链路状态和状态位置执行 request routing、workflow placement 和 state migration。协同优化的关键不是把每一层都独立做到局部最优，而是让它们在统一 SLO、资源约束和状态约束下避免互相抵消。

从控制变量看，三类算法的输入和输出也存在明显依赖。模型切分层输出可部署模板、阶段容量、KV handoff cost 和 fallback path；batching 层输出队列等待、active batch occupancy、TTFT/TPOT、KV 水位和阶段 backlog；offloading 层输出请求入口、计算节点、状态处理动作和后续 session 的 KV owner。这些输出又会成为其他层下一轮决策的输入。换言之，协同优化不应只追求一个全局静态最优解，而应维护一个持续更新的系统状态视图。

7.2 Model / Service Placement – Request Offloading 协同

模型/服务部署与请求卸载之间的耦合，是 CEC-LLM 推理系统中最直接的协同优化关系。由于边缘节点无法缓存所有 LLM、SLM、expert、adapter 或工具服务，模型能力是否提前部署会直接决定请求能否在入口附近处理，以及是否需要转发到其他协作节点或调用可选外部服务。反过来，请求流量、用户会话分布和模型调用频率又会影响下一周期的模型缓存和服务部署收益。

已有工作已经开始显式处理部署规划与请求卸载之间的时间尺度差异。SLM-LLM-Collab [67] 将 SLM caching 与 local/D2D/edge inference offloading 进行双时间尺度建模，说明模型缓存不是独立部署问题，而是为后续请求卸载提供可行路径。T2DRL [76] 进一步将 long-context LLM serving 中的 model caching、request management 和资源匹配联系起来，强调上下文长度和移动边缘资源会共同影响模型缓存与请求处理策略。H-MEC-Placement [77] 虽然不是 LLM-specific 工作，但从 general MEC 角度说明 service placement、task scheduling 和 resource allocation 在规划周期内存在强耦合。LVM-Offload [78] 则在 foundation model 服务中联合考虑模型选择、推理步数、任务卸载和资源分配。

这一类协同的核心变量包括 model/service placement、SLM/LLM cache、adapter/expert residency、service capacity、request arrival pattern、routing action、cloud fallback 和 cache update period。低频部署层如果只根据历史平均流量放置模型，可能无法应对移动热点和突发请求；高频请求卸载如果只根据当前节点负载选择路径，又可能频繁调用未缓存模型，造成冷启动、跨域传输和质量回退。因此，部署层和卸载层需要形成“长期能力布局 + 短期路径选择 + 在线反馈更新”的闭环。

在 CEC 中，模型部署层应当被视为协同优化框架中的低频规划层，而不是一次性静态配置。其输出不应只是某个模型放在哪个节点，而应包括模型能力布局、服务容量边界、可服务请求类型、切换条件和回退路径。请求卸载层则在给定部署状态下进行高频决策，并把流量、命中率、队列和质量反馈回部署层，用于下一周期更新模型缓存和服务布局。

7.3 Model Partitioning – Batching / Scheduling 协同

模型切分与 batching/scheduling 的耦合主要体现在推理阶段的容量匹配和时延控制上。LLM inference 中 prefill、decode、verification 等阶段具有不同资源画像：prefill 更偏计算密集，decode 更偏 memory bandwidth 和低时延 token streaming。模型切分或阶段解耦改变这些阶段所在的资源池，而 batching 和 scheduling 决定请求如何进入这些资源池以及 token 如何被迭代生成。

已有 LLM serving 工作从不同角度揭示了这种耦合。Splitwise 和 DistServe 通过 prefill/decode disaggregation 说明，将两类阶段部署到不同资源池可以提高 goodput，但也会引入阶段间协调和 KV handoff 成本 [29], [4]。Orca [1] 和 vLLM [2] 等系统说明，iteration-level scheduling 和 continuous batching 可以显著提高 GPU 利用率，但其效果受到模型部署方式、decode worker 容量和 KV cache 管理策略影

响。Llumnix 进一步展示了动态请求迁移、实例调度和 serving 稳定性之间的关系 [52]。Mooncake 从 KVCache-centric serving 出发，将 KV cache 管理、prefill/decode 分离和请求调度结合起来，说明状态位置会影响 batching 和阶段执行路径 [13]。Jupiter 将 generative LLM 在 edge devices 上的 collaborative inference 与 pipeline parallelism、speculative decoding 结合起来，进一步说明在 CEC 场景中，阶段拆分和 serving scheduling 必须一起考虑 [21]。

这一类协同的关键变量包括 partition template、stage boundary、prefill/decode pool capacity、batching window、active sequence set、chunk size、KV transfer time、TTFT/TPOT target 和 worker scaling。模型切分模板不能只描述模型层或阶段放在哪里，还需要暴露阶段容量、KV handoff 成本、batching 可容纳规模和 TTFT/TBT 估计。Batching 层也不能只根据当前队列最大化吞吐，而需要感知切分拓扑、状态位置 and 用户 SLA。

在 CEC 系统中，模型切分与 batching 的协同尤其需要避免局部优化互相抵消。例如，阶段切分把 prefill 放到算力更强的协作节点可以提升 prefill 吞吐，但如果 KV 传向 decode 节点的链路不稳定，decode TTFT 和后续 token latency 可能恶化；batching 策略为了追求高吞吐扩大等待窗口，可能让移动用户在 handover 前无法及时完成关键阶段；continuous batching 扩大 active set 能提升利用率，但也会提高 KV 显存压力，使请求迁移和状态复制更困难。因此，协同控制器需要同时观察阶段负载、batch 状态、KV 水位和链路状态。

7.4 Request / Workflow Routing – Runtime State Management 协同

请求路由和 runtime state 管理之间的耦合，是 LLM inference 相比传统 task offloading 最突出的新问题之一。传统 offloading 中，任务完成后状态通常不再影响后续请求；而在 LLM 多轮对话、长上下文和 agent workflow 中，context、KV cache、prefix cache、agent memory 和 tool outputs 会持续影响后续推理。此时，请求应该路由到哪里，不仅取决于哪个节点算力空闲，还取决于相关状态是否已经在该节点或邻近节点存在。

Mooncake、Preble 和 CachedAttention 等 cache-centric serving 工作说明，prefix/KV cache 已经从单纯的内存优化对象变成影响请求调度和服务路径选择的核心资源 [13], [49], [79]。FuseSpill [73] 从显存受限场景出发说明，KV cache spillover 可能导致 decode 请求迁移到辅助 GPU 或其他资源位置，从而改变原有的执行路径。CA-AIGC-Migration [75] 则直接面向移动边缘场景，将用户 long context 视为可迁移状态，而不是迁移完整模型，从而在回复质量和迁移成本之间做权衡。QoS-LLM-Router [62] 和 CES-CB [63] 虽然主要关注 request-level routing，但它们也说明了 routing action 会改变队列状态、batch 组成和后续请求的等待时延。

这一类协同的关键变量包括 session ID、prefix hash、KV block location、state size、state freshness、remote access latency、migration cost、recompute cost、queue state 和 request quality target。对于连续

session, 路由动作和状态动作应被联合枚举: 在已有 KV 的节点本地执行、将 KV 迁移到新节点执行、远程访问旧节点状态, 或在新节点重新 prefill / recompute。不同动作的当前成本和长期收益不同, 不能只用单次请求的最小时延判断。

对于 multi-agent workflow, 这种耦合更加复杂。MasRouter、Conductor 和 AFLOW 等工作主要从算法层研究如何选择 agent 数量、角色、模型和工作流结构, 但在 CEC 系统中, 每个 agent 背后的模型、工具、数据库和 memory 可能位于不同物理节点。因此, workflow orchestration 必须进一步转化为 workflow offloading: 不仅决定 planner、solver、critic、verifier 等角色如何协作, 还要决定这些组件在哪里执行、哪些 context 可以共享、哪些中间状态需要迁移, 以及何时调用能力更强的协作节点或外部模型服务进行 verification。

7.5 Mobility-aware Hierarchical Control

多用户移动性会同时影响模型部署、batching、请求路由和状态管理, 因此它是三类算法协同优化中最重要的外部扰动之一。用户移动会改变区域请求热点, 使原有 model placement 失效; 会改变接入链路和候选服务节点, 使 request routing 的当前最优路径失效; 会在请求或 workflow 执行过程中触发 handover, 使中间状态和后续阶段的执行路径需要重新选择; 也会使 long context、KV cache 和 agent memory 留在原服务节点, 从而产生状态迁移、远程访问或重算之间的权衡。

已有移动性相关工作提供了局部研究基础。MGCO [68] 说明, generative computation offloading 不能只根据当前状态短视决策, 而需要利用长程移动信息进行规划。DT-MADTO [72] 通过 digital twin 建模 DAG task、用户移动和中间数据流, 说明移动性会改变 workflow/subtask offloading 的传输代价和拥塞状态。CA-AIGC-Migration [75] 进一步将移动性与 long context migration 结合, 说明状态迁移可以成为替代完整服务迁移的轻量机制。T2DRL [76] 则提示, 在 mobile edge 场景中, long-context serving 的模型缓存、请求管理和资源匹配需要与移动场景共同考虑。

这一类协同的关键变量包括用户轨迹、切换概率、未来入口节点、服务热点、状态所在节点、迁移带宽、会话继续概率和负载预测。低频层根据用户移动和服务热点调整模型与服务布局; 中频层根据负载和状态命中率调整 batching 与实例分流; 高频层根据 handover、链路抖动和 context/KV 位置进行请求、工作流与状态路由。三个时间尺度不需要每次都同时重优化, 但必须共享预测和反馈。

现有研究还没有形成完整的 CEC-LLM 跨层闭环。很多工作只优化其中一层: 有的关注 request offloading, 有的关注 DAG subtask, 有的关注 context migration, 有的关注 model caching。未来更需要一个 mobility-aware hierarchical controller, 使模型能力、请求路径、workflow 结构和状态位置能够随用户移动和 workload 变化自适应调整。

7.6 小结

总体来看，三类算法的协同优化不是简单地把模型切分、batching 和 task offloading 合并成一个大优化问题，而是要在实际工作揭示的局部耦合关系基础上，建立分层、多时间尺度、状态感知的控制框架。本章将相关耦合关系组织为四类：Model / Service Placement – Request Offloading 协同、Model Partitioning – Batching / Scheduling 协同、Request / Workflow Routing – Runtime State Management 协同，以及 Mobility-aware Hierarchical Control。

已有工作已经分别展示了 model caching 与 inference offloading、prefill/decode disaggregation 与 scheduling、KV cache 与 request routing、context migration 与 mobility-aware serving 之间的耦合，但有些机制仍然缺少面向 CEC-LLM inference 的统一组织。CEC 场景的核心挑战在于，模型能力、batch 状态、请求路径和运行时状态会随着用户移动、负载变化和链路波动持续变化；局部最优策略如果缺少共享状态和长期反馈，容易造成阶段失衡、尾延迟放大、KV 状态黏附或频繁迁移。探索有效的协同优化方法是重要的研究方向。

八、面向 Ascend 验证环境的 CEC 推理工程实现方案

8.1 工程定位

面向验证环境的工程实现需要把前文三个算法落到同一套可执行闭环中：第四章的动态规划切分策略依赖 Ascend GPU/NPU 上可运行的 layer/block 级 pipeline stage，并在此基础上搜索 layer-wise stage boundary、stage placement 和混合并行动作；第五章的 RL batching 优化器以参数自适应、请求排序/准入为递进路线；第六章的请求调度则把就近路由扩展为 SLA、显存、链路和 KV 状态共同约束的长期成本决策。本章的任务是说明这些规划与策略学习算法如何映射到 vLLM-Ascend 和异构 Ascend 边缘验证环境中的可执行接口。

vLLM-Ascend 是 vLLM 面向 Ascend NPU 的硬件插件，依赖 Linux、Ascend NPU、CANN、torch-npu、torch 和 NNAL 等基础环境 [80], [81]。其功能文档已经列出 dynamic batch、fine-grained tensor parallelism、layer sharding、context parallel 和 dynamic chunked pipeline parallel 等能力 [82]；KV Cache Pool 和 speculative decoding 也分别有独立说明 [83], [84]。这些能力可以作为工程底座，但不能直接等同于 CEC 场景下的资源自适应推理系统：它们需要被暴露为策略可调用的安全动作，并由外层控制面根据节点资源、链路状态、请求负载和业务 SLO 做受控决策。

本章建议采用“执行底座 + 规划/学习器 + 安全过滤器 + 分层控制”的实现路线。执行底座先保证 Ascend 侧能够稳定运行 layer/block 级 pipeline stage；模型切分规划器把 layer-wise split、PP/TP/EP/DP、context parallel、KV Cache Pool、PD 分离和 speculative decoding 等能力作为可部署候选；batching 学习器把 max running requests、max batched tokens、chunked prefill、等待窗口和准入阈值作为动作空间；安全过滤器根据显存、链路、SLA 和框架能力裁剪不可执行动作；分层控制器再根据统一元数据在低频、中频和高频三个周期上执行模型切分、batching 参数控制和请求/状态路由。工程实现的核心是在现有 runtime 能力之上补齐 CEC 场景需要的资源画像、动态规划、策略学习、状态管理和在线安全闭环，使算法输出能够被部署、压测、回退和解释。

8.2 总体架构

图 22 给出了推荐的工程架构。与旧式“集中入口加调度器”的架构不同，这里把系统拆成四个互相反馈的平面：执行面负责 vLLM-Ascend / MindIE 等 CANN 兼容实例运行；画像面负责模型、硬件、拓扑和 workload profiling，为模型切分动态规划生成 stage/link 代价表，并为 RL batching 提供训练和评估数据；状态面负责队列、KV、链路、session 和故障元数据；控制面负责低频切分策略、中频 batching 策略和高频请求调度策略。

从请求路径看，业务请求从多个流量入口分别进入系统，携带 session ID、业务优先级、SLO、

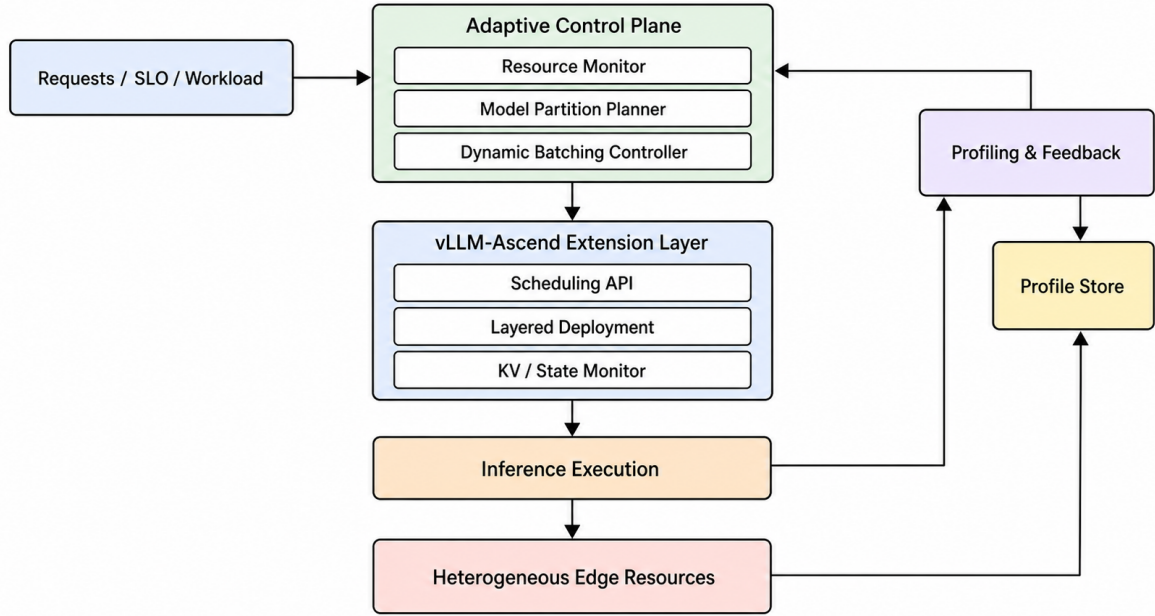


图 22 面向 Ascend/CANN 验证环境的 CEC 推理工程框架

隐私标签、输入长度估计和上下文标识。各入口节点采用分布式接入方式，中心 router 查询共享状态面并执行任务路由决策，同时接受控制面的全局约束和策略参数。中心 router 先判断已有 KV 或 prefix cache 位于哪个协作节点，再把节点显存、队列长度、链路状态和状态位置输入请求调度策略。若请求进入本地或邻近实例，batching 策略决定其是否立即接纳、等待合批、进入长请求隔离队列或触发 chunked prefill；若请求需要跨节点执行，请求调度策略同时决定 KV 是本地读取、迁移、复制还是重算。

从反馈路径看，推理实例持续上报 TTFT、TPOT、ITL、E2E latency、active sequence 数、waiting queue 长度、KV block 占用、显存峰值、NPU 利用率、stage transfer time、链路 RTT/带宽和错误码。画像面将离线 profiling 与在线观测合并，形成 stage/link 代价、状态特征、动作效果、奖励和安全约束数据；状态面维护 KV owner、prefix block hash、session 入口变化和实例健康状态；控制面据此更新动态规划代价表、RL 策略、动作裁剪边界和请求路由代价。这个闭环需要为每次动作保留可追溯原因，例如选择某个切点和 placement、缩小 batch、迁移 KV 或重算 KV 时对应的状态、约束和收益估计。

8.3 分阶段实现路线

第一阶段是 Ascend 层级切分执行能力。优先目标是让 CodeLlama34B、Qwen2-VL-7B-Instruct 等目标模型能够按连续 layer/block 切成多个 pipeline stage，并映射到 230T/64GB、70T/24GB、310P、910B4 或多机多卡 Ascend 节点上执行 [43]。工程上需要解决四个接口：分段权重加载、stage 间 hidden state

传输、pipeline stage 启动参数和异常回退。接口稳定后，第四章的拓扑感知 layer-wise split 才能从规划结果转化为真实可部署方案，并进一步支撑 TP、EP、DP、PD 分离和 KV pool 等高阶动作。

第二阶段是 EdgeShard 风格的动态规划切分规划器。系统先对单实例、平均 layer split、固定 PP、固定 TP、PP+TP、layer sharding、context parallel、KV Cache Pool、PD 分离和 speculative decoding 等运行形态做离线 profiling，记录每层 latency、activation 大小、KV 增长、stage transfer cost、显存峰值和可支持并发，并用这些数据校准 stage 代价表、链路代价表和安全过滤器。切分规划的顺序应与第四章保持一致：先用动态规划搜索 layer-wise stage boundary、stage placement 和 pipeline balance；再判断单个 stage 是否需要叠加 TP，MoE 或组件化模型是否需要 EP，流量承载是否需要 DP；最后才把 PD 分离、KV Cache Pool 和 semantic/token-level collaboration 作为高阶候选。

第三阶段是分阶段增强的 RL 在线 batching 优化器。实现上先只训练参数自适应策略，保持框架默认请求顺序或 FIFO 不变，让策略根据并发、prompt 长度分布、KV 水位和 TTFT/TPOT/ITL 反馈调节 max running requests、max batched tokens、等待窗口、chunk size 和 prefill/decode quota。这样可以先确认收益是否来自 batching 参数本身。第二步再加入请求排序和准入控制，用 SLO slack、预测 prefill/decode 时间、KV 增长和对其他请求 ITL 的边际影响学习排序权重，使出队顺序同时反映业务紧迫度和系统边际成本。第三步再启用联合策略：短 prompt 和紧 SLO 时策略收紧 active set 与等待窗口，高并发且 slack 充足时扩大 token budget，长 prompt 或 VLM 比例升高时启用 chunked prefill 或长请求隔离，具备稳定 KV handoff 时才允许策略选择 PD 分离动作。

第四阶段是长期成本感知请求调度和 KV 管理。三个节点都应维护统一的节点负载、显存、网络 and KV 元数据；请求到达后，系统枚举目标计算节点以及 local、migrate、recompute 等状态处理动作，并先用 SLA 和 memory 约束过滤不可行动作。正常路由不只选择当前请求时延最低的节点，还要估计动作执行后 KV owner、节点负载、未来入口位置和后续 session 成本的变化。KV 管理应采用 block-level 索引和增量同步：迁移前先查询目标节点已有 prefix/KV blocks，只传输缺失部分；旧节点短期保留状态副本，以支持迁移失败、链路退化或负载突变时回退。

第五阶段是统一元数据驱动的协同控制。低频周期更新模型切分策略和服务能力布局，可按分钟级或配置变更事件触发；中频周期调节 batching 参数、admission threshold、prefill/decode quota 和实例分流，可按数秒到十秒级窗口触发；高频周期执行 request routing、workflow placement、KV migrate/recompute/local 动作和异常回退，可按单请求或 session 边界触发。三类决策必须共享同一状态视图：模型切分层输出 stage 容量和 KV handoff cost，batching 层输出队列等待和 KV 水位，请求调度层输出请求入口和状态位置。控制器重点处理三类冲突：切分带来的状态传输成本超过并行收益，batching 扩大等待窗口导致高优先级请求违约，路由因 KV 黏附远端节点造成长期跨节点通信。

8.4 关键工程模块

第一个模块是 vLLM-Ascend 接口适配层。它需要梳理模型加载、并行配置、scheduler、KV Cache 管理、metrics 暴露和错误码路径，并把底层能力转化为控制面可调用的动作接口。例如，模型切分策略输出的动作需要能够落到 stage 数、每个 stage 的 layer 范围、节点映射、tensor parallel degree、context parallel 开关、KV pool 策略和回退动作；batching 策略输出的动作需要能够落到 max running requests、max batched tokens、等待窗口、chunk size、prefill/decode quota 和 admission threshold；请求调度策略输出的动作需要能够落到目标实例、KV local/migrate/recompute、状态副本保留时间和失败回退路径。MindIE 可作为 Ascend 生态中生产化推理、调试、调优和迁移接口的参考底座 [85]；vLLM 的 OpenAI-compatible server 可作为服务入口和接口兼容性的参考 [86]。

第二个模块是 Profile Store、Cost Table / Replay Buffer 与 Safety Guard。Profile Store 保存模型-硬件-拓扑的观测数据，包括 layer latency、activation、KV 增长、stage transfer cost、prefill/decode 吞吐、OOM 边界和链路状态；Cost Table 保存动态规划需要的 stage 代价、link 代价、显存约束和切换成本；Replay Buffer 保存 batching 与请求调度的状态、动作、奖励、违约和回退记录，用于离线训练、在线微调 and 策略回放评估；Safety Guard 保存显存上限、链路带宽下限、P99 TTFT 约束、动作冷却时间和回退条件。三者需要区分离线压测值和在线校正值：离线值用于生成初始代价表和安全边界，在线值用于修正当前窗口的动作裁剪、代价估计和奖励估计。这样可以避免系统在负载波动时盲目切换，也能避免长期使用过期画像。

第三个模块是在线策略决策器。模型切分策略只在低频周期或明显资源变化时输出动作，并设置冷却时间和最小收益阈值；batching 策略在中频周期更新参数、排序权重和准入阈值，但不直接替代 runtime 内部 token 调度；请求调度策略在高频周期处理每个 session/request 的目标节点和 KV 动作。三个决策器共享同一组约束：显存不溢出、SLA 不违约、链路带宽可承载、动作可启动、状态迁移成本可控。若安全过滤后可行动作为空，系统必须显式进入回退路径，例如降低并发、缩小 batch、回退固定 PP/TP，或转发到其他可用的完整模型实例。

第四个模块是监控、审计和回退。验证环境中的算法收益必须能被复现，因此每次策略动作、batch 参数调整、请求迁移和 KV 重算都要记录触发状态、策略输出、安全过滤结果、奖励估计、最终动作和实际结果。监控指标至少包括 P90/P99 TTFT、TPOT、ITL、平均 E2E latency、SLA violation、满足 SLA 的 rps、最高并发用户数、显存峰值、OOM 次数、NPU 利用率、KV 命中率、状态迁移量和回退次数。对外报告时，应能区分收益来自层级切分策略、batching 参数自适应、请求排序策略、KV 迁移还是三者协同。

8.5 验证路线

验证应按能力递进组织，先验证执行底座，再验证单类策略，最后验证协同闭环。第一组实验验证 Ascend 层级切分执行能力：比较单实例、平均 layer split、固定 PP/TP 和拓扑感知 layer-wise split，观察 OOM、P99 TTFT、TPOT、stage bubble、stage transfer time 和满足 SLA 的并发用户数。该组实验回答的是“Ascend 推理栈是否已经具备可执行的层级切分底座”。

第二组实验验证动态规划模型切分策略：在完成 profiling、stage/link 代价表和安全边界校准后，比较平均切分、固定 PP/TP、负载感知 placement、启发式拓扑感知 layer-wise split、EdgeShard 风格动态规划切分、动态规划加安全过滤和收益阈值。重点指标是满足 SLA 的请求处理数、最高并发用户数、显存峰值、通信开销、不可执行方案比例、pipeline bubble 和回退次数。这里的目标与第四章一致：相对于多节点平均切分基线，验证 goodput 和 SLA-constrained concurrency 是否有稳定提升 [43]。

第三组实验验证 RL 在线 batching 优化器：先比较固定参数、离线调优参数和 actor-critic 参数自适应；再比较 FIFO、EDF/slack baseline、priority queue、边际代价排序和 RL 学习到的排序权重；最后在短 prompt、长 prompt、VLM 和多优先级混合 workload 上测试联合策略。指标应覆盖满足 SLA 的请求处理速度、平均 E2E latency、P99 TTFT、ITL 抖动、高优先级请求违约率、普通请求饥饿比例、策略切换次数和回退次数。

第四组实验验证长期成本感知请求调度和 KV 管理：比较就近执行、当前最小时延 Greedy、长期成本路由、长期成本路由加 block-level KV 管理。除 TTFT、E2E latency、吞吐和并发数外，还要统计用户移动后的累计通信时延、请求转发次数、KV 迁移/重算次数、状态传输量、prefix/KV 命中率和 session 节点切换次数。该组实验用于说明为什么 CEC 中的请求调度不能只看当前节点负载。

第五组实验验证协同控制：分别启用第四章切分策略、第五章 batching 策略和第六章请求调度但不共享状态，作为独立算法组合基线；再启用统一元数据但不做长期预测；最后启用完整的低频部署策略、中频 batching 策略、高频请求/状态路由闭环。扰动项包括链路带宽、RTT、可用显存、长 prompt 比例、移动用户比例和请求优先级分布。最终报告应同时给出性能收益和稳定性成本，尤其是策略切换频率、策略震荡、回退次数和状态传输开销。

8.6 项目进度与交付安排

8.3 给出的分阶段实现路线用于说明各项工程能力之间的技术依赖关系，本小节进一步给出项目的日历计划和交付节奏。项目周期为 12 个月，自 2026 年 6 月 1 日起至 2027 年 5 月 31 日止。按照启动当月为第 1 个项目月计算，第 1–2 个月开展技术调研，第 3–8 个月完成前两项算法，第 9–12 个月完成用户移动场景请求调度算法，三个阶段分别占 2、6 和 4 个月。

每个阶段在计划结束月份的 15 日前形成可供评审和联调的交付版本，包括设计文档、算法源码、

必要的推理框架修改以及测试结果；当月 16 日至月末由项目团队与华为团队共同检查交付内容，讨论环境适配、功能完整性、性能结果和遗留问题，并根据反馈完成最后一轮修改。阶段最终版本在当月月底提交。具体安排见表 30。

表 30 项目实施与交付时间安排

项目阶段	工作时间	工作内容	交付内容	联合评审与最终迭代
第一阶段	2026 年 6 月 1 日–7 月 31 日	调研资源自适应模型切分、业务感知 batching 和用户移动场景请求调度的学术研究与工程方案。	完成三类算法的主流方案对比、面向 Ascend 验证环境的技术路线分析，以及技术调研和研究综述。	7 月 15 日前形成交付版本；7 月 16–31 日与华为团队评审并完成修改，7 月 31 日提交最终版本。
第二阶段	2026 年 8 月 1 日–2027 年 1 月 31 日	完成资源自适应模型切分算法和业务感知 batching 优化算法的设计、实现、适配与测试。	两项算法分别交付详细设计文档、完整可运行源码、必要的推理框架修改源码，以及测试结果与分析报告。	2027 年 1 月 15 日前形成两项算法的交付版本；1 月 16–31 日联合评审并完成修改，1 月 31 日提交最终版本。
第三阶段	2027 年 2 月 1 日–5 月 31 日	完成用户移动场景下请求调度与 KV 管理算法的设计、实现、适配与测试。	交付算法详细设计文档、完整可运行源码、必要的推理框架修改源码，以及测试结果与分析报告。	5 月 15 日前形成交付版本；5 月 16–31 日联合评审并完成修改，5 月 31 日提交最终版本。

8.7 小结

工程结论是：面向 Ascend 验证环境的 CEC LLM 推理系统应先补齐层级切分执行底座，再把模型切分、batching 和请求调度分别实现为可观测、可回退、可解释的在线决策能力。第四章对应低频的动态规划混合切分部署；第五章对应中频的分阶段 RL batching 优化器；第六章对应高频的长期成本感知请求路由和 KV 管理；第七章则把三者收敛为统一元数据驱动的分层协同控制。

这样的工程路线与现有 vLLM-Ascend 能力并不冲突。vLLM-Ascend 继续负责模型加载、算子执行、KV 管理和实例内调度；外层系统负责画像、动态规划、策略学习、状态、路由、动作裁剪和回退。只有把“可执行动作接口”和“在线状态”放在同一个闭环中，前文提出的算法建议才会从综述中的方法对比，转化为验证环境中可以部署、压测和解释的工程方案。

项目交付按照“提前形成交付版本–联合评审与联调–完成最终迭代”的节奏推进，使每个阶段在正式截止前预留半个月处理验证环境适配、结果复核和交付问题，降低算法实现与最终验收之间的偏差。

九、挑战与未来方向

9.1 异构资源与性能建模

CEC LLM 推理中的性能建模比云端 serving 更难，因为影响时延的因素不再只是模型大小和 GPU 利用率。终端 NPU、分布在各协作节点中的 GPU/NPU、CPU 内存、显存容量、无线接入链路、边缘节点间链路以及各节点队列都可能进入同一条服务路径。即使模型和请求相同，只要入口位置、batch 组成或 KV 状态位置不同，TTFT 和 TPOT 都可能出现明显差异。HexGen、HexGen-2 和分布式协同智能综述均表明，异构资源建模是跨设备、跨节点推理优化的基础难点 [12], [87], [9]。

因此，未来更可行的方向不是建立一个全局精确模型，而是构建可在线校准的组合性能画像。模型画像记录不同 prompt/output 长度下的 prefill、decode 和 KV 增长特征；硬件画像记录不同 batch、并行度和量化配置下的吞吐与显存水位；链路画像记录终端接入链路、边缘节点间链路和可选外部服务链路的 RTT、带宽与抖动；状态画像记录 prefix cache、KV Cache 和 session state 的位置与复用价值。调度器再基于这些画像做近似决策，并用线上观测持续修正误差。这个方向的关键不是预测得多精细，而是预测误差不能把系统带入错误的路由或切分策略。

9.2 状态迁移与一致性

KV Cache、prefix cache 和 session state 让 LLM 推理天然带有强状态性。传统边缘 offloading 往往把任务看成可独立迁移的计算单元，但 LLM 会话不是这样：一次 prefill 生成的 KV 可能在后续数百个 decode step 中持续使用，多轮对话还会不断累积上下文。用户移动后，如果只把新请求路由到最近节点，却忽略旧状态的位置，就可能出现重复 prefill、远程访问延迟升高或上下文连续性破坏。PagedAttention、CacheGen、Mooncake、Preble 和 CachedAttention 分别从 KV 分页、KV 压缩/流式传输、KVCache-centric architecture、prompt scheduling 和多轮会话缓存复用角度说明了该问题的重要性 [2], [13], [49], [79], [88]。

未来需要研究的是状态生命周期管理，而不只是状态迁移。系统应能判断哪些状态值得保留，哪些状态可以压缩、降精度、迁移、复制或淘汰；还要判断在 handover 前是否预热候选边缘节点，在 handover 后是否临时远程访问旧状态，何时再把状态真正迁移过去。更进一步，状态一致性也不能只用数据库式强一致性来理解。LLM serving 更需要的是服务连续性一致性：用户看到的上下文、权限、个性化信息和生成流不能中断，但底层 KV 是否完全同步可以根据成本和剩余生成长度做权衡。

9.3 在线策略稳定性

动态 batching、动态路由和动态切分都可能带来策略震荡。一个典型例子是：调度器看到某个边缘实例负载低，于是把更多请求转过去；转发后该实例排队升高，下一轮调度又把请求切回原实例；与此同时 prefix cache 命中下降，KV 状态在多个实例之间来回移动，最终平均负载看似均衡，尾延迟却变差。类似问题在无线移动场景中会更明显，因为链路抖动和入口变化本身就是高频扰动。动态调度和迁移相关系统通常会显式讨论 SLO、尾延迟、迁移开销或公平性边界，这也是 CEC 推理调度需要继承的工程原则 [4], [52], [50]。

因此，在线策略需要从“每次都选当前最优”转向“在收益足够大时才切换”。可研究的机制包括切换冷却时间、收益阈值、置信区间、状态迁移预算、SLA 违约保护和灰度发布。对于生产系统，还应保留影子调度能力：新策略先在旁路计算推荐决策，与线上实际决策比较一段时间，再逐步接管小比例流量。这样的机制看起来不如端到端强化学习漂亮，但在无线软件环境中更容易解释、回滚和验收。

9.4 隐私与合规

无线场景中的用户位置、业务上下文、终端标识和会话历史都可能具有隐私敏感性。CEC 推理如果只按时延和负载选择路径，可能把敏感上下文转发到不合适的节点或区域。尤其是长上下文会话中，KV Cache 与 prefix cache 虽然不是原始文本，但仍可能包含与用户输入相关的隐含信息，因此状态放置和状态迁移也应纳入隐私约束。

未来更合理的做法是把隐私边界作为调度约束，而不是事后补丁。请求进入系统时就应带有数据等级、可出域范围、允许执行的节点集合、是否允许调用外部服务以及是否允许跨节点复制状态等标签。路由、batching 和模型切分都必须在这些约束下优化：隐私敏感请求可以优先在本地或指定边缘域内处理，复杂但敏感的请求可以先由本地模型提取允许共享的信息，再交给其他协作节点继续处理；状态迁移也可以只迁移必要的 session metadata，而不迁移完整 KV。这个方向的难点在于同时维持性能、质量和合规，而不是简单地把敏感请求全部留在本地。

9.5 未来研究方向

后续研究可以沿着五条线推进。第一是协作边缘一体化 LLM serving，把模型模板、batching 策略、状态索引和路由策略纳入统一控制面，而不是分别优化。第二是面向无线移动性的 KV/state 管理，重点研究状态何时保留、何时迁移、何时远程访问、何时重算。第三是多模型多租户调度，在 SLM、LLM、LoRA adapter、MoE expert 和可选外部大模型共存时，同时保证资源隔离、公平性和 SLA。第四是隐私保护协同推理，把数据本地性、状态脱敏、token-wise partition 和模型路径选择结合起来。

第五是面向生产系统的 MLOps 与策略治理，包括监控、灰度、回滚、异常检测和策略审计。其中，异构节点协同和移动服务连续性可由分布式协同智能综述与 mobility-aware service placement 工作支撑 [9], [89]; KV/cache runtime 可由 Mooncake、CacheGen、Preble 和 CachedAttention 等工作支撑 [13], [49], [79], [88]; 多用户公平调度可由 Fairness in Serving LLMs 支撑 [50]。

从项目落地角度看，最值得优先投入的是“预测-决策-回退”一体化框架。预测模块不追求完美，只要能够估计请求长度、实例服务时间、链路状态和移动风险；决策模块基于有限模板做 batching、routing 和部署选择；回退模块在预测失准、SLA 风险升高或状态迁移失败时快速切回保守策略。这样的框架既能承接前文三类算法，也能符合无线系统对稳定性和可解释性的要求。

十、总结

本文围绕 CEC 场景下的大语言模型推理优化进行了系统综述。与传统数据中心 LLM serving 相比，CEC 推理需要额外处理异构边缘资源、无线与边缘网络波动、节点负载变化、用户移动、KV/状态连续性、隐私边界和业务 QoS。现有研究已经分别在异构推理、LLM serving runtime、KV/cache 管理和移动边缘服务连续性方面提供了可借鉴基础，但这些机制需要被重新组织到一个可观测、可回退、可验证的协作式边缘推理框架中 [1], [2], [3], [4], [12], [9], [29], [13], [49], [50], [79], [88], [89]。

从算法主线看，资源自适应模型切分部署解决“模型如何跨异构节点展开”，业务感知 batching 解决“请求和 token 如何在实例内组织执行”，LLM inference task offloading 解决“请求、workflow 和运行时状态如何在协作网络中移动”。三者不是彼此独立的模块：模型切分会改变阶段容量和状态传输代价，batching 会改变实例队列和 SLO 风险，请求路由又会改变负载分布、prefix/KV 命中和后续状态位置。因此，CEC LLM 推理的系统价值来自跨时间尺度的协同闭环，而不是某一个单点算法。

从工程实现看，更稳妥的路线是在 vLLM-Ascend 等现有 Ascend 推理栈上扩展资源自适应控制能力。底层 runtime 继续负责模型加载、算子执行、token 生成、KV 管理和实例内调度；外层控制面补齐多节点资源感知、layer-level profiling、模型分层部署规划、动态 batching 参数控制、状态索引、监控反馈和异常回退。EdgeShard 提供了 layer-level profiling、设备选择、model shard placement 和动态规划搜索的基础，但在 Ascend 环境中需要转化为 vLLM-Ascend 可执行的模板化部署方案。

面向验证环境，报告建议将主问题落到三个可测闭环上：第一，以 layer-wise stage boundary 和 stage placement 为主变量，叠加 TP/EP/DP 等补充能力，验证拓扑感知混合切分是否优于平均层级切分和固定并行模板；第二，以 SLO slack、请求优先级、队列状态和实例画像为输入，验证动态 batching 控制是否优于固定 batch size 与 FIFO；第三，在用户人口变化和 KV 状态分离场景下，验证长期成本感知 routing 是否能够减少重复 prefill、远程状态访问和跨节点通信开销。只有把算法收益落实到 TTFT、TPOT/ITL、P95/P99 E2E、SLA 达标率、goodput、显存水位、KV/state 迁移成本和回退次数等指标上，才能判断其是否具备工程部署价值。

总体而言，CEC LLM inference 的核心并不是把云端 serving 简单搬到边缘，而是在异构资源和动态网络中重构模型部署、请求调度和状态管理之间的关系。资源画像、有限候选模板、在线代价选择、动态 batching 控制、状态感知 routing 和可解释回退机制，是把综述中的研究方法转化为可运行系统的关键路径。

参考文献

- [1] Y. Sheng et al., "FlexGen: High-throughput generative inference of large language models with a single GPU," in Proc. Int. Conf. Machine Learning (ICML), 2023.
- [2] A. Borzunov et al., "Petals: Collaborative inference and fine-tuning of large models," in Proc. 61st Annu. Meeting Assoc. Comput. Linguistics (ACL), vol. 3, 2023, pp. 558-568.
- [3] Y. Jiang et al., "HexGen: Generative inference of large language model over heterogeneous environment," in Proc. Int. Conf. Machine Learning (ICML), 2024, pp. 21946-21961.
- [4] G.-I. Yu et al., "Orca: A distributed serving system for Transformer-based generative models," in Proc. USENIX Symp. Operating Systems Design and Implementation (OSDI), 2022, pp. 521-538.
- [5] W. Kwon et al., "Efficient memory management for large language model serving with PagedAttention," in Proc. ACM Symp. Operating Systems Principles (SOSP), 2023, pp. 611-626.
- [6] A. Agrawal et al., "Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve," in Proc. USENIX Symp. Operating Systems Design and Implementation (OSDI), 2024, pp. 117-134.
- [7] Y. Zhong et al., "DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving," in Proc. USENIX Symp. Operating Systems Design and Implementation (OSDI), 2024.
- [8] L. Zheng et al., "SGLang: Efficient execution of structured language model programs," in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), 2024.
- [9] NVIDIA, "TensorRT-LLM documentation," 2026. [Online]. Available: <https://nvidia.github.io/TensorRT-LLM/>
- [10] NVIDIA, "Triton Inference Server: Dynamic batcher," 2026. [Online]. Available: <https://docs.nvidia.com/deeplearning/triton-inference-server/>
- [11] Hugging Face, "Text Generation Inference documentation," 2026. [Online]. Available: <https://huggingface.co/docs/text-generation-inference/>
- [12] Y. Jiang, R. Yan, and B. Yuan, "HexGen-2: Disaggregated generative inference of LLMs in heterogeneous environment," in Proc. Int. Conf. Learn. Representations (ICLR), 2025.
- [13] S. Duan, D. Wang, J. Ren, F. Lyu, Y. Zhang, H. Wu, and X. Shen, "Distributed artificial intelligence empowered by end-edge-cloud computing: A survey," IEEE Communications Surveys & Tutorials, vol. 25, no. 1, pp. 591-624, 2023.
- [14] Y. Huang et al., "GPipe: Efficient training of giant neural networks using pipeline parallelism," in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), 2019, pp. 103-112.

- [15] D. Narayanan et al., "Efficient large-scale language model training on GPU clusters using Megatron-LM," in Proc. Int. Conf. High Performance Computing, Networking, Storage and Analysis (SC), 2021.
- [16] L. Zheng et al., "Alpa: Automating inter- and intra-operator parallelism for distributed deep learning," in Proc. USENIX Symp. Operating Systems Design and Implementation (OSDI), 2022, pp. 559-578.
- [17] R. Y. Aminabadi et al., "DeepSpeed-Inference: Enabling efficient inference of transformer models at unprecedented scale," in Proc. Int. Conf. High Performance Computing, Networking, Storage and Analysis (SC), 2022.
- [18] P. Patel et al., "Splitwise: Efficient generative LLM inference using phase splitting," in Proc. ACM/IEEE Int. Symp. Computer Architecture (ISCA), 2024, pp. 118-132.
- [19] B. Sun et al., "Llumnix: Dynamic scheduling for large language model serving," in Proc. USENIX Symp. Operating Systems Design and Implementation (OSDI), 2024, pp. 173-191.
- [20] R. Prabhu, A. Nayak, J. Mohan, R. Ramjee, and A. Panwar, "vAttention: Dynamic memory management for serving LLMs without PagedAttention," in Proc. ACM Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS), 2025, pp. 1133-1150.
- [21] R. Qin et al., "Mooncake: Trading more storage for less computation – A KVCache-centric architecture for serving LLM chatbot," in Proc. USENIX Conf. File and Storage Technologies (FAST), 2025, pp. 155-170.
- [22] Y. Liu et al., "CacheGen: KV cache compression and streaming for fast large language model serving," in Proc. ACM SIGCOMM, 2024, pp. 38-56.
- [23] V. Srivatsa et al., "Preble: Efficient distributed prompt scheduling for LLM serving," in Proc. Int. Conf. Learn. Representations (ICLR), 2025.
- [24] B. Gao et al., "Cost-efficient large language model serving for multi-turn conversations with Cache-dAttention," in Proc. USENIX Annu. Tech. Conf. (ATC), 2024, pp. 111-126.
- [25] T. Ouyang, Z. Zhou, and X. Chen, "Follow Me at the Edge: Mobility-aware dynamic service placement for mobile edge computing," IEEE J. Sel. Areas Commun., vol. 36, no. 10, pp. 2333-2345, Oct. 2018.
- [26] Y. Sheng et al., "Fairness in serving large language models," in Proc. USENIX Symp. Operating Systems Design and Implementation (OSDI), 2024, pp. 965-988.
- [27] S. Ye, B. Ouyang, L. Zeng, T. Qian, X. Chu, J. Tang, and X. Chen, "Jupiter: Fast and Resource-Efficient Collaborative Inference of Generative LLMs on Edge Devices," in Proc. IEEE INFOCOM, 2025.
- [28] B. Zhang, J. Zhang, J. Hou, and Y. Wang, "TensAllo: Adaptive Deployment of LLMs on Resource-Constrained Heterogeneous Edge Devices," in Proc. IEEE INFOCOM, 2025.

- [29] M. Zhang, J. Cao, X. Shen, and Z. Cui, "EdgeShard: Efficient LLM Inference via Collaborative Edge Computing," *IEEE Internet of Things Journal*, 2025.
- [30] S. Ye et al., "Resource-Efficient Collaborative Edge Transformer Inference With Hybrid Model Parallelism," *IEEE Transactions on Mobile Computing*, vol. 24, no. 10, pp. 10945-10962, 2025.
- [31] J. Du, Y. Wei, S. Ye, J. Jiang, X. Chen, D. Huang, and Y. Lu, "Co-Designing Transformer Architectures for Distributed Inference With Low Communication," *IEEE Transactions on Parallel and Distributed Systems*, vol. 36, no. 4, 2025.
- [32] X. Feng et al., "E2LLM: Structure-Guided Efficient Inference for LLMs in Distributed Edge-IoT Environments," *IEEE Transactions on Mobile Computing*, 2026.
- [33] Y. Lin, S. Peng, C. Lu, C. Xu, and K. Ye, "FlexPipe: Adapting Dynamic LLM Serving Through Inflight Pipeline Refactoring in Fragmented Serverless Clusters," in *Proc. EuroSys*, 2026.
- [34] G. Qu, T. Li, Q. Chen, X. Chen, and S. Zhou, "SLIDE: Simultaneous Model Downloading and Inference at the Wireless Network Edge," *IEEE Transactions on Mobile Computing*, 2026.
- [35] D. Xu et al., "EdgeLLM: Fast On-Device LLM Inference With Speculative Decoding," *IEEE Transactions on Mobile Computing*, vol. 24, no. 4, pp. 3256-3271, 2025.
- [36] F. Chen, P. Li, T. H. Luan, Z. Su, and J. Deng, "SPIN: Accelerating Large Language Model Inference with Heterogeneous Speculative Models," in *Proc. IEEE INFOCOM*, 2025.
- [37] X. Miao et al., "SpecInfer: Accelerating Large Language Model Serving with Tree-based Speculative Inference and Verification," in *Proc. ACM Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2024.
- [38] R. Kong et al., "SwapMoE: Serving Off-the-shelf MoE-based Large Language Models with Tunable Memory Budget," in *Proc. Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2024.
- [39] H. Yu, X. Cui, H. Zhang, H. Wang, and H. Wang, "Taming Latency-Memory Trade-Off in MoE-Based LLM Serving via Fine-Grained Expert Offloading," in *Proc. EuroSys*, 2026.
- [40] L. Xue, Y. Fu, Z. Lu, C. Sun, L. Mai, and M. Marina, "MoE-Infinity: Efficient MoE Inference on Personal Machines with Sparsity-Aware Expert Cache," in *Proc. Int. Conf. Machine Learning (ICML)*, 2025.
- [41] R. Hwang et al., "Pre-gated MoE: An Algorithm-System Co-Design for Fast and Scalable MoE Inference," in *Proc. ACM/IEEE Int. Symp. Computer Architecture (ISCA)*, 2024.
- [42] R. Yi, L. Guo, S. Wei, A. Zhou, S. Wang, and M. Xu, "EdgeMoE: Empowering Sparse Large Language Models on Mobile Devices," *IEEE Transactions on Mobile Computing*, 2025.
- [43] N. Xue et al., "WDMoE: Wireless Distributed Mixture of Experts for Large Language Models,"

IEEE Transactions on Wireless Communications, 2025.

[44] R. Lu, Y. Liu, and D. Wang, "P4LLM: Protecting Embedding Privacy against Model Inversion Attacks for EaaS LLM Serving via Token-wise Partition and Perturbation," IEEE Transactions on Mobile Computing, 2026.

[45] W. Zhang, Z. Wu, Y. Mu, R. Ning, B. Liu, N. Sarda, M. Lee, and F. Lai, "JITServe: SLO-aware LLM Serving with Imprecise Request Information," in Proc. USENIX Symp. Networked Systems Design and Implementation (NSDI), 2026.

[46] K. Hong et al., "SOLA: Optimizing SLO Attainment for Large Language Model Serving with State-Aware Scheduling," in Proc. Conf. Machine Learning and Systems (MLSys), 2025.

[47] A. Patke et al., "One Queue Is All You Need: Resolving Head-of-Line Blocking in Large Language Model Serving," in Proc. Workshop on Cloud Intelligence/AIOps (AIOps@ASPLOS), 2024.

[48] N. Ling, G. Chen, and L. Zhong, "TimelyLLM: Segmented LLM Serving System for Time-sensitive Robotic Applications," in Proc. ACM MobiSys, 2026.

[49] B. Wu et al., "FastServe: Iteration-Level Preemptive Scheduling for Large Language Model Inference," in Proc. USENIX Symp. Networked Systems Design and Implementation (NSDI), 2026.

[50] J. Chen et al., "TokenFlow: Responsive LLM Text Streaming Serving under Request Burst via Preemptive Scheduling," in Proc. EuroSys, 2026.

[51] K. Zhu et al., "NanoFlow: Towards Optimal Large Language Model Serving Throughput," in Proc. USENIX Symp. Operating Systems Design and Implementation (OSDI), 2025.

[52] C. Ruan et al., "Libra: Flexible Request Partitioning and Scheduling for Serving Unbalanced and Dynamic LLM Workloads," in Proc. USENIX Symp. Networked Systems Design and Implementation (NSDI), 2026.

[53] Y. Shen, Y. Zhang, and D. Yuan, "A Hybrid Online and Offline Requests Inference Serving System for LLM in Private Computer Environment," IEEE Transactions on Computers, vol. 75, no. 3, 2026.

[54] Z. Li et al., "AdaServe: Accelerating Multi-SLO LLM Serving with SLO-Customized Speculative Decoding," in Proc. EuroSys, 2026.

[55] J. Yang, Q. Wu, Z. Feng, Z. Zhou, D. Guo, and X. Chen, "Quality-of-Service Aware LLM Routing for Edge Computing with Multiple Experts," IEEE Transactions on Mobile Computing, 2025.

[56] Y. Li, J. Guo, Z. Tang, X. Ding, J. Wang, T. Wang, and W. Jia, "Cloud-Edge System for Scheduling Unpredictable LLM Requests With Combinatorial Bandit," IEEE Transactions on Services Computing, vol. 18, no. 6, pp. 3567-3580, 2025.

[57] I. Ong et al., "RouteLLM: Learning to Route LLMs from Preference Data," in Proc. Int. Conf.

Learn. Representations (ICLR), 2025.

[58] F. Wu and S. Silwal, "PORT: Efficient Training-Free Online Routing for High-Volume Multi-LLM Serving," in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), 2025.

[59] Y. He, J. Fang, F. R. Yu, and V. C. Leung, "Large Language Models (LLMs) Inference Offloading and Resource Allocation in Cloud-Edge Computing: An Active Inference Approach," IEEE Transactions on Mobile Computing, vol. 23, no. 12, pp. 11253-11264, 2024.

[60] X. Xu, G. Feng, Y. Liu, S. Qin, J. Wang, and Y. Wang, "Joint Inference Offloading and Model Caching for Small and Large Language Model Collaboration," IEEE Transactions on Mobile Computing, vol. 25, no. 2, 2026.

[61] A. Ghosh, N. Sharma, S. Mishra, R. Misra, and S. K. Das, "MGCO: Mobility-Aware Generative Computation Offloading in Edge-Cloud Systems," IEEE Transactions on Services Computing, vol. 19, no. 1, pp. 477-490, 2026.

[62] Y. Yue et al., "MasRouter: Learning to Route LLMs for Multi-Agent Systems," in Proc. Annu. Meeting Assoc. Comput. Linguistics (ACL), 2025.

[63] S. Nielsen, E. Cetin, P. Schwendeman, Q. Sun, J. Xu, and Y. Tang, "Learning to Orchestrate Agents in Natural Language with the Conductor," in Proc. Int. Conf. Learn. Representations (ICLR), 2026.

[64] J. Zhang et al., "AFlow: Automating Agentic Workflow Generation," in Proc. Int. Conf. Learn. Representations (ICLR), 2025.

[65] X. Chen, J. Cao, Y. Sahni, M. Zhang, Z. Liang, and L. Yang, "Mobility-Aware Dependent Task Offloading in Edge Computing: A Digital Twin-Assisted Reinforcement Learning Approach," IEEE Transactions on Mobile Computing, vol. 24, no. 4, pp. 2979-2994, 2025.

[66] J. Jiang et al., "Efficient KV Cache Spillover Management on Memory-Constrained GPU for LLM Inference," IEEE Transactions on Parallel and Distributed Systems, vol. 37, no. 1, 2026.

[67] S. Gao, Q. Wang, S. Zeng, Y. Lu, and J. Shu, "Weaver: Efficient Multi-LLM Serving with Attention Offloading," in Proc. USENIX Annu. Tech. Conf. (ATC), 2025, pp. 587-595.

[68] M. Xu, D. Niyato, and C. G. Brinton, "Serving Long-Context LLMs at the Mobile Edge: Test-Time Reinforcement Learning-Based Model Caching and Inference Offloading," IEEE/ACM Transactions on Networking, vol. 34, pp. 3808-3823, 2026.

[69] J. Wang et al., "Context-Aware AIGC Service Migration in Edge Intelligence Networks via Transformer DRL," IEEE Transactions on Services Computing, vol. 19, no. 2, pp. 1020-1033, 2026.

[70] LMDeploy Team, "LMDeploy documentation," 2026. [Online]. Available: <https://lmdeploy.readthedocs.io/>

- [71] A. Du, J. Jia, S. Dustdar, J. Chen, and X. Wang, "Online Service Placement, Task Scheduling, and Resource Allocation in Hierarchical Collaborative MEC Systems," *IEEE Transactions on Services Computing*, vol. 18, no. 2, pp. 983-997, 2025.
- [72] X. Zhuang, J. Wu, H. Wu, T. Zhang, and L. Gao, "Joint Optimization of Model Inferencing and Task Offloading for MEC-Empowered Large Vision Model Services," in *Proc. IEEE INFOCOM*, 2025.
- [73] Z. Zhou et al., "A Survey on Efficient Inference for Large Language Models," *arXiv:2404.14294*, 2024.
- [74] R. Zhen et al., "Taming the Titans: A Survey of Efficient LLM Inference Serving," in *Proc. Int. Natural Language Generation Conf. (INLG)*, 2025, pp. 522-541.
- [75] J. Pan and G. Li, "A Survey of LLM Inference Systems," *arXiv:2506.21901*, 2025.
- [76] 华为, 《边缘 AI 计算技术合作项目资料》, 2026.
- [77] vLLM Ascend Team, "vLLM Ascend documentation," 2026. [Online]. Available: <https://docs.vllm.ai/projects/ascend/en/latest/>
- [78] vLLM Ascend Team, "vLLM Ascend installation guide," 2026. [Online]. Available: <https://docs.vllm.ai/projects/ascend/en/latest/installation.html>
- [79] vLLM Ascend Team, "Supported features," 2026. [Online]. Available: https://docs.vllm.ai/projects/ascend/en/latest/user_guide/feature_guide
- [80] vLLM Ascend Team, "KV Cache Pool," 2026. [Online]. Available: https://docs.vllm.ai/projects/ascend/en/latest/user_guide/feature_guide/kv_pool.html
- [81] vLLM Ascend Team, "Speculative decoding," 2026. [Online]. Available: https://docs.vllm.ai/projects/ascend/en/latest/user_guide/feature_guide/speculative_decoding.html
- [82] Huawei Ascend, "MindIE," 2026. [Online]. Available: <https://www.hiascend.com/en/developer/software/mindie>
- [83] vLLM Project, "OpenAI-compatible server," 2026. [Online]. Available: https://docs.vllm.ai/en/latest/serving/openai_compatible_server.html
- [84] L. Chen, Z. Ye, Y. Wu, D. Zhuo, L. Ceze, and A. Krishnamurthy, "Punica: Multi-Tenant LoRA Serving," in *Proc. Conf. Machine Learning and Systems (MLSys)*, 2024.
- [85] Y. Sheng et al., "SLoRA: Scalable Serving of Thousands of LoRA Adapters," in *Proc. Conf. Machine Learning and Systems (MLSys)*, 2024.
- [86] J. Duan, R. Lu, H. Duanmu, X. Li, X. Zhang, D. Lin, I. Stoica, and H. Zhang, "MuxServe: Flexible Spatial-Temporal Multiplexing for Multiple LLM Serving," in *Proc. Int. Conf. Machine Learning (ICML)*, 2024, pp. 11905-11917.

- [87] Y. He, H. Yang, Y. Lu, A. Klimovic, and G. Alonso, "Resource Multiplexing in Tuning and Serving Large Language Models," in Proc. USENIX Annu. Tech. Conf. (ATC), 2025, pp. 1639-1655.
- [88] G. Oliaro, X. Miao, X. Cheng, V. Kada, M. Wu, R. Gao, Y. Huang, R. Delacourt, A. Yang, Y. Wang, C. Unger, and Z. Jia, "FlexLLM: Token-Level Co-Serving of LLM Inference and Finetuning with SLO Guarantees," in Proc. USENIX Symp. Networked Systems Design and Implementation (NSDI), 2026, pp. 1359-1379.